ФСП Карьера

Сопроводительная документация к решению

Хакатон ЛЦТ · спецтрек ФСП · октябрь 2026

Содержание

О документе

1. Архитектура решения

1.1. Функциональная архитектура
1.2. Компонентная архитектура
1.3. Модель данных
1.4. Нефункциональные требования

2. Механика тестирования кандидатов

2.1. Коротко
2.2. Путь кандидата
2.3. Банк заданий
2.4. Адаптивный тест на IRT
2.5. Присвоение и изменение грейда
2.6. Устойчивость к распространению заданий
2.7. Симуляция LLM-читера
2.8. Качество теста
2.9. Как из описания вакансии формируется тест
2.10. Регулярные короткие задания от работодателей
2.11. Ограничения и план развития

3. Механика подбора и логика категоризации

3.1. Категория кандидата
3.2. Потребность работодателя: от текста к структуре
3.3. Ранжирование подборки
3.4. Объяснимость, анонимная карточка и поиск по банку
3.5. Выход на контакт: приглашение-оффер
3.6. Квалифицированная лотерея
3.7. Вакансии и отклики (дополнительный сценарий)
3.8. Регуляторы рынка и зачем они нужны
3.9. Кандидат без истории ФСП
3.10. Ограничения MVP

4. Валидация решения

Синтетические данные
Набор пар «вакансия — кандидат» и эталон релевантности
Метрики подбора
Симуляция рынка
Валидация теста
Как воспроизвести
Ограничения

5. Интеграция с реестром ФСП

Принципы
Модель данных
Влияние на ранжирование и показ работодателю
Каналы поступления данных
Согласие на внешние данные
Целевая схема интеграции с реальным реестром

Ограничения MVP

6. Описание API

Спецификация OpenAPI
Аутентификация
Общие коды ошибок
Методы по группам
Примеры ключевых методов
Ограничения

7. Используемые библиотеки и компоненты

7.1. Бэкенд (Python)
7.2. Фронтенд
7.3. Шрифты PDF-профиля (бэкенд)
7.4. Инфраструктура: контейнеры
7.5. Необязательные модели (LLM, эмбеддинги, OCR)
7.6. CI (GitHub Actions)
7.7. Лицензионная чистота и открытые вопросы

8. Сборка, развёртывание и локальный запуск

8.1. Что разворачивается
8.2. Требования
8.3. Быстрый старт (стенд для проверки)
8.4. Песочница задач на код
8.5. Переменные окружения
8.6. Боевое развёртывание
8.7. Локальная разработка без Docker
8.8. Тесты
8.9. Типовые проблемы и решения

9. Безопасность и 152-ФЗ

9.1. Согласия кандидата
9.2. Что видит работодатель
9.3. Права субъекта ПДн: выгрузка и удаление
9.4. Хранение секретов
9.5. Роли и разграничение доступа
9.6. ПДн не в логах и не в LLM
9.7. Песочница для кода кандидатов
9.8. Демо-режим — только явно
9.9. Известные ограничения

10. ML и LLM в решении

Провайдеры и модели
Где вызывается модель
Фолбэк: модель никогда не обязательна
Персональные данные и модель
LLM-читер как инструмент валидации
Как включить и проверить
Ограничения

11. Дополнительный функционал за рамками MVP: защита от недобросовестных работодателей и интеграция с ATS

11.1. Защита от фиктивных вакансий и недобросовестных работодателей: что уже работает

11.2. Защита: концепция развития
11.3. Интеграция с ATS: ключи API, лента событий и концепция вебхуков

О документе

ФСП Карьера — платформа обратного найма ИТ-специалистов. Кандидат один раз проходит адаптивный тест и получает категорию «специализация · трек · грейд». Работодатель описывает потребность, получает анонимную подборку с объяснением, почему кандидат в ней, и сам направляет приглашение с обязательной вилкой зарплаты. Контакты кандидата открываются только после того, как он принял приглашение.

Документ покрывает обязательный перечень ТЗ (п. 3.4) и вопросы, которые постановщики попросили раскрыть в письменных ответах.

Требование	Раздел
Функциональная и компонентная архитектура	1. Архитектура
Механика тестирования и её устойчивость к распространению заданий; механика повышения и пороги грейдов	2. Механика тестирования
Механика подбора и логика категоризации	3. Механика подбора
Процедура валидации и её результаты; собственный набор пар «вакансия — кандидат»	4. Валидация
Схема интеграции с реестром ФСП	5. Интеграция с ФСП
Описание API	6. API
Библиотеки и компоненты с версиями	7. Зависимости
Сборка, развёртывание и локальный запуск	8. Развёртывание
Безопасность и 152-ФЗ (п. 3.5)	9. Безопасность
Используемые модели ML и LLM	10. ML и LLM
Защита от фиктивных работодателей, интеграция с ATS (частично реализовано: ключи API, лента событий; вебхуки — концепция)	11. Концепции развития

Код открыт: репозиторий — исходники без сборки и обфускации. Все компании, люди и цифры в демо-данных синтетические. Что сделано и что осталось — `table.md` в корне репозитория; осознанные упрощения MVP отмечены в нём знаком ➖ с причиной.

1. Архитектура решения

«ФСП Карьера» — веб-платформа обратного найма ИТ-специалистов. Кандидат один раз проходит адаптивный тест и попадает в категорию «специализация · трек · грейд». Работодатель описывает потребность, получает анонимную подборку с объяснением и сам отправляет конкретному человеку приглашение с обязательной вилкой зарплаты. Контакты кандидата раскрываются только после его согласия.

Раздел описывает, что делает система (функциональная архитектура), из каких частей она собрана (компонентная архитектура), как устроены данные и что сделано для нефункциональных требований. Механики теста, подбора, валидации, интеграции с ФСП, API и безопасность подробно разобраны в разделах 2–10; здесь на них только ссылки.

1.1. Функциональная архитектура

Роли

Роль	Кто это	Как входит	Где работает
Кандидат	ИТ-специалист, ищет работу	код на почту или ФСП ID (OIDC)	кабинет `/candidate…`
Работодатель	рекрутер компании	код на почту или SSO через Keycloak	кабинет `/employer…`
Организатор	сотрудник ФСП или организатор соревнования	токен `ORGANIZER_TOKEN` из окружения или пользователь с ролью `admin`	страница `/org` (импорт протоколов)
Публичный доступ	жюри, любой посетитель	без входа	дашборд валидации `/v`, публичная проверка лотереи, публичный план теста по вакансии (только при `DEMO=1`)

Роль хранится в `users.role` (`candidate`, `employer`, `admin`). Кабинеты кандидата и компании — отдельные адреса: в одном браузере можно держать оба кабинета в соседних вкладках, у каждого свой токен.

Сквозной сценарий (ТЗ, п. 3.3)

Основной путь — выход на контакт по инициативе работодателя:

1. **Регистрация.** Кандидат вводит почту на `/candidate/signup` и получает шестизначный код (`POST /api/auth/email` → `POST /api/auth/verify`). Аккаунт создаётся только после подтверждения адреса. Альтернатива — вход через ФСП ID (раздел 5).
2. **Опрос.** Анкета профиля: специализация (12 направлений из `catalog.SPECS`), трек, предполагаемый грейд (`catalog.GRADES`: Intern, Junior, Middle, Senior, Lead), навыки, стаж, ожидания по зарплате, формат работы, город. Поля можно предзаполнить из PDF-резюме (`POST /api/candidate/cv`). Здесь же даются согласия 152-ФЗ (раздел 9). Сохранение — `POST /api/candidate/profile`.
3. **Тест.** Адаптивный тест на заявленный грейд (`POST /api/quest/start` → серия `POST /api/quest/answer` → `POST /api/quest/finish`): 6–12 заданий из банка специализации, оценка уровня по IRT. Есть экспресс-режим из 5 заданий, он даёт только предварительную категорию.

Механика — раздел 2.

4. **Категория.** По итогу теста кандидату присваивается грейд и с ним категория «специализация · трек · грейд». Без пройденного теста кандидат работодателю не виден нигде (`matching.visible`).
5. **Подборка или поиск.** Работодатель описывает потребность свободным текстом. Система разбирает текст в структуру (`POST /api/employer/needs/parse`), работодатель её проверяет и сохраняет (`POST /api/employer/needs`), после чего получает ранжированную подборку с объяснением (`GET /api/employer/needs/{id}/picks`). Другой путь — поиск по всему банку с фильтрами (`GET /api/employer/search`). Механика подбора — раздел 3.
6. **Приглашение.** Работодатель открывает анонимную карточку кандидата и отправляет приглашение: роль, описание, вилка «от–до», формат работы (`POST /api/employer/invites`). Действуют регуляторы: недельная квота компании, не больше 4 открытых приглашений у кандидата, срок ответа 7 дней.
7. **Принятие.** Кандидат видит приглашение с компанией, вилкой и контактом рекрутера ещё до ответа. Он принимает приглашение (`POST /api/candidate/invites/{id}/accept`) или отклоняет его с причиной.
8. **Контакты.** После принятия работодатель видит ФИО, почту и Telegram кандидата в статусах приглашений (`GET /api/employer/pipeline`), открывается внутренний чат. Раскрытие пишется в журнал `AuditLog` (`contacts_revealed`).

Дополнительные сценарии:

- **Отклик на вакансию.** Компания публикует вакансию с обязательной вилкой. Кандидат, прошедший тест, откликается сам (`POST /api/vacancies/{id}/apply`). До ответа компания видит только анонимную карточку. Контакты и чат открываются, когда компания приняла отклик. Так защищаемся от фиктивных вакансий, которые собирают контакты (раздел 11).
- **Задания работодателя.** Короткие задачи («решить» или «предложить подход») лежат на витрине для специализации или прикладываются к приглашению. Ответ с эталоном проверяет система (`services/autocheck.py`), открытый ответ оценивает работодатель. Оценка попадает в карточку кандидата как свежий сигнал, но на грейд и место в выдаче не влияет.
- **Чат.** Переписка кандидата и компании без обмена контактами вне платформы. Доступен, пока приглашение принято кандидатом или отклик принят компанией. Новые сообщения доставляются через SSE (`GET /api/chat/stream`).
- **Квалифицированная лотерея.** Работодатель задаёт порог требований. Пул прошедших порог замораживается, публикуется хеш секрета (`sha256(seed)`), затем бросается жребий. Результат может проверить кто угодно по публичному адресу `GET /api/lottery/{id}/public`. Механика — раздел 3.
- **Отзыв о компании.** По принятому приглашению кандидат может оставить отзыв. Отзывы меняют только квоту компании, а при сигнале «вакансии не было» блокируют её до ручной проверки (`services/reputation.py`, раздел 11).
- **Импорт протоколов.** Организатор загружает протокол соревнования ФСП, хакатона или CTF в формате CSV или JSON (`POST /api/achievements/import`). Достижения привязываются к кандидатам, которые дали согласие на внешние данные (раздел 5).

Карта функций по кабинетам

Кабинет кандидата (`/candidate…`): приглашения (главный экран), тест и его результат, профиль с резюме и PDF-профилем, вакансии и мои отклики, чат, задания, настройки (согласия, приватность, уведомления, ФСП ID, Codeforces и CTFtime, выгрузка и удаление данных).

Кабинет компании (`/employer…`): подборка по потребностям (главный экран), новая потребность, поиск по банку, карточка кандидата и форма приглашения, статусы приглашений, чат, вакансии и отклики, задания, данные компании.

Публичные страницы: главная с двумя входами («Кандидатам» и «Компаниям»), `/v` — дашборд валидации, `/org` — импорт протоколов организатором.

Соответствие функциональным требованиям ТЗ (п. 2.1 и 2.2)

Пути в таблице — от `backend/app/`, фронт — `frontend/index.html`.

Личный кабинет «Кандидат»

Требование ТЗ	Как реализовано	Где в коде
Профиль и резюме: форма или PDF с автораспознаванием (ФИО, контакты, стек, грейд, роли, стаж, софт-скиллы), PDF-профиль	Разбор PDF правилами по справочникам, LLM — по желанию (без неё всё работает). Распознаются контакты, навыки, стаж, подсказка грейда, роли, софт-скиллы, места работы. Сканы без текста распознаёт OCR, если он включён. Стандартизированный PDF-профиль генерируется на сервере	`services/parsing.py` (`parse_cv`), `services/ocr.py`, `services/profile_pdf.py`; `POST /api/candidate/cv`, `POST /api/candidate/profile`, `GET /api/candidate/profile.pdf`
Тестирование: опрос, выбор грейда, тест; результат определяет категорию	Анкета профиля (специализация, трек, заявленный грейд), тест на выбранный грейд (`target_grade`), адаптивный 2PL IRT; грейд и категория ставятся по итогу	`routers/quest.py`, `services/irt.py`, `services/items.py`, `services/bank/`
Регулярные задания от работодателя (желательно)	Витрина заданий специализации и задание в приглашении; автопроверка по эталону или оценка работодателя; сигнал в карточке	`routers/tasks.py`, `services/autocheck.py`
Категория и грейд: статус, история, пересдача выше или ниже с ограничением частоты	Текущая категория в кабинете, история попыток; повышение только новой попыткой и не чаще раза в 90 дней (`UPGRADE_COOLDOWN_DAYS`); принудительного понижения нет, нижний грейд кандидат принимает сам	`GET /api/candidate/me`, `GET /api/quest/history`, `POST /api/quest/accept-lower`, `catalog.UPGRADE_COOLDOWN_DAYS`
Приглашения: вакансия и зарплата, принять или отклонить	Список и карточка приглашения с вилкой, компанией, контактом рекрутера и объяснением «почему вас пригласили»; принятие, отказ с причиной	`routers/candidate.py` (`/invites…`)
Вакансии и отклик (желательно)	Список вакансий с фильтрами, отклик, список своих откликов со статусами, отзыв отклика	`routers/vacancies.py`
Настройки: приватность, согласия, связь с ФСП ID	Три раздельных согласия, статус «открыт / скрыт», «скрыть профиль от компаний» по ИНН, уведомления, привязка и отвязка ФСП ID	`PATCH /api/candidate/consents`, `/status`, `/hidden-from`, `/notif`; `routers/fsp_id.py`

Требование ТЗ	Как реализовано	Где в коде
Кандидат видит только свои приглашения и отклики	Все эндпоинты кабинета берут кандидата из токена (`deps.current_candidate`) и проверяют владельца записи (404 на чужую)	`deps.py`, `candidate._own_invite`, `vacancies.withdraw` и др.

Личный кабинет «Работодатель»

Требование ТЗ	Как реализовано	Где в коде
Профиль компании: описание, направление, контакты	ИНН (с проверкой контрольной суммы), название, описание, сайт, контакты рекрутера; автопроверки «почта на домене сайта»	`POST/GET /api/employer/company`
Описание потребности	Свободный текст → структура (специализация, трек, грейд, обязательные и желательные навыки, вилка, формат, город); категорию можно поправить; по потребности строится план теста	`POST /api/employer/needs/parse`, `/needs/category`, `/needs`; `POST /api/quest/plan`
Подборка: рекомендованные категории, кандидаты, обоснование	Ранжированный список анонимных карточек с текстом «почему в подборке» и разбивкой оценки; соседние категории со счётчиками	`GET /api/employer/needs/{id}/picks`, `GET /api/employer/candidates/{public_id}`; `services/matching.py`
Поиск по банку с фильтрами (специализация, грейд, категория, стек, достижения ФСП)	Вся база без привязки к потребности; фильтры по специализации, треку, грейду, навыкам, ФСП, Codeforces, хакатонам, формату, городу; фасеты	`GET /api/employer/search`
Выход на контакт: приглашение с предложением и зарплатой без обязательной вакансии; статусы	Приглашение из подборки, поиска или лотереи; потребность необязательна; статусы приглашений	`POST /api/employer/invites`, `GET /api/employer/pipeline`
Публикация вакансий, отклики (желательно)	Создание, правка, закрытие, продление (срок публикации 30 дней), отклики и ответ на них	`routers/vacancies.py`
Контакты не видны до принятия приглашения или отклика	Карточка — белый список полей без ФИО и контактов; контакты отдаются только по принятому приглашению или отклику и при действующем согласии	`matching.card`, `employer._row`, `vacancies._shown`

Поиск и подбор

Требование ТЗ	Как реализовано	Где в коде
Выдача — по категориям из теста, а не по резюме	Грейд категории берётся только из теста (`category_grade`); без теста кандидата нет в выдаче	`matching.tested`, `category_grade`, `visible`, `eligible`
Ранжирование по силе подтверждённого профиля и соответствию потребности, с объяснением	Оценка соответствия: θ с поправкой на погрешность, покрытие навыков, достижения; итог учитывает вероятность принятия и нагрузку кандидата; к каждой карточке — объяснение	`matching.score_candidates`, `matching.rank`; раздел 3

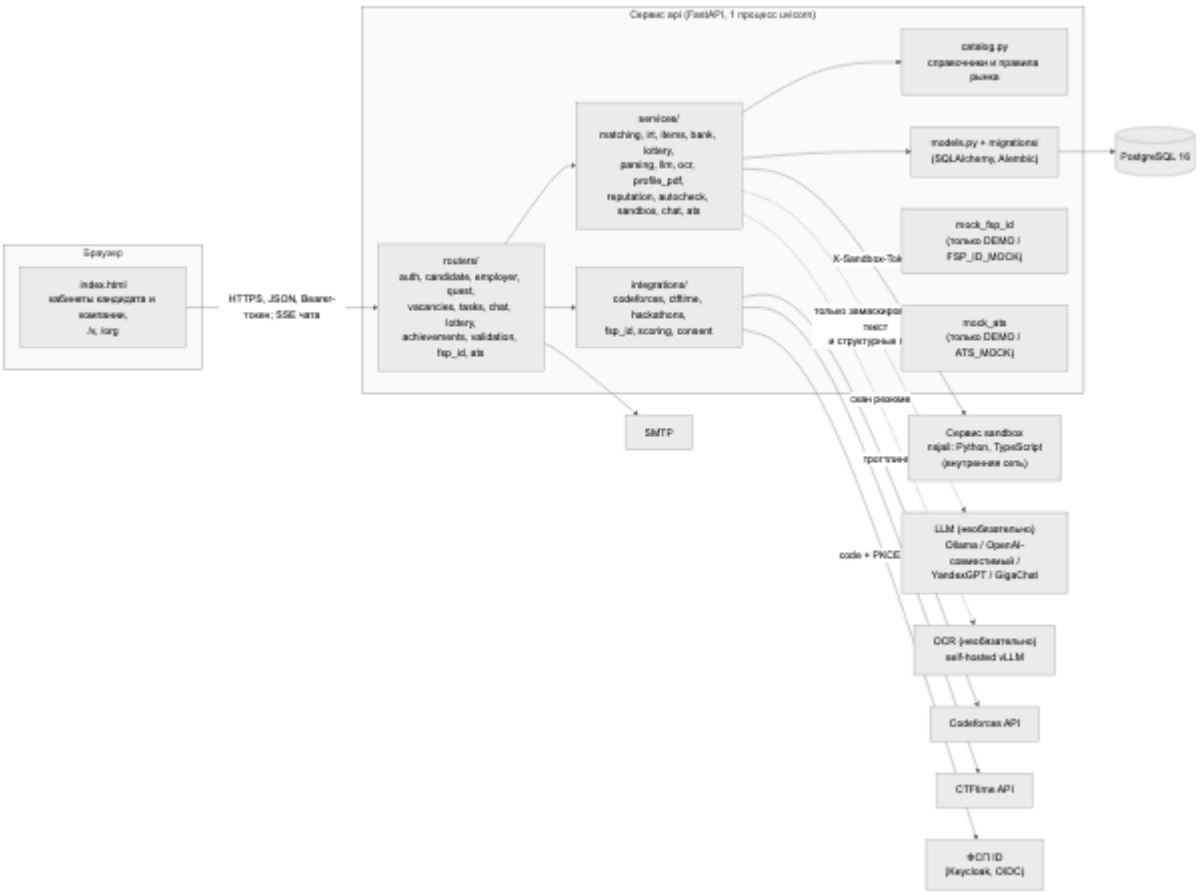
Требование ТЗ	Как реализовано	Где в коде
Фильтрация без потери подборки	Фильтры подборки (`filters`: формат, свободен, ФСП, Codeforces) сужают уже ранжированный список, порядок сохраняется	`employer.picks`

Данные и правила видимости (п. 2.2)

Правило ТЗ	Как реализовано
Вилка «от–до» обязательна в вакансии и приглашении	`employer.validate_fork`: обе границы, нижняя не ниже 25-го перцентиля рынка, ширина не больше 40 %; отдельно схема запроса ограничивает любую сумму `catalog.SALARY_MAX` (10 млн ₽ на руки, тип `employer.Salary`)
Приглашение содержит описание, зарплату, компанию и способ связи; контакты кандидата — после принятия или отклика	Контакт рекрутера виден кандидату сразу, контакты кандидата — после `accepted`
Статусы приглашений и откликов видят обе стороны	Приглашение: `sent → seen → accepted / declined / expired`; отклик: `sent → seen → accepted / declined`, кандидат может отозвать (`withdrawn`)
Регистрация по почте с подтверждением	Код на почту, пользователь создаётся только после проверки кода (`deps.send_code`, `deps.verify`)

1.2. Компонентная архитектура

Решение — модульный монолит с явным разделением слоёв. Фронт отделён от API, слой хранения — PostgreSQL. Исполнение чужого кода вынесено в отдельный изолированный сервис. Всё собирается `docker compose` из трёх сервисов: `db`, `api`, `sandbox`.



Фронтенд

Один файл `frontend/index.html` без сборки: разметка, стили по брендбуку ФСП и скрипт. Ассеты (шрифт JetBrains Mono, иконки, манифест) лежат в `frontend/assets/`. Страницу раздаёт тот же сервер, что и API, поэтому CORS не нужен. Наружу отдаются только `/` и `/assets` (`main.py`).

Маршрутизация — History API. Таблица `ROUTES` переводит адрес в ключ экрана и обратно. Основные адреса:

Кабинет	Адреса
Кандидат	`/candidate` (приглашения), `/candidate/login`, `/candidate/signup`, `/candidate/profile`, `/candidate/test`, `/candidate/test/run`, `/candidate/test/result`, `/candidate/invites/<id>`, `/candidate/vacancies[/<id>]`, `/candidate/applications`, `/candidate/tasks[/<id>[/<приглашение>]]`, `/candidate/chat[/<id>]`, `/candidate/settings`
Компания	`/employer` (подборка), `/employer/login`, `/employer/signup`, `/employer/company`, `/employer/needs/new`, `/employer/search`, `/employer/candidates/<id>[?need=<id>]`, `/employer/candidates/<id>/invite`, `/employer/invites`, `/employer/vacancies[/<id>]`, `/employer/tasks[/<id>]`, `/employer/chat[/<id>]`
Публичные	`/`, `/v`, `/org`

Кабинет без входа уводит на вход своей роли с `?next=` и после входа возвращает на исходный экран. Сервер отдаёт `index.html` только на явных префиксах `/candidate…`, `/employer…`, `/v`, `/org`, без перехвата всех адресов: опечатка в `/api/…` по-прежнему получает JSON-ответ 404.

Бэкенд

FastAPI, Python ≥ 3.12, синхронный SQLAlchemy 2.0 с драйвером psycopg. Слои:

Слой	Каталог	Назначение
Роутеры	`backend/app/routers/`	HTTP API по ролям и фичам: `candidate`, `employer`, `quest` (тест), `vacancies`, `tasks`, `chat`, `lottery`, `achievements` (внешние профили и импорт протоколов), `validation`, `fsp_id` (вход через ФСП ID), `mock_fsp_id` (мок провайдера); вход по коду — `deps.py`
Сервисы	`backend/app/services/`	Логика: подбор (`matching.py`), IRT (`irt.py`), банк заданий (`items.py` + сценарные банки специализаций `bank/`), задачи на код (`code_items.py`) и клиент песочницы (`sandbox.py`), лотерея, разбор резюме и потребностей (`parsing.py`), клиент LLM (`llm.py`), OCR, PDF-профиль, репутация компаний, автопроверка заданий, чат, валидация
Интеграции	`backend/app/integrations/`	Codeforces, CTFtime, импорт протоколов хакатонов и ФСП (`hackathons.py`), клиент ФСП ID (`fsp_id.py`), веса достижений (`scoring.py`), согласие на внешние данные (`consent.py`); общий HTTP-слой с троттлингом (`common.py`)
Справочники	`backend/app/catalog.py`	Специализации, треки, грейды и их пороги, навыки, компетенции, рынок зарплат, квоты, сроки, версия текста согласия
Данные	`backend/app/models.py`, `backend/migrations/`	Схема SQLAlchemy и ревизии Alembic. При старте API применяет все ревизии (`alembic upgrade heads`) под advisory-блокировкой Postgres и засевает банк заданий теста (`items.seed_items`)
Доступ	`backend/app/deps.py`	Токены, вход по коду, зависимости FastAPI для проверки роли (раздел 9)

Документация API генерируется FastAPI по OpenAPI и доступна на `/docs` (раздел 6).

Хранилище

PostgreSQL 16 (`postgres:16-alpine`), данные — в именованном томе `pgdata`. Эмбеддинги (если включена LLM) хранятся массивом `ARRAY(Float)` в строках кандидата и потребности, косинусное сходство считается в приложении (numpy) по пулу категории. Расширений Postgres (pgvector и т. п.) нет.

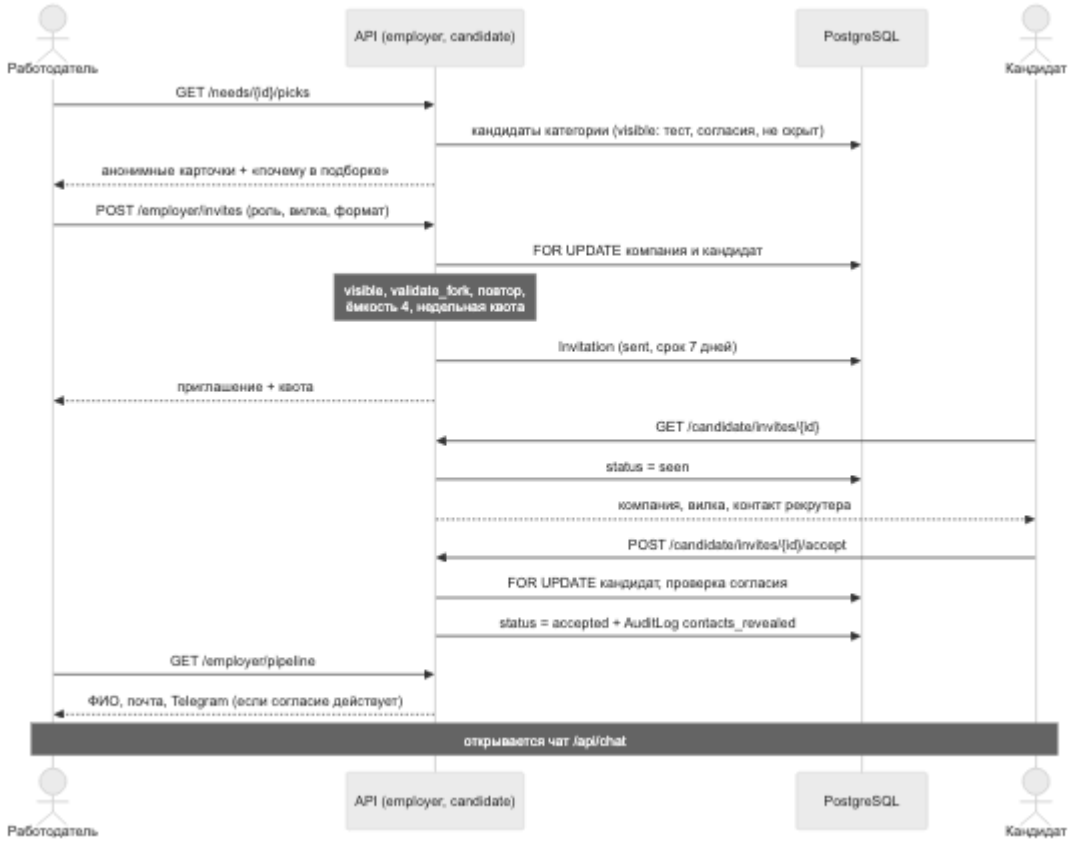
Песочница задач на код

Отдельный контейнер `sandbox` (`sandbox/`, HTTP-сервис на stdlib Python). Каждый тест решения запускается в своём nsjail: Python 3.11 или TypeScript на Node 22. Контейнер сидит во внутренней сети compose (`internal: true`), портов на хост и выхода наружу у него нет. С песочницей говорит только `api` по общему секрету `SANDBOX_TOKEN`. Без токена тест идёт без задач на код. Модель безопасности — раздел 9, механика задач — раздел 2.

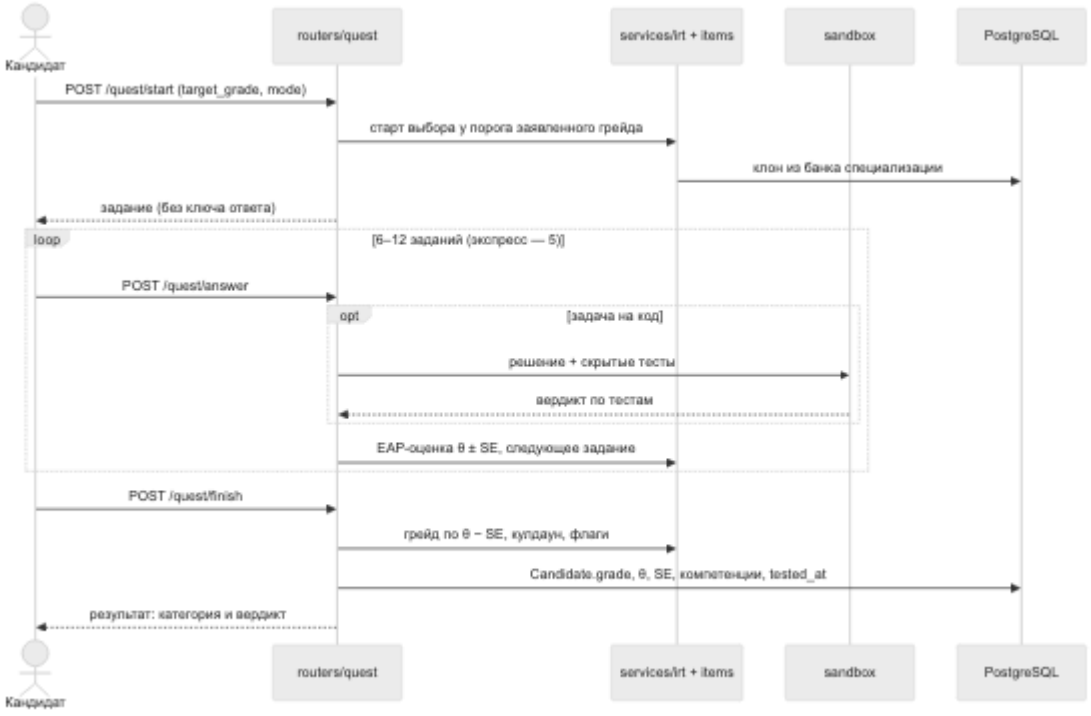
Внешние компоненты (все необязательны для запуска)

Компонент	Зачем	Без него
LLM (`LLM_PROVIDER`: `ollama`, `openai_compatible`, `yandexgpt`, `gigachat`)	разбор потребности и резюме, реранк топ-30 подборки, эмбеддинги	всё работает на правилах (`LLM_PROVIDER=none` по умолчанию); раздел 10
OCR (`OCR_URL`, self-hosted vLLM с vision-моделью)	распознавание сканов резюме без текстового слоя	скан → 422 «заполните поля вручную»
Codeforces, CTFtime	рейтинг и места в соревнованиях	достижений из этих источников нет, штрафа нет
ФСП ID (Keycloak OIDC)	вход и привязка ФСП ID, подтверждённые достижения из реестра	вход по коду почты; на стенде — встроенный мок `/mock-fsp-id`; раздел 5
SMTP	письма с кодом входа и уведомления о приглашениях	вне `DEMO=1` вход по почте отвечает 503; в демо код приходит в ответе

Поток: приглашение и раскрытие контактов



Поток: тест и присвоение категории



1.3. Модель данных

Схема описана в `backend/app/models.py`, изменения — только ревизиями Alembic (`backend/migrations/versions/`, корень `0001_baseline`).

Сущность (таблица)	Что хранит	Ключевые связи	ПДн
`User` (`users`)	аккаунт: почта входа, роль, sha256 токена и срок, `fsp_id`, флаг подтверждения почты кодом	1:1 с `Candidate` или `Company`	почта, ФСП ID
`Candidate` (`candidates`)	профиль: анонимный `public_id`, специализация, трек, заявленный грейд, ожидания, формат, город, навыки, роли, софт-скиллы, резюме; результат теста (грейд, θ, SE, компетенции, подтверждённые навыки, срок повышения); три согласия с версиями; статус и «скрыть от»	→ `User`; ← достижения, внешние аккаунты, приглашения, отклики, сессии теста, сообщения	**ФИО, почта, Telegram, текст и имя файла резюме**; город, ожидания по зарплате
`ExternalAccount`	привязка Codeforces или CTFtime: handle, sha256 кода подтверждения, рейтинг	→ `Candidate` (каскадное удаление)	handle
`Event`, `Achievement`	соревнование (ФСП, хакатон, CTF, Codeforces) и подтверждённое достижение: место, результат, разряд, вес	`Achievement` → `Candidate`, `Event`	косвенно (команда, роль)
`Company`	компания: ИНН, название, описание, сайт, контакты рекрутера, флаг проверки, квота, блок по отзывам	→ `User`	ФИО, почта, Telegram рекрутера
`Need`	потребность работодателя: текст и структура после разбора	→ `Company`	—
`Invitation`	приглашение: роль, вилка, формат, описание, задача, статус и даты, причина отказа, отметки сторон	→ `Company`, `Candidate`, `Need`, `Lottery`	комментарий к отказу
`Lottery`	жеребьёвка: порог, пул псевдонимов, хеш секрета, коммитмент, раскрытый секрет, выбранные	→ `Company`, `Need`	— (псевдонимы `sha256(salt + public_id)`)
`Item`, `QuestSession`	задание банка IRT (семейство, a, b, содержимое с ключом) и сессия теста (ответы, θ, SE, грейд попытки, флаги)	`QuestSession` → `Candidate`	ответы кандидата

Сущность (таблица)	Что хранит	Ключевые связи	ПДн
`Vacancy`, `Application`	вакансия с обязательной вилкой и сроком публикации; отклик со статусом и отметкой раскрытия контактов	`Application` → `Vacancy`, `Candidate`	заметка к отклику
`Task`, `TaskSubmission`	задание работодателя (эталон — только серверу и автору) и решение кандидата с оценкой	→ `Company`, `Candidate`, `Invitation`	текст ответа
`Message`	сообщение чата кандидат ↔ компания	→ `Company`, `Candidate`	текст переписки
`AuditLog`	журнал операций с ПДн: кто, что, о ком, когда; только добавление, без внешних ключей	идентификаторы	нет (только id и основания)
`ConsentEvent`	журнал согласий: цель, выдано или отозвано, версия текста; без внешнего ключа — переживает удаление	`candidate_id`	нет
`LoginChallenge`, `FspLogin`	ожидающий вход по коду (HMAC кода) и поток входа через ФСП ID (sha256 state, куки, билета)	`FspLogin` → `User`	почта в `LoginChallenge`
`RetiredPublicId`	`public_id` удалённых кандидатов, чтобы не выдавать их повторно	—	—

Принцип разделения: персональные данные кандидата (`name`, `email`, `tg`, резюме) живут только в `Candidate` и не участвуют в ранжировании. Анонимная карточка для работодателя собирается белым списком полей (`matching.card`). Поиск по банку даже не читает эти колонки из БД (`employer.SEARCH_SKIP`). Поле `truth` («скрытая правда» синтетики для валидации) в API не отдаётся.

1.4. Нефункциональные требования

Один процесс API и почему

API работает строго одним процессом uvicorn (`WEB_CONCURRENCY=1` в `backend/Dockerfile` и `docker-compose.yml`). В памяти процесса живут:

- подписчики SSE чата (`services/chat.Hub`) — уведомления «есть новое»;
- троттлинг запросов к Codeforces и CTFtime (`integrations/common.py`: не чаще раза в 2 с и 0,5 с к хосту);
- случайный ключ `SECRET_KEY`, если он не задан в окружении: на нём держатся HMAC кодов входа и общий демо-токен;
- блокировка «одна проверка кода на сессию» (`quest._one_check`) и кэш `/health` песочницы;
- блокировка одновременного пересчёта валидации;
- коды и ключи мок-провайдера ФСП ID.

Второй воркер сломал бы доставку сообщений чата, коды входа и лимиты. Запаса одного процесса хватает с избытком (см. ниже). Это осознанное упрощение MVP, путь масштабирования описан ниже.

Производительность

Нагрузочный прогон — `backend/loadtest/locustfile.py` (locust, команда запуска — в докстринге). 50 одновременных пользователей обоих кабинетов с паузой 1–3 с, полный сид (1500 кандидатов, 250 потребностей), один процесс uvicorn, 2 минуты:

Показатель	Значение
Ошибки	0 из 3424 запросов
Пропускная способность	~29 запросов в секунду
Подборка, p50 / p95 / p99	90 / 170 / 240 мс
Поиск по банку, p50 / p95 / p99	120 / 190 / 240 мс
Карточка кандидата, p50 / p95 / p99	29 / 99 / 140 мс
Все запросы, p50 / p95 / p99	27 / 140 / 190 мс
Предел одного процесса без пауз	~72 запроса в секунду, p95 1,2 с, 0 ошибок (запас ×2,5)

Ресурсы под этой нагрузкой: стек `docker compose` занимает ~1–1,5 ядра и до ~320 МБ памяти (api до 253 МиБ, db до 61 МиБ; в простое 119 и 34 МиБ). GPU не нужен. Песочница — отдельный контейнер с потолком 4 CPU / 3 ГБ, занята только во время проверки решений. Цифры сняты на машине разработчика (`table.md`, строка «Нагрузочная проверка»). Под требование ТЗ «десятки одновременных пользователей» запас есть.

Масштабирование

Модули слабо связаны и общаются через БД, поэтому при росте их можно выносить по частям:

Что	Сейчас	При росте
Несколько воркеров API	один процесс	постоянный `SECRET_KEY` из окружения; SSE чата — Redis pub/sub; троттлинг внешних API, `_one_check`, кэш песочницы и блокировка валидации — Redis или advisory-блокировки Postgres
Песочница	privileged-контейнер на том же хосте	отдельная ВМ или gVisor / Firecracker; горизонтально — несколько экземпляров за общим секретом
LLM и OCR	внешние по конфигурации	отдельный сервис инференса; контракт «`None` → правила» уже есть
Письма	фоном в том же процессе, без повторов	outbox-таблица и воркер с повтором
Поиск по эмбеддингам	numpy по пулу категории	pgvector или внешний векторный индекс (ревизией Alembic)
Ограничение частоты запросов	только по адресу почты для кода входа	rate limit на reverse proxy

Перечень — из строк `—` и `` в `table.md`. Для MVP он не нужен.

Адаптивность и мобильная версия

Вёрстка адаптивная: телефон, планшет и десктоп (ТЗ п. 3.5 делает это необязательным для MVP, критерии п. 6 считают плюсом). Что сделано:

- шапка в одну строку на ширине до 900 px: логотип, тема и круглая аватарка с меню профиля (у кандидата — «Тест», «Профиль», «Настройки и приватность», «Выйти»; при входе в оба кабинета — переключатель между ними); разделы кабинета прокручиваются строкой;
- таблицы на телефоне превращаются в карточки «подпись — значение»;
- фильтры и боковые панели сворачиваются;
- поля ввода на сенсорном экране 16 px (iOS не увеличивает страницу), клавиатура подбирается под поле (`inputmode`);
- зона нажатия не меньше 36 px;
- чат: плавающий кружок в правом нижнем углу (красная точка — есть непрочитанные) открывает мини-окно (на телефоне почти во весь экран), «Раскрыть» — полный экран: либо список диалогов, либо открытый диалог на всю высоту;
- в тесте шапка задания и кнопка «Дальше» прилипают к краям экрана;
- манифест и иконка: стенд добавляется на домашний экран как приложение.

Проверено e2e-сценарием `frontend/e2e/09-mobile.spec.js` на ширинах 320, 375, 768 px и на телефоне боком: горизонтальной прокрутки страницы нет нигде. Это эмуляция Chromium. На живых iOS Safari и Android Chrome вручную ещё не проверяли (`table.md`, ⬜).

Тёмная тема

Цвета заданы токенами на `:root`. Тема берётся из системной настройки (`prefers-color-scheme`), а переключатель в шапке задаёт её явно. Выбор хранится в `localStorage` и применяется до отрисовки страницы. Контраст текста не ниже 4,5:1 (WCAG AA). Сценарий — `frontend/e2e/10-theme.spec.js`. Ограничение: логотип в тёмной теме инвертируется CSS-фильтром, точного тёмного варианта ассета нет (`table.md`, ➖).

Рей — талисман теста

Джек-рассел-терьер Рей нарисован CSS из `span`-ов (`ray()` и стили `.ray*` в `index.html`); иконка вкладки — знак ФСП (`assets/favicon.svg`, тот же знак, что в иконке приложения `assets/icon.svg`, но без полей). Он сидит на вводной теста, бежит после отправки ответа, ложится и засыпает после долгой паузы без действий, радуется в конце теста и на результате, бежит в индикаторе загрузки экрана. Реакция не зависит от верности ответа: её проверяет сервер, фронт её не знает. У каждого состояния есть подпись для скринридера, при `prefers-reduced-motion` остаётся только поза. Сценарий — `frontend/e2e/10-ray.spec.js`.

Качество и проверяемость

- Покрытие тестами бэкенда и фронта — не ниже 80 % (бэкенд — 98 %, фронт — 86,0 % строк JS по README).
- Каждый набор бэкенд-тестов идёт в своей базе.
- CI (`.github/workflows/ci.yml`) прогоняет тесты бэкенда, самопроверки модулей, тесты песочницы против настоящего nsjail и синтаксис JS.

Подробности — разделы 7 и 8.

2. Механика тестирования кандидатов

Раздел описывает, как кандидат получает категорию «специализация · трек · грейд»: из чего состоит банк заданий, как адаптивный тест выбирает задания и оценивает уровень, по каким правилам присваивается, повышается и понижается грейд и почему тест устойчив к утечке заданий (ТЗ, разделы 2 и 3.4; критерий оценки № 2). Процедура валидации и все её результаты — в разделе 4. Здесь из неё приведены только цифры, которые нужны для аргументации. Как категория и θ участвуют в подборе — раздел 3.

2.1. Коротко

- **Тест обязателен и единственный задаёт категорию.** Заявленный грейд — только цель теста. Пока кандидат не прошёл тест, работодатели его не видят: ни в подборке, ни в поиске, ни в лотерее. Пригласить его нельзя, сам откликнуться он тоже не может (`matching.tested`, `matching.visible`).
- **Нет единого набора заданий.** Банк состоит из **параметрических семейств**. Семейство — шаблон сцены, который эксперт написал один раз. Из него детерминированно получаются 5–8 **клонов** с разными данными. У семейства 3–4 **варианта механики**: разные дефекты или разные причины одного симптома. Сложность (параметр b) задана на семейство, поэтому клоны сопоставимы по трудности, а утечка одного клона не раскрывает остальные.
- **Тест адаптивный.** Модель — 2PL IRT, оценка уровня — EAP, следующее задание выбирается по максимуму информации. Кандидаты одного уровня получают разные траектории: другие семейства, другие клоны, случайный выбор из трёх лучших. Всего в банке 1912 рабочих заданий 12 специализаций, в сессии — до 12.
- **Задачи на код исполняются в своей песочнице** (nsjail) на скрытых тестах. Тесты пересоздаются из сида клона и в базе не лежат.
- **Правильность не раскрывается.** Ни во время теста, ни после кандидат не узнаёт, какие ответы верны: ключ задания наружу не уходит. Набрать «базу ответов» по собственным попыткам нельзя.
- **Грейд присваивается осторожно и принудительно не понижается.** Правило: $\theta - SE \geq$ порога грейда. Повышение возможно не чаще раза в 90 дней. Попытка ниже текущего грейда в профиль не идёт, нижний грейд кандидат может принять только сам.
- **Античит.** Таймер с серверным временем, флаги несогласованности ответов (`inconsistent`), слишком быстрых ответов (`too_fast`) и вставленного кода (`code_pasted`). LLM-читер в симуляции получает Junior (раздел 2.7).

Где это в коде:

Что	Файл
IRT: вероятность ответа, EAP, выбор задания, остановка, флаги, правило грейда	`backend/app/services/irt.py`
Банк: семейства Backend, простые шаблоны, генерация клонов, засев, план теста под вакансию	`backend/app/services/items.py`
Сценарные семейства остальных специализаций	`backend/app/services/bank/*.py`
Задачи на код и скрытые тесты	`backend/app/services/code_items.py`, `backend/app/services/sandbox.py`, `sandbox/`
API теста: старт, ответ, завершение, история, принятие нижнего грейда, план под вакансию	`backend/app/routers/quest.py`

Что	Файл
Пороги грейдов, кулдаун, справочник компетенций	`backend/app/catalog.py`
Сброс теста при смене специализации	`backend/app/routers/candidate.py`, функция `_reset_test`
Регулярные задания работодателей и автопроверка	`backend/app/routers/tasks.py`, `backend/app/services/autocheck.py`

2.2. Путь кандидата

ТЗ требует обязательный путь: опрос по отрасли и специализации, выбор предполагаемого грейда, тест на заявленный грейд. Отрасль у платформы одна — ИТ; внутри неё справочник делит роли на восемь разработческих специализаций и четыре нетехнические (аналитика, управление, дизайн, администрирование и поддержка), поэтому «опрос по отрасли» сводится к выбору специализации и трека. В продукте путь выглядит так.

1. **Профиль (опрос).** На экране профиля кандидат выбирает специализацию из справочника `catalog.SPECS` (12 направлений, см. 2.3), трек (язык, стек или подроль из `catalog.TRACKS`) и заявленный грейд (`grade_claimed`: Intern, Junior, Middle, Senior или Lead). Там же указываются навыки, стаж и ожидания. Навыки можно заполнить вручную или распознать из PDF-резюме. Без согласия на обработку данных профиль не сохраняется.
2. **Выбор грейда теста.** На стартовом экране теста есть переключатель «На какой грейд проходите тест». По умолчанию выбран заявленный грейд, его можно сменить на любой другой (`target_grade` в `POST /api/quest/start`). Грейд теста определяет только **точку старта** адаптивного выбора и вердикт «подтверждён / ниже / выше заявленного». На итоговую оценку он не влияет: итог считается с нейтральным априором (2.4).
3. **Тест.** Кандидат выбирает полный тест или экспресс-тест. Задания идут сценами сюжета специализации. Сквозной герой всех сюжетов — Рей, джек-рассел-терьер кандидата, а у каждой специализации своя история с ним: например, «Где Рей?» у Backend (Рей сбежал, а сервис его GPS-ошейника лёг) или «Переезд до прививки» у DevOps / SRE (сеть ветклиник переезжает в облако в ночь перед прививкой Рея). Рабочие задачи вырастают из этой истории, но условие задания от сюжета не зависит (`items.STORY`, обёртка — поля `scene_title` и `story` в банках). Сюжет можно выключить, задания при этом остаются теми же.
4. **Результат.** Кандидат получает категорию «специализация · трек · грейд», θ ± SE, профиль по шести компетенциям с сильными и слабыми сторонами, перцентиль в категории, подтверждённые тестом навыки, вердикт относительно заявленного грейда и дату, с которой возможно повышение (`GET /api/quest/result`). Кандидат видит все свои попытки (`GET /api/quest/history`): даты, режим, θ, SE, грейд и пометку, применена ли попытка.

Полный и экспресс-тест

	Полный тест (`mode: "full"`)	Экспресс-тест (`mode: "express"`)
Назначение	Присвоение и подтверждение грейда	Знакомство за 3–5 минут, например для жюри
Число заданий	От 6 до 12 (`irt.MIN_ITEMS`, `irt.MAX_ITEMS`), остановка по SE < 0,25 (`irt.SE_STOP`)	Ровно 5 (`irt.EXPRESS_ITEMS`), без остановки по SE

	Полный тест (`mode: "full"`)	Экспресс-тест (`mode: "express"`)
Типы заданий	Все: вопросы, разметка строки кода, ревью, одна задача на код	Только вопросы и разметка строки (`irt.EXPRESS_KINDS` = `choice`, `mark`)
Время на задание	Из банка: 240 с на вопрос, 300 с на разметку, 360 с на ревью, 900 с на задачу на код (`items.TIME_LIMIT`)	60 с на любое задание (`quest.EXPRESS_TIME_LIMIT_S`)
Пилотное задание	Есть, если в банке специализации есть пилотные (2.4)	Нет
Точность	Средняя SE 0,32–0,33 (README «Результаты валидации»)	SE ≈ 0,47; грейд выше Middle почти не присваивается (README)
Что даёт	Грейд, кулдаун повышения, подтверждённые тестом навыки	Только **предварительную категорию** с пометкой «экспресс-оценка» (`Candidate.grade_express`)

Правила экспресс-теста (`quest.finish`):

- Он ставит грейд, только если у кандидата грейда ещё нет. Существующий грейд он не меняет ни вверх, ни вниз, в том числе свой прошлый экспресс-грейд: попытка уходит в историю с `applied = False` и пояснением.
- Кулдаун повышения он не запускает.
- Навыков он не подтверждает: по одному заданию на компетенцию этого делать нельзя, поэтому в карточке остаются только заявленные навыки.
- Полный тест после экспресса считается **первым присвоением**: результат применяется сразу, и с него начинается кулдаун.
- Работодатель видит кандидата с экспресс-оценкой с пометкой «экспресс-оценка». В подборе θ штрафуется на величину, пропорциональную SE, поэтому экспресс-кандидат стоит заметно ниже (раздел 3).

Экспресс-тест введён для знакомства с продуктом. ТЗ требует, чтобы категорию определял тест, поэтому экспресс даёт только предварительную, явно помеченную категорию. Его правило грейда то же (θ − SE ≥ порога), но при большой SE оно осторожнее. Самопроверка `python -m app.services.irt` следит, чтобы для каждой специализации экспресс завышал грейд не чаще полного теста (с допуском 3 п. п.).

2.3. Банк заданий

Специализации и компетенции

Справочник специализаций составлен по общепринятым в ИТ ролям (ТЗ разрешает команде сделать это самостоятельно). В нём восемь разработческих направлений и четыре нетехнические ИТ-роли. У каждой специализации шесть компетенций (`catalog.COMPETENCIES`). Тест оценивает каждую из них отдельно, по ним же строится профиль кандидата.

Специализация	Компетенции
Backend	Диагностика инцидентов, Базы данных и транзакции, Ревью кода, Проектирование API, Конкурентность, Алгоритмы
Frontend	Отладка UI, Производительность, Ревью кода, Архитектура клиента, Доступность, Алгоритмы

Специализация	Компетенции
Mobile	Отладка, Жизненный цикл приложения, Ревью кода, Сеть и офлайн, Производительность, Алгоритмы
ML / DS	Постановка задачи, Валидация моделей, Дрейф данных, Feature engineering, MLOps, Алгоритмы
Data Eng / Analytics	SQL, Моделирование данных, Пайплайны, Качество данных, Визуализация, Алгоритмы
DevOps / SRE	Диагностика инцидентов, Сети, Kubernetes, CI/CD, Наблюдаемость, План отката
QA	Тест-дизайн, Приёмка релиза, Автотесты, API-тестирование, Баг-репорты, Регресс
Security	Веб-уязвимости, Ревью кода, Сети, Криптография, Реагирование, Реверс
Analysis (системный и бизнес-аналитик)	Выявление требований, Моделирование процессов, Интеграции и API, Данные и SQL, Спецификации, Анализ изменений
Management (project и product)	Планирование и сроки, Риски, Метрики продукта, Приоритизация, Стейкхолдеры, Процессы команды
Design (UX/UI и продуктовый)	Исследования, Юзабилити, Доступность, Дизайн-системы, Прототипирование, Метрики и эксперименты
SysAdmin / Support	Диагностика инцидентов, Сети, Учётные записи и доступ, Резервное копирование, Серверы и ОС, Работа с обращениями

Типы заданий

Тип (`kind`)	Что делает кандидат	Как проверяется	Лимит времени
`choice` — вопрос с вариантами	Читает ситуацию и, как правило, панель с данными (логи, график, код), выбирает один из четырёх вариантов	Совпадение с ключом	240 с
`mark` — разметка кода	Отмечает в листинге строку с дефектом	Номер строки входит в список допустимых	300 с
`review` — ревью кода	Отмечает строку с дефектом и пишет комментарий	Номер строки; комментарий сохраняется, но на оценку пока не влияет	360 с
`code` — задача на код	Пишет функцию на Python или TypeScript, может запускать её на открытых примерах	Исполнение в песочнице на 10–15 скрытых тестах, засчитано только при прохождении всех	900 с

Задания — это рабочие ситуации, а не определения из учебника. Сцена показывает панель данных, которую фронт рисует сам (`payload.panel`): ленту логов, график с метками или листинг кода. Кандидат ставит диагноз по этим данным, как на дежурстве или ревью. Например, в семействе `be_inc_env` выкладка застряла, а в логах в зависимости от варианта видно одно из трёх: `KeyError` по переменной окружения, `OOMKilled` или `ImagePullBackOff`. Кандидат должен назвать причину.

Семейства, варианты и клоны

Семейство — функция-шаблон `render(rng)`, зарегистрированная декоратором `items.family` (или `items.simple` для простых вопросов). Семейство привязано к специализации, компетенции и типу задания и несёт экспертные параметры IRT: различающую способность a и сложность b.

Вариант механики (`items._variant`) — одна из 3–4 разных ситуаций внутри семейства: другой дефект, другая причина того же симптома, другая деталь условия. Клоны распределяются по вариантам по кругу (номер клона mod число вариантов). Правильные ответы других вариантов попадают в дистракторы клона (`items._options`). Поэтому ответ, запомненный по одному клону, на клоне другого варианта неверен, причём он прямо стоит среди вариантов и выглядит правдоподобно.

Клон — результат `render` с детерминированным сидом «семейство:номер клона» (`items.generate_bank`). Клоны различаются вариантом механики, подставленными данными (имена сервисов, версии, числа, время в логах, место дефекта в коде) и порядком вариантов ответа. Если клон совпал по содержанию с уже выпавшим, он перегенерируется с другим сидом. Число клонов на семейство: 8 у Backend (`items.CLONES`), 5 у остальных специализаций (`items.CLONES_DEFAULT`), 4 у пилотных семейств (`items.CLONES_PILOT`).

Почему сложность клонов сопоставима. Параметры клона наследуются от семейства с малым шумом: b — ±0,1, a — ±5 % (`generate_bank: b + uniform(-0,1, 0,1)`, `a × uniform(0,95, 1,05)`). Разброс b клонов внутри семейства в валидации — около 0,05 (README). Сопоставимость задаётся конструкцией: клоны одного семейства проверяют одно умение на одинаковом по объёму материале и отличаются данными и конкретным дефектом. В этом и состоит отличие от генерации нейросетью, у которой сложность каждого задания неизвестна. Честная оговорка: варианты механики по трудности не строго равны, и при проверке банка отмечено несколько семейств, где один вариант проще (`table.md`, строка «Варианты механики меняют сложность клона»). Разброс 0,05 заложен в генерацию, а не измерен на живых ответах. План — калибровать b отдельно по варианту вместе с общей калибровкой (2.11).

Обновление банка. Банк засевается при каждом старте API (`items.seed_items`, под advisory-блокировкой Postgres). Пустой банк засевается целиком. В засеянный добавляются новые семейства, а изменённые клоны обновляются на месте. id задания не меняется, потому что на него ссылаются ответы в истории.

Состав банка

Подсчёт по `items.FAMILIES` и `items.generate_bank()` на текущем коде:

Специализация	Рабочих семейств без задач на код (вопросы / разметка / ревью)	Семейств задач на код	Пилотных семейств	Клонов на семейство	Заданий в банке (с пилотными)
Backend	29 (20 / 7 / 2)	5	2 сценарных + 1 на код	8	284
Frontend	28 (22 / 6 / 0)	5	1 на код	5	169
Mobile	28 (21 / 7 / 0)	5	1 на код	5	169
ML / DS	28 (21 / 7 / 0)	5	1 на код	5	169

Специализация	Рабочих семейств без задач на код (вопросы / разметка / ревью)	Семейств задач на код	Пилотных семейств	Клонов на семейство	Заданий в банке (с пилотными)
Data Eng / Analytics	28 (23 / 5 / 0)	5	1 на код	5	169
DevOps / SRE	28 (22 / 6 / 0)	—	—	5	140
QA	28 (23 / 5 / 0)	—	—	5	140
Security	28 (19 / 9 / 0)	—	—	5	140
Analysis	28 (19 / 9 / 0)	—	—	5	140
Management	28 (23 / 5 / 0)	—	—	5	140
Design	28 (21 / 7 / 0)	—	—	5	140
SysAdmin / Support	28 (23 / 5 / 0)	—	—	5	140
Итого					**1940**, из них рабочих **1912**

На каждую компетенцию приходится не меньше трёх рабочих семейств (тест `test_bank_size_per_competency`). Сложность b семейств разнесена от −1,8 до 2,9, так что у каждой компетенции есть простые, средние и сложные задания. Сложные семейства уровня Senior и Lead (b 2,4–2,9) есть в банках всех 12 специализаций (`bank/*.py`, у Backend часть — в `items.py`). Различающая способность a задана экспертно по типу задания:

Тип семейства	a
Сценарные задания с логами, графиками или кодом (271 семейство)	1,9–2,5
Простые вопросы без панели (`items.simple`, по одному на компетенцию у специализаций кроме Backend, 66 семейств)	1,7
Задачи на код	1,5–1,7
Пилотные	1,5

Сценарные задания с контекстом различают уровни лучше голых вопросов, поэтому a у них выше. У задач на код a занижено намеренно: LLM решает их почти всегда (2.7).

Качество клонов проверяют тесты банка (`backend/tests/test_bank.py`):

- генерация детерминирована, а клоны каждого семейства попарно различаются по содержанию;
- **утечка клона**: тот, кто запомнил ответ одного клона и выбирает на другом клоне самый похожий вариант (или строку кода), решает его не чаще чем в 35 % пар (`LEAK_MAX = 0,35`, проверка для каждого семейства). При одном варианте механики эта доля была бы 1,0, при трёх вариантах на пять клонов она около 0,2;
- правильный вариант не выдаёт себя длиной: он строго самый длинный не больше чем в 40 % клонов специализации и в 60 % клонов семейства;
- у каждой специализации есть панели с логами, графиками и кодом, а также задания-разметки, и не меньше половины заданий содержит панель;

- у всех клонов $a \in [1{,}4;\ 2{,}7]$ и $b \in [-2{,}5;\ 3{,}0]$, у каждого вопроса ровно четыре разных варианта, ключ корректен.

Задачи на код

У пяти специализаций с компетенцией «Алгоритмы» (Backend, Frontend, Mobile, ML / DS, Data Eng / Analytics; `code_items.SPECS`) в полный тест входит одна задача на код. Семейства и параметры (`code_items.py`, README «Песочница»):

Семейство	Что проверяет	Чем отличаются клоны	a	b
`code_hash_count`	подсчёт частот словарём	самое частое (при равенстве — меньшее) / первый уникальный / сколько значений встречается ≥ k раз	1,5	−1,4
`code_prefix_sums`	префиксные суммы	отрезок [l, r] с 0 / полуинтервал [l, r) / [l, r] с 1	1,6	−1,0
`code_brackets`	стек	правильность / глубина вложенности; набор скобок; пропуск прочих символов	1,6	−0,6
`code_binary_search`	бинарный поиск по запросам	< x / ≤ x / первый ≥ x / в отрезке [lo, hi]	1,7	0,0
`code_two_pointers`	два указателя	пары с суммой ≤ t / < t / ≥ t	1,7	0,6
`code_rotated_search` (пилотное)	поиск в циклически сдвинутом массиве	индекс или −1 / есть ли	1,5	1,2

У каждого клона своё имя функции, своя деталь условия и свои данные. Кандидат видит условие, сигнатуры на Python и TypeScript и 1–2 открытых примера. Скрытых тестов 10–15: примеры, крайние случаи, случайные и один большой на 10^5 элементов, на котором решение за $O(n \cdot q)$ не укладывается в лимит. Тесты **не хранятся в базе**: при проверке они пересоздаются из сида клона (`code_items.hidden_tests`), ожидаемые ответы считает эталонное решение на Python. Наружу не уходят ни сид, ни тесты, ни эталон. «Запустить на примерах» (`POST /api/quest/run`) прогоняет только открытые примеры: не больше 20 удачных запусков на задачу, не чаще раза в 3 с, на оценку это не влияет. Засчитано, только если пройдены все скрытые тесты, что для 2PL даёт бинарный исход, как у остальных заданий. Устройство и модель безопасности песочницы — README «Песочница» и раздел 9.

Если песочница недоступна, задание помечается `void` и не засчитывается ни в плюс, ни в минус. Если песочница перегружена, кандидат получает 503 «отправьте ещё раз», а время ответа фиксируется по первой отправке. Если песочница отклонила само решение (размер, недопустимые символы), ответ считается неверным, а не `void`. Так нарочно испорченным решением задачу не пропустить. Без настроенной песочницы (`SANDBOX_URL`, `SANDBOX_TOKEN`) задачи на код в тест не выдаются.

2.4. Адаптивный тест на IRT

Модель

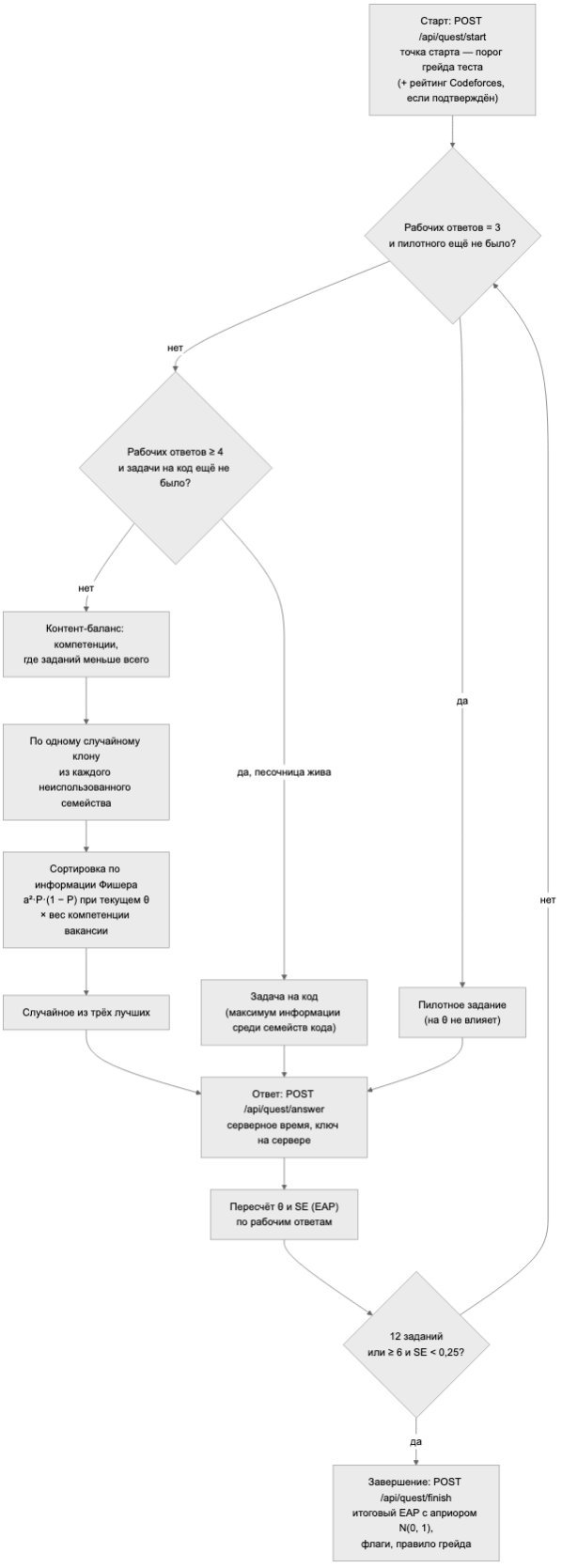
Используется **двухпараметрическая логистическая модель IRT (2PL)**. IRT (Item Response Theory, теория ответа на задание) описывает вероятность верного ответа через уровень кандидата θ и параметры задания (`irt.prob`):

P(верно | θ) = 1 / (1 + e^(−a·(θ − b)))

- **θ** — уровень кандидата на общей шкале специализации: 0 — средний уровень, единица шкалы — одно стандартное отклонение;
- **b** — сложность задания: θ, при котором вероятность верного ответа равна 50 %;
- **a** — различающая способность: насколько резко растёт вероятность верного ответа при переходе θ через b. Чем больше a, тем лучше задание отделяет кандидатов ниже b от кандидатов выше b.

Уровень оценивается методом **EAP** (expected a posteriori, `irt.eap`): апостериорное распределение θ считается на сетке из 161 точки от −4 до 4 с нормальным априором, оценка θ — среднее этого распределения, **SE** (стандартная ошибка) — его стандартное отклонение. SE показывает точность оценки: чем больше заданий и чем лучше они нацелены на уровень кандидата, тем SE меньше.

Цикл сессии



Старт. Адаптивный выбор начинается у порога грейда, на который кандидат проходит тест (`irt.start_prior`, `irt.grade_anchor`). Там информация теста максимальна именно для вопроса «подтвердил грейд или нет». У Intern порога нет, поэтому старт берётся на 0,7 ниже порога Junior. Если у кандидата есть подтверждённый аккаунт Codeforces, старт — среднее порога и оценки по рейтингу (`irt.cf_prior`: 1500 → 0, ±600 → ±1, с обрезкой и половинным весом). Точка старта только ускоряет

выбор заданий. **Итоговая оценка считается с нейтральным априором N(0, 1)** (`quest._estimate`), поэтому ни заявленный грейд, ни рейтинг Codeforces, ни достижения ФСП на грейд не влияют. Это закреплено тестом `test_cf_and_focus_do_not_change_grade`: одинаковые ответы дают одинаковые θ, грейд и компетенции, хотя у одного из кандидатов рейтинг CF 2600 и фокус вакансии.

Выбор задания (`irt.next_item`):

1. **Контент-баланс.** Сначала каждая из шести компетенций получает хотя бы одно задание, дальше допускается перекос не больше чем на одно задание. Без этого тест мог бы целиком уйти в одну компетенцию с самыми информативными заданиями.
2. **Одно семейство — один раз за сессию.** Два клона одного семейства в сессии не встречаются.
3. **Случайный клон семейства**: из каждого подходящего семейства берётся один случайный клон.
4. **Максимум информации Фишера.** Информация $I(\theta) = a^2 \cdot P \cdot (1 - P)$ показывает, насколько ответ на задание уточнит оценку при текущем θ. Она максимальна, когда b близко к θ.
5. **Случайное из трёх лучших** (`irt.TOP_K = 3`, приём randomesque). Задание выбирается случайно среди трёх лучших семейств, а не строго лучшее. Это ограничивает засвечивание самых информативных заданий: иначе кандидаты одного уровня получали бы одни и те же семейства.

Задача на код выдаётся отдельным слотом после четырёх рабочих заданий (`irt.CODE_AT = 4`), когда θ уже грубо оценён. Она одна за сессию и выбирается по тому же максимуму информации среди семейств кода (`irt.choose`).

Пилотное задание выдаётся после трёх рабочих (`quest.PILOT_AT = 3`), одно за сессию и только в полном тесте. На θ и грейд оно не влияет: ответ сохраняется для будущей калибровки параметров новых семейств. Пилотная задача на код занимает и слот задачи на код этой сессии. Сейчас пилотные семейства есть у Backend (`be_pilot_stampede`, `be_pilot_percentiles`) и задача на код `code_rotated_search` у пяти специализаций с «Алгоритмами». У семи остальных специализаций пилотных заданий нет (`table.md`, план — добавить по образцу Backend).

Остановка (`irt.should_stop`): 12 рабочих заданий, или не меньше 6 при SE < 0,25. На текущем банке сессия обычно идёт все 12 заданий со средней SE 0,32–0,33 (README). На главной странице длительность полного теста обозначена как 40–60 минут.

Оценка по компетенциям (`irt.competency_estimates`) — тот же EAP, но только по заданиям одной компетенции и с априором N(общий θ; 0,7). На двух-трёх заданиях оценка компетенции стягивается к общему θ и не скачет. Компетенции дают профиль сильных и слабых сторон. Навык считается подтверждённым тестом, если θ соответствующей компетенции не ниже max(порог грейда − 0,5; порог Junior) (`quest._skills`, соответствие «компетенция → навыки» — `quest.COMP_SKILLS`, `quest.TRACK_SKILLS`).

Что кандидат видит во время теста

- Ответ сохраняется сразу. Правильность и текущий θ до конца теста не показываются. Ключ задания (`payload.key`) наружу не уходит: ни в задании, ни в результате, ни в истории (`quest._public`; тесты `_no_leak` в `test_quest.py`).
- Незаконченную сессию нельзя перезапустить ради новых заданий. `POST /api/quest/start` при незаконченной сессии возвращает её же с текущим заданием (у кандидата не больше одной активной сессии, уникальный частичный индекс `uq_quest_active`).
- Время считается на сервере (`served_at` при выдаче, `server_ms` при ответе), клиентский таймер только показывает остаток. На сеть даётся запас 30 с (`quest.TIMEOUT_GRACE_S`). Ответ, пришедший позже лимита плюс 30 с, засчитывается как неверный.

2.5. Присвоение и изменение грейда

Пороги

Грейд присваивается по правилу **θ − SE ≥ порога** (`catalog.grade_from_theta`): берётся старший грейд, порог которого не выше нижней границы оценки θ − SE.

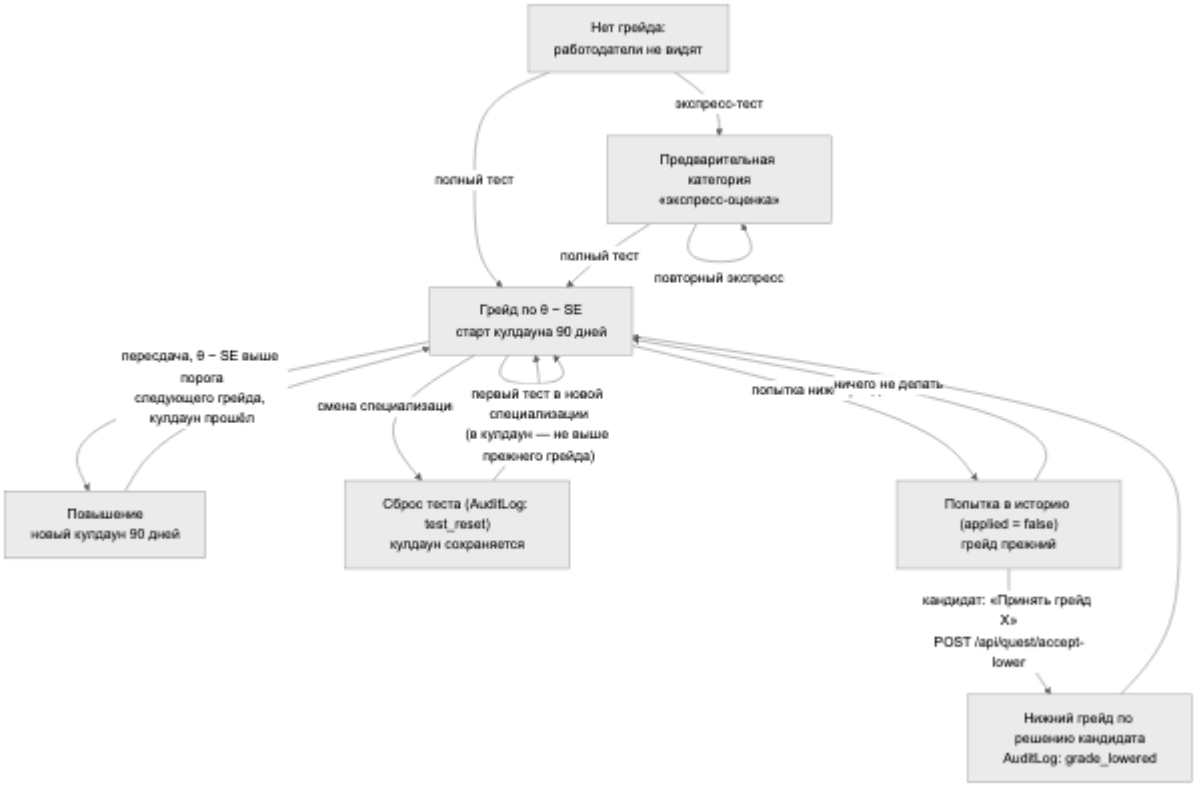
Грейд	Порог θ − SE (`catalog.GRADE_TH`)
Intern	нет порога (всё, что ниже Junior)
Junior	−0,5
Middle	0,5
Senior	1,5
Lead	2,3

Пороги заданы командой экспертно на шкале θ. Синтетика использует те же полосы грейдов: истинный θ Middle лежит в [0,5; 1,5), Senior — в [1,5; 2,3) и так далее (`seed/generate.py`, `THETA_BAND`). Вычитание SE — **запас против дрожания и завышения**. Грейд присваивается, только если тест уверен, что уровень не ниже порога. Цена этого запаса — систематическая ошибка вниз: в симуляции грейд выше истинного получают не больше 1,5 % в строке матрицы ошибок, но часть Senior и Lead получает грейд ниже (2.8). Это осознанный выбор. Завышенный грейд обманывает работодателя и подрывает доверие к категории. Заниженный кандидат может поправить пересдачей, когда пройдёт кулдаун.

Правила

Ситуация	Что происходит	Где
Первое присвоение (грейда нет или был только экспресс)	Применяется результат полного теста: θ − SE ≥ порога. С этого момента идёт кулдаун 90 дней	`quest.finish`, `irt.decide_grade`
Пересдача после кулдауна, результат выше	Грейд повышается, если θ − SE выше порога нового грейда. Кулдаун 90 дней запускается заново	`irt.decide_grade`
Пересдача после кулдауна, результат в том же грейде	Обновляются θ, SE и компетенции, грейд прежний	`quest.finish`
Пересдача во время кулдауна	Грейд не повышается. Обновляется только θ, и только если он ниже прежнего: так нельзя пересдавать ради лучшего θ внутри грейда	`quest.finish`
Измеренный грейд ниже текущего	Попытка **в профиль не идёт** (`applied = False`): грейд, θ и место в выдаче прежние. Кандидат видит результат попытки и предложение «Принять грейд X»	`quest.finish`, `quest._lower_offer`
Кандидат сам принимает нижний грейд	`POST /api/quest/accept-lower` применяет θ, SE и компетенции этой попытки. Кулдаун не сдвигается, в `AuditLog` пишется `grade_lowered`	`quest.accept_lower`
Экспресс-тест	Предварительная категория, только если грейда нет. Кулдаун не запускает, существующий грейд не меняет	`quest.finish`

Ситуация	Что происходит	Где
Смена специализации в профиле	Тест сбрасывается: грейд, θ, компетенции, подтверждённые навыки, экспресс-пометка и подтверждения работодателей. Незаконченная попытка удаляется. До нового теста кандидат работодателям не виден. В `AuditLog` пишется `test_reset` без персональных данных	`candidate._reset_test`
Первый тест после смены специализации, пока идёт кулдаун	Применяется, но грейд не выше того, что был до смены (`quest.reset_cap`), кулдаун не продлевается	`quest.finish`, `quest.reset_cap`



Обоснование

- **Повышение — новой попыткой, а не автоматическим прыжком.** Постановщики в ответах на вопросы предпочли, чтобы повышение шло отдельным тестом. Так и сделано: грейд повышается только по результату новой попытки, которую кандидат начал сам и для которой может выбрать грейд теста выше текущего. Прыжок через грейд не запрещён: если θ − SE выше порога Senior, а текущий грейд Junior, кандидат получит Senior. Каждое повышение требует запаса SE над порогом нового грейда.
- **Интервал 90 дней** (`catalog.UPGRADE_COOLDOWN_DAYS`) — пример из ТЗ («не чаще одного раза в три месяца»). Кулдаун не даёт пересдавать тест до удачной случайности: при SE около 0,3 серия попыток рано или поздно дала бы грейд выше истинного. Кулдаун запускается только полным тестом (первым присвоением или фактическим повышением), экспресс-тест его не запускает.

- **Неудача не сжигает попытку и не наказывает.** ТЗ требует, чтобы грейд принудительно не понижался, поэтому слабая попытка (усталость, плохой день) ничего не меняет в профиле. Понизиться можно только добровольно, например чтобы попасть в категорию, где больше подходящих приглашений. В интерфейсе для этого есть кнопка «Принять грейд X» с подтверждением.
- **Тест на грейд ниже** (ТЗ: «может пройти тест на уровень ниже»). Кандидат выбирает грейд теста ниже текущего. Адаптивный выбор стартует с этого уровня, а результат, если он ниже текущего грейда, становится предложением «Принять грейд X».
- **Сброс при смене специализации.** θ разных специализаций измерен на разных банках и разных компетенциях, поэтому перенос результата был бы некорректен. Кулдаун при сбросе сохраняется, а первый тест в новой специализации ограничен прежним грейдом. Иначе сменой специализации туда и обратно можно было бы обойти кулдаун. Остаточная лазейка — пересдавать θ внутри того же грейда через двойную смену специализации. Она описана в `table.md` как осознанное упрощение: грейд защищён потолком, а каждый круг стоит двух сбросов и невидимости до нового теста.

2.6. Устойчивость к распространению заданий

Почему не единый набор и не генерация нейросетью

ТЗ отвергает оба крайних подхода. Единый набор быстро разойдётся по рынку вместе с ответами. Генерация уникальных заданий нейросетью под каждого кандидата не гарантирует сопоставимой сложности и объективности. Наш подход сочетает сильные стороны обоих.

Свойство	Единый набор	LLM под каждого	Параметрические семейства + адаптивный IRT (наш)
Сопоставимая сложность между кандидатами	да	нет: сложность каждого задания неизвестна	да: b и a заданы на семейство, клоны отличаются данными и вариантом механики с разбросом b ±0,1
Объективная шкала результата	да	нет	да: θ на общей шкале 2PL, независимо от того, какие задания выпали
Устойчивость к утечке	нет	да	да: 1912 рабочих заданий, 3–4 варианта механики в семействе, адаптивные траектории, ключи не раскрываются
Проверка корректности заданий до выдачи	да	нет: ошибку в задании никто не видел	да: все клоны генерируются заранее и проверяются тестами банка, эталоны задач на код прогоняются в настоящей песочнице
Зависимость от внешнего сервиса	нет	да	нет: банк генерируется детерминированно, без LLM

Аналитическая работа (сюжет, механика дефекта, правильный ответ и правдоподобные ошибки) выполняется один раз, при написании шаблона. Вариативность затем даёт код. Поэтому на выходе получаются много разных заданий, но известного качества и сложности.

Модель угроз и защита

Угроза	Защита	Что остаётся
Утечка конкретного задания с ответом (скриншот, пересказ)	Клоны семейства различаются вариантом механики и данными. Правильный ответ утёкшего клона на другом клоне стоит среди дистракторов. Запомненный ответ решает другой клон не чаще чем в 35 % пар (проверка для каждого семейства, `test_clone_answer_does_not_leak`)	Если утекли все клоны семейства с ответами, это семейство скомпрометировано. Лекарство — пополнение банка новыми семействами и вариантами (засев идёт на старте без потери истории)
Сбор «базы ответов» сообществом	Кандидат не узнаёт правильность своих ответов ни во время теста, ни после: ключи наружу не уходят, в результате только θ, компетенции и грейд. Чтобы собрать базу, нужно восстанавливать ключи по косвенным признакам на 1912 заданиях	Эксперт, знающий ответ, может опубликовать решения. Это то же, что научить кандидата, и защищает от этого объём и вариативность банка, а не секретность
Одинаковый тест у кандидатов одного уровня	Адаптивный выбор с контент-балансом, случайный клон семейства, случайный выбор из трёх лучших семейств. Траектории расходятся с первых заданий	По природе адаптивного теста семейства, самые информативные для типичного уровня, выпадают чаще остальных. Контроль экспозиции по счётчикам выдачи — следующий шаг
Пересдача до удачи	Кулдаун 90 дней на повышение, правило θ − SE. В кулдаун пересдача может только понизить θ внутри грейда	Обход сменой специализации ограничен потолком прежнего грейда (2.5)
Перезапуск сессии ради других заданий	Незаконченная сессия возвращается та же, с тем же текущим заданием. Активная сессия у кандидата одна	—
Подсказчик или LLM рядом с кандидатом	Таймер с серверным временем. Задания на диагностику по логам, графикам и коду, где LLM помогает мало. Флаги `inconsistent`, `too_fast`, `code_pasted` (ниже). Умеренный вес задач на код	Полностью закрывает только прокторинг, его в MVP нет (`table.md`)
Подбор скрытых тестов задачи на код	Скрытые тесты не хранятся и не передаются, «на примерах» — только открытые примеры, до 20 запусков	—
Обход проверки кода сбоем песочницы	`void` только при недоступности песочницы. Решение, которое песочница отклонила, — неверный ответ. Журнал nsjail ведётся отдельно от вывода решения, поэтому подделать «сбой» нельзя	—

Флаги для разбора

После завершения теста `irt.suspicion_flags` и `quest.finish` проставляют флаги:

- **`inconsistent`** — несогласованный профиль ответов: сложные задания решены, а простые нет. Критерий — статистика person-fit l_z (Drasgow и соавт., 1985), стандартизованное логарифмическое правдоподобие ответов при оценённом θ. Флаг ставится при l_z < −1,96,

считается от шести ответов. Это типичный след подсказок на сложных заданиях.
- `too_fast` — не меньше двух ответов быстрее 15 % лимита времени задания (по серверному времени).
- `code_pasted` — верное решение задачи на код, больше половины которого вставлено из буфера. Счётчик вставок ведёт клиент, поэтому это только сигнал, его можно обойти (`table.md`).

Любой из этих флагов, а также грейд Senior и выше без пройденного «разбора решения» ставит флаг `review_required`. Флаги сохраняются в попытке (`QuestSession.flags`) и возвращаются в результате теста. **На грейд они не влияют**: флаг — повод для разбора, а не наказание, потому что ложные срабатывания неизбежны. Сам «разбор решения» (короткое интервью по решённым заданиям) пока не реализован. Это помечено в коде как осознанное упрощение (`# ponytail` в `quest.finish`).

2.7. Симуляция LLM-читера

Чтобы проверить, насколько тест устойчив к кандидату, который отвечает с помощью LLM, валидация (`validation.cheater`) прогоняет адаптивные сессии на банке Backend с моделью ответов «читера». Вероятности заданы вручную, без прогона настоящей LLM (`validation._cheater_p`):

- вопрос с вариантами без контекста — 0,9 независимо от сложности;
- ревью и разметка кода с тонкими дефектами — как у человека с θ = −0,5;
- задачи с контекстом (логи, графики) — как у человека с θ = −0,3;
- задачи на код со стандартными алгоритмами — 0,95 независимо от сложности.

Результат (README «Результаты валидации», 200 сессий на строку):

Тип заданий	Грейд читера
Вопросы с вариантами	Senior
Ревью кода	Intern
Задачи с контекстом	Intern
Задачи на код (песочница)	Junior
Смешанный адаптивный тест	Junior

Выводы:

- Тест только из вопросов с вариантами LLM прошёл бы на Senior. Именно поэтому банк строится на сценариях с логами, графиками и кодом, где нужно ставить диагноз по данным, а не вспоминать определение.
- Задачи на код LLM решает почти всегда. Тест только из них дал бы Junior с θ ≈ +0,95, а выше его держат лишь пять семейств и большая SE. Поэтому задача на код одна из 6–12 заданий, а её a (1,5–1,7) ниже, чем у сценарных заданий.
- Смешанный адаптивный тест даёт читеру Junior.
- Проверка несогласованности ловит 30 % читеров при 2 % ложных тревог у честных кандидатов. Без задач на код на той же синтетике было 45 % и 0 %: решённая читером задача на код совместима по сложности со средним θ и вытесняет задание, на котором профиль ответов расходится сильнее. Это компенсируют флаги `too_fast` и `code_pasted` (`table.md`).

Ограничение: модель читера задана руками. Следующий шаг — заменить её долями верных ответов реальной LLM на банке.

2.8. Качество теста

Процедура валидации, её допущения и полные таблицы — в разделе 4. Ниже — цифры, которые относятся к критерию № 2 (README «Результаты валидации», один прогон, seed 42).

Согласованность при пересдаче (симуляция на банке Backend, 200 кандидатов, пересдача на других клонах и по другой траектории):

- грейд в профиле после пересдачи не меняется у 88,5 % кандидатов (с учётом правил: повышение по θ − SE, без принудительного понижения);
- сырая оценка грейда по θ − SE совпадает у 76 %;
- корреляция истинного и оценённого θ — 0,95.

Разделяющая сила (200 симулированных кандидатов на грейд, банки всех 12 специализаций по кругу, сессия как в продукте):

- точное совпадение грейда — 57,7 %, в пределах ±1 грейда — 99 %, 96,9 % ошибок приходится на соседний грейд, к с квадратичными весами — 0,877;
- если считать грейд по самой оценке θ, без запаса SE, точность 75 %. Разница с 57,7 % — цена осторожного правила θ − SE: ошибки почти только вниз, грейд выше истинного — не больше 1,5 % в строке;
- AUC оценки θ между соседними грейдами: Intern|Junior 0,966, Junior|Middle 0,948, Middle|Senior 0,919, Senior|Lead 0,914. AUC — вероятность того, что случайный кандидат старшего грейда получит θ выше случайного кандидата младшего;
- по специализациям: точно 49–66 %, ±1 грейд — 97,5–100 %, SE 0,32–0,33.

Senior и Lead чаще получают грейд ниже истинного (точно 42,5 % и 29,5 %). При SE около 0,3 правилу θ − SE нужен запас над порогом. Это осознанная защита от завышения. Варианты на обсуждение с постановщиками перечислены в `table.md`: θ − 0,5·SE для повышения до Senior и выше или удлинение теста.

Дискриминативность заданий (1912 рабочих заданий):

- параметр a: медиана 2,19 (Q1–Q3 1,84–2,29), у всех заданий a ≥ 1,44, что по шкале Baker соответствует «высокой» различающей способности;
- точечно-бисериальная корреляция ответа с суммой остальных (r_pb, item-rest) на калибровочной выборке из 500 симулированных кандидатов: медиана 0,48, слабых заданий (r < 0,2) 1,9 %.

Честная оговорка из README: a и b заданы при составлении банка и не откалиброваны на живых ответах, а r_pb считается на ответах, смоделированных по той же 2PL. Поэтому эти цифры проверяют согласованность банка и механики, но не его калибровку на реальных людях. Экспертной разметки ФСП пока нет.

Внешний сигнал корректности грейда заложен в продукт. После принятого приглашения работодатель может отметить «грейд подтвердился», «не подтвердился» или «наняли» (`POST /api/employer/invites/{id}/feedback`, `catalog.FEEDBACK`). Подтверждения увеличивают счётчик `employer_confirmed` в карточке кандидата. Отметка «не подтвердился» пока только сохраняется, очередь перепроверки не реализована.

2.9. Как из описания вакансии формируется тест

ТЗ просит «подготовить несколько описаний вакансий и на их основе показать, как система формирует задания и тесты». Банк заданий общий для специализации, а категория кандидата одна для всех работодателей. Поэтому вакансия не создаёт новых заданий. Она определяет, **какие компетенции, какой диапазон сложности и какие семейства банка** тест проверит под эту потребность. Так результат остаётся сопоставимым между кандидатами, а тест — под вакансию.

Как это работает (`items.plan_for_need`):

1. Текст вакансии разбирается в структуру: специализация, трек, грейд, обязательные и желательные навыки (`parsing.parse_need`; правила и справочник навыков, по желанию LLM, см. раздел 10).
2. Навыки переводятся в веса компетенций по таблице `items.SKILL_COMP`. Например, PostgreSQL относится к «Базам данных и транзакциям» и «SQL», Kafka — к «Конкурентности» и «Пайплайнам». Обязательный навык добавляет к весу 0,25, желательный — 0,1, потолок веса — 1,5.
3. Диапазон сложности — порог грейда вакансии ± 1 по шкале b.
4. Подбираются семейства банка этой специализации в диапазоне сложности. Они упорядочиваются по весу компетенции и близости b к порогу, для каждого показывается пример условия (отдельный сид, такого клона в банке нет).
5. Оцениваются число заданий и время.

Где это видно. Работодатель на экране новой потребности получает панель «Как система проверит кандидатов» (`POST /api/quest/plan` по сохранённой потребности или по структуре, поправленной на экране). При `DEMO=1` есть публичный предпросмотр по тексту без входа и без LLM (`POST /api/quest/plan/preview`).

Как это влияет на тест кандидата. Если кандидат начинает тест из приглашения (`invitation_id` в `POST /api/quest/start`), веса компетенций вакансии сохраняются в сессии (`QuestSession.focus`). Они умножают информацию задания при выборе (2.4), поэтому тест чаще берёт задания по компетенциям вакансии. **На оценку и грейд фокус не влияет**: θ считается по всем ответам одинаково, грейд — по тем же порогам (тест `test_cf_and_focus_do_not_change_grade`).

Пример — результат `plan_for_need` на текущем коде (разбор правилами, песочница включена). Вакансия: «Ищем backend-разработчика уровня Middle на Python в команду платежей. Обязательно: PostgreSQL, Kafka, Docker. Желательно Kubernetes.»

- Разбор: Backend · Python · Middle; обязательные — Python, PostgreSQL, Kafka, Docker; желательные — Kubernetes.
- Веса компетенций: Диагностика инцидентов 1,35 (Docker, Kubernetes); Базы данных и транзакции 1,25 (PostgreSQL); Ревью кода 1,25 (Python); Конкурентность 1,25 (Kafka); Проектирование API 1,0; Алгоритмы 1,0.
- Диапазон сложности b: от −0,5 до 1,5 (порог Middle 0,5 ± 1).
- Первые семейства плана: `be_inc_rollback` и `be_inc_pool` (диагностика), `be_db_lost_update` (разметка, БД), `be_cc_race` (разметка, конкурентность), `be_rv_mutable_default` и `be_rv_double_charge` (ревью кода), `be_db_migration`, `be_cc_gather`; всего 14 семейств.
- Заданий в тесте — 12, оценка времени в плане — 48 минут.

Для вакансии «Нужен senior DevOps / SRE: Kubernetes, Terraform, Prometheus, Linux. Будет плюсом Kafka.» план другой. Веса: Диагностика инцидентов и План отката по 1,5, остальные компетенции по 1,25. Диапазон b — от 0,5 до 2,5 (порог Senior 1,5 ± 1). Семейства начинаются с `ops_rb_replace` (план отката) и `ops_inc_throttle` (диагностика). Время — около 37 минут. Синтетика содержит 250 потребностей разных специализаций (`seed/generate.py`), и для любой из них план строится так же.

2.10. Регулярные короткие задания от работодателей

ТЗ (п. 2.1) предлагает периодически давать кандидату короткую задачу от работодателя, чтобы профиль оставался актуальным. Реализация — `backend/app/routers/tasks.py`.

- **Кто создаёт.** Работодатель (`POST /api/tasks`) указывает название, текст, специализацию, компетенцию (по желанию), формат («решить» или «предложить подход»), ориентировочное время и тип ответа. Задание может быть публичным, тогда оно лежит на витрине для всех кандидатов специализации. Непубличное задание привязано к потребности, и его видят только кандидаты с действующим приглашением по этой потребности. Кроме того, задание можно приложить к приглашению вместо первого созвона.
- **Как доходит до кандидата.** Без механик давления: ни пушей, ни серий — это осознанное отступление от формулировки ТЗ «система периодически предлагает»: рассылки заданий нет. Задание лежит на витрине (`GET /api/tasks/showcase`, экран `/candidate/tasks`), куда кандидат заходит сам, или приходит вместе с конкретным приглашением (постановщики в ответах подтвердили: «адресно»). Решать задания можно только после теста, потому что работодатель видит решение вместе с категорией. Попытка на задание одна.
- **Автопроверка** (`backend/app/services/autocheck.py`). Если у задания есть эталон, система проверяет ответ сама. Поддерживаются типы: текст (с нормализацией регистра, «ё», пробелов и пунктуации по краям или посимвольно), число с допуском, один вариант, набор вариантов, регулярное выражение (с таймаутом и ограничением сложности шаблона против ReDoS). Эталон хранится только на сервере, кандидат видит лишь «совпало / не совпало». Открытый ответ (`open`) оценивает работодатель по шкале «сильное решение / нормально / слабо». Он же может переоценить автопроверку, и его оценка главнее.
- **Влияние на профиль.** Последнее оценённое решение становится **свежим сигналом** в карточке кандидата у всех работодателей: «решил задачу N дней назад, оценка …» (`tasks.task_signal`). Неверный автоответ в сигнал не идёт: одна неудачная попытка не перекрывает прежнюю оценку, её видит только компания-автор. **На грейд и место в выдаче сигнал не влияет.** Грейд задаёт только стандартизированный тест, а задания работодателей не калиброваны и несопоставимы между компаниями.
- **Ограничения** (`table.md`): эталон можно перебирать новыми аккаунтами (попытка одна на аккаунт), план — ограничение по почте или ФСП ID и ротация вариантов. Задания с исполнением кода по тестам работодателя — следующий шаг: песочница для этого уже есть.

2.11. Ограничения и план развития

Ограничение	Почему так в MVP	План
Параметры a и b всех семейств заданы экспертно и не откалиброваны на живых ответах	Живых кандидатов на хакатоне нет, эталонной разметки нет (ТЗ)	Пересчёт a и b методом максимального маргинального правдоподобия (MML/EM), когда наберётся около 300 ответов на семейство. Новые семейства сначала идут пилотными: ответы на них собираются без влияния на грейд (`table.md`)
Варианты механики внутри семейства могут различаться по сложности	b задан на семейство	Калибровать b по варианту (номер варианта = номер клона mod число вариантов)
Пилотные задания есть не у всех специализаций	Пилотные семейства пока написаны только для Backend и задач на код	Добавить пилотные семейства остальным специализациям по образцу `be_pilot_*`
Цифры качества получены на синтетике и по той же 2PL, на которой построен тест	Внешних данных нет	Валидация на живых ответах и сверка с экспертной оценкой ФСП; в симуляции разделяющей силы — доля завышенных заявок грейда, как в синтетике (раздел 4)
Модель LLM-читера задана вручную	Прогон реальной LLM по банку не делался	Заменить долями верных ответов реальной модели на банке
Комментарий к ревью кода не оценивается	Автоматическая оценка свободного текста ненадёжна	Оценка комментария работодателем или LLM с проверкой по рубрике
«Разбор решения» для Senior+ и флагов — только флаг `review_required`	Отдельный сервис интервью не входил в MVP	Асинхронный разбор: 3–4 вопроса по решённым заданиям
Нет прокторинга (камера, экран)	За рамками MVP	Прокторинг для подтверждения Senior и Lead
Senior и Lead присваиваются осторожно (точно 42,5 % и 29,5 %)	Правило θ − SE защищает от завышения	Обсудить с постановщиками θ − 0,5·SE для повышения до Senior+ или 14 заданий у кандидатов, близких к порогу
Одна категория на кандидата	Постановщики: для MVP достаточно одной	Отдельный тест и свой кулдаун на каждую категорию
Контроль экспозиции заданий только случайным выбором из трёх лучших	Достаточно для объёма MVP	Счётчики выдачи и ограничение экспозиции популярных семейств (Sympson–Hetter), ротация семейств

3. Механика подбора и логика категоризации

Раздел описывает, как кандидат попадает в категорию, как работодатель получает ранжированную подборку и как происходит выход на контакт. Механика тестирования, которая задаёт грейд, описана в разделе 2, процедура валидации и её результаты — в разделе 4.

Идея продукта — обратный найм. Кандидат не ищет вакансии и не пишет сопроводительных писем. Он один раз проходит тест и попадает в категорию. Работодатель описывает потребность, получает анонимную подборку с объяснением оценки и сам отправляет конкретному человеку приглашение-оффер. Вилка зарплаты в приглашении обязательна. Контакты кандидата работодатель видит только после того, как кандидат принял приглашение.

Термины:

- **θ (тета)** — оценка уровня кандидата по шкале IRT (Item Response Theory, модель «ответ зависит от уровня человека и сложности задания»). Ноль — средний уровень популяции.
- **SE** — стандартная ошибка оценки θ. Чем меньше SE, тем точнее измерение.
- **Потребность** — описание того, кого ищет работодатель: специализация, трек, грейд, обязательные и желательные навыки, вилка, формат работы, город. Вакансией потребность не является и кандидатам не публикуется.
- **Приглашение** — адресный оффер конкретному кандидату с описанием роли, вилкой «от–до», форматом и способом связи с компанией.

3.1. Категория кандидата

Справочник

Категория — связка «специализация · трек · грейд», например «Backend · Python · Middle». Справочник лежит в `backend/app/catalog.py`.

Измерение	Значения	Сколько
Специализация (`SPECS`)	Backend, Frontend, Mobile, ML / DS, Data Eng / Analytics, DevOps / SRE, QA, Security, Analysis, Management, Design, SysAdmin / Support	12: восемь разработческих и четыре нетехнические ИТ-роли
Трек (`TRACKS`) — стек или подроль внутри специализации	Backend: Python, Go, Java / Kotlin, Node.js, C# / .NET; Frontend: React, Vue, Angular; Mobile: iOS, Android, Flutter; ML / DS: Classic ML, CV, NLP / LLM; Data Eng / Analytics: Data Engineering, BI / Analytics; DevOps / SRE: Kubernetes, Cloud; QA: Manual, Automation; Security: AppSec, Pentest; Analysis: System, Business; Management: Project, Product; Design: UX/UI, Product Design; SysAdmin / Support: SysAdmin, Support	30
Грейд (`GRADES`)	Intern, Junior, Middle, Senior, Lead	5

Всего 30 × 5 = 150 категорий. Справочник составлен командой по общепринятым в ИТ ролям, как разрешает ТЗ. Постановщики в письменных ответах подтвердили, что для MVP достаточно связки «специализация + грейд» и что широта справочника оценку не повышает (`table.md`, раздел «Письменные ответы постановщиков»).

Рядом в `catalog.py` лежат справочники, на которые опирается подбор:

- навыки `SKILLS`: 51 каноническое название с синонимами («k8s», «кубер» → Kubernetes);
- компетенции `COMPETENCIES`: по шесть на специализацию, их измеряет тест;
- рыночные перцентили зарплат `MARKET_MOSCOW` с множителями по специализации (`SPEC_MULT`) и городу (`CITY_MULT`).

Как кандидат попадает в категорию

1. **Опрос.** В профиле кандидат выбирает специализацию и трек из справочника и заявляет предполагаемый грейд (`POST /api/candidate/profile`). Резюме может подсказать специализацию и навыки (`services/parsing.py`), но решение остаётся за кандидатом.
2. **Тест.** Адаптивный тест по банку заданий специализации измеряет θ ± SE и θ по каждой компетенции (раздел 2).
3. **Грейд категории только по тесту.** Грейд присваивается, если θ − SE не ниже порога грейда (`catalog.GRADE_TH`: Junior −0,5, Middle 0,5, Senior 1,5, Lead 2,3; ниже Junior — Intern). Вычитание SE защищает от завышения: при неуверенном измерении кандидат получает грейд ниже.
4. **Навыки, подтверждённые тестом.** Навык из профиля считается подтверждённым, если θ связанной с ним компетенции не ниже порога (`routers/quest.py`, `_skills`). Остальные навыки из профиля остаются заявленными.

Заявленный грейд категорию не задаёт. Он нужен только как цель теста: с него начинается выбор заданий, а результат показывается вердиктом «подтверждён / ниже / выше». Если тест показал грейд ниже заявленного, кандидат попадает в категорию по тесту, а работодатель видит в карточке оба значения, например «заявлен Senior, тест — Middle».

Без теста кандидата не видно. Правило задают функции `matching.tested`, `matching.category_grade` и `matching.visible`. Кандидат без пройденного теста не появляется в подборке, в поиске по банку и в лотерее, его нельзя пригласить, и сам он не может откликнуться на вакансию. В кабинете ему прямо сказано, что работодатели не увидят его до теста.

Экспресс-тест (5 заданий по минуте) даёт только предварительную категорию с пометкой «экспресс-оценка» и только тому, у кого грейда ещё нет. Работодатель видит пометку в карточке (поле `express`).

Смена специализации сбрасывает грейд, θ и подтверждённые навыки (`candidate._reset_test`, событие `test_reset` в журнале). До нового теста кандидат работодателям не виден. Правила повышения грейда, кулдаун 90 дней и запрет принудительного понижения описаны в разделе 2.

Помимо теста, работодатель видит **уровни доверия** к профилю (`card.trust`): тест пройден, разбор решения пройден, сколько раз кандидата подтвердили работодатели (отметки «грейд подтвердился» и «наняли» после принятого приглашения). Отдельно показывается свежий сигнал по заданиям от работодателей (`task_signal`). На грейд и место в выдаче он не влияет.

3.2. Потребность работодателя: от текста к структуре

Работодатель пишет потребность обычным текстом, например: «Нужен бэкенд на Python, мидл, Kafka и PostgreSQL обязательно, Kubernetes будет плюсом, гибрид в Москве, до 300 тысяч».

1. **Разбор** — `POST /api/employer/needs/parse` → `parsing.parse_need`. Сначала работают правила:
   - навыки ищутся по справочнику синонимов;
   - навык, стоящий после маркеров «хорошо бы», «будет плюсом», «желательно», «nice to have», попадает в желательные, остальные — в обязательные;
   - специализация, трек, грейд, формат, город и суммы определяются регулярными выражениями.
2. **LLM (если включена).** Ответ модели принимается, только если значения есть в справочниках (`SPECS`, `TRACKS`, `GRADES`, `FORMATS`, `SKILLS`); исключение — город: он берётся из ответа модели свободным текстом до 64 символов и только если правила город не нашли. Суммы модели используются, только если правила не нашли ни одной суммы. Перед вызовом текст проходит маскировку ПДн (`parsing.mask_pii`). Без модели всё работает на правилах (раздел 10).
3. **Персональные признаки отбрасываются.** Упоминания возраста, пола, вуза, фото и национальности вырезаются ещё до разбора (`parsing._strip_personal`). Работодатель видит подсказку «Персональные признаки не используются в подборе: …».
4. **Достройка вилки по рынку** (`parsing.fill_fork`):
   - «до X» → нижняя граница = min(медиана, X / 1,25), но не ниже 25-го перцентиля (p25);
   - «от X» → верхняя граница = max(медиана, X · 1,25);
   - вилка не указана → от медианы до 75-го перцентиля (p75). Каждое такое допущение показывается подсказкой. Не указан грейд — ставится Middle с подсказкой.
5. **Правка и сохранение.** Работодатель правит структуру на экране. Для исправленной категории `POST /api/employer/needs/category` возвращает рыночные перцентили, число кандидатов в категории и подсказку о положении вилки относительно рынка. `POST /api/employer/needs` сохраняет потребность: навыки приводятся к справочнику, при включённой модели считается эмбеддинг (только по структурным полям: специализация, трек, грейд, навыки).

Рекомендованные категории

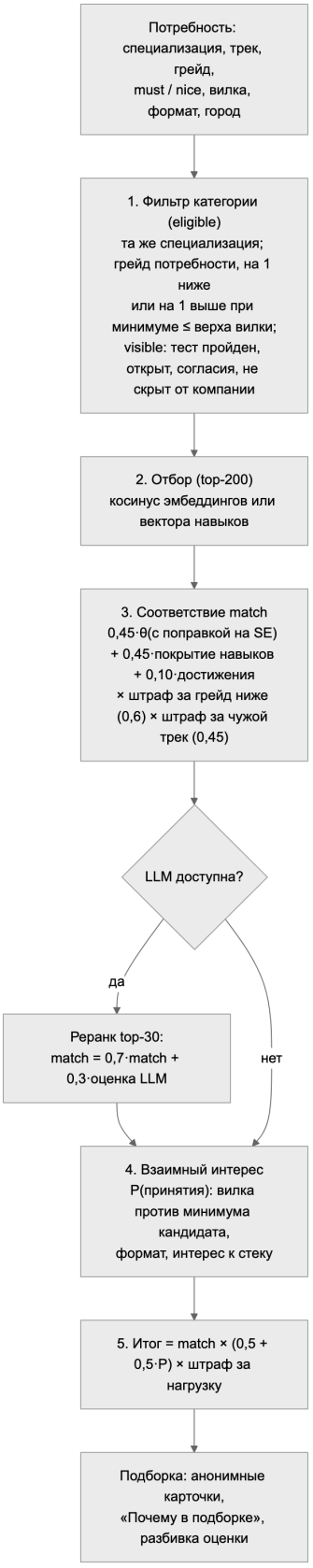
Вместе с подборкой (`GET /api/employer/needs/{id}/picks`, поле `categories`) работодатель получает до пяти категорий (`employer._categories`):

- **точная** — трек и грейд потребности;
- **два соседних трека** того же грейда, где больше всего открытых кандидатов, с пометкой «Соседний трек: стек …, навыки частично переносимы»;
- **тот же трек грейдом ниже и выше**, с пометкой, укладывается ли вилка в рынок этого грейда.

Для каждой категории указаны число видимых кандидатов (только прошедшие тест, открытые, с согласием на публикацию) и оценка вероятности принятия при текущей вилке. Кнопка соседней категории ведёт в поиск по банку с теми же специализацией, треком и грейдом.

3.3. Ранжирование подборки

Конвейер реализован в `backend/app/services/matching.py` (`rank` = `eligible` + `score_candidates`).



Шаг 1. Фильтр категории

Функция `matching.eligible` оставляет кандидатов, у которых совпадает специализация и выполнено одно из условий по грейду (грейд — по тесту):

- **грейд потребности** — основной пул;

- **на один ниже** — попадает в выдачу, но со штрафом ×0,6 (`LOWER_GRADE_PEN`): «стек совпал, грейд ниже»;
- **на один выше** — только если минимум кандидата помещается в верх вилки. Если верхней границы нет, сравнение идёт с медианой рынка. Штрафа нет: дороговизну учитывает вероятность принятия.

Кроме того, кандидат должен быть видимым (`matching.visible`):

- статус «открыт к предложениям»;
- тест пройден;
- действуют согласия на обработку и на публикацию в текущей редакции текста (`matching.consent_ok`);
- компания не внесена кандидатом в список «скрыть профиль от».

Чужие треки той же специализации в пул попадают, но получают штраф ×0,45 (`TRACK_PEN`), если в потребности задан трек.

Шаг 2. Отбор кандидатов (retrieval)

Отбор сужает пул до 200 кандидатов (`RETRIEVE_K`) по косинусной близости:

- **эмбеддинги** — если они есть у потребности и у всех кандидатов пула с одинаковой размерностью;
- **иначе вектор навыков** по справочнику. В потребности обязательный навык весит 1,0, желательный — 0,5. У кандидата подтверждённый тестом навык весит 1,0, заявленный — 0,5.

В режиме по умолчанию (`LLM_PROVIDER=none`) отбор всегда идёт по навыкам. Какой способ сработал, видно в объяснении (`retrieval`: `embedding` или `skills`).

Шаг 3. Соответствие (match)

`matching._relevance` считает три части и складывает их с весами `W_THETA = 0,45`, `W_COVER = 0,45`, `W_ACH = 0,10`.

Часть	Формула	Смысл
θ с поправкой на SE	σ((θ − 0,5·SE − порог грейда потребности) / 0,5), σ — логистическая функция; для Intern порог берётся не ниже −1,5	На пороге грейда — 0,5; на 0,5 выше порога — 0,73. Неточное измерение (большая SE) снижает оценку
Покрытие навыков	Средневзвешенный «кредит» по навыкам потребности: обязательный навык весит 1, желательный 0,5. Кредит: подтверждён тестом — 1,0; только заявлен — 0,4; подтверждён близкий навык (Kafka ↔ RabbitMQ, PostgreSQL ↔ MySQL, FastAPI ↔ Django, Java ↔ Kotlin, TypeScript ↔ JavaScript) — 0,3	Подтверждённое тестом весит в 2,5 раза больше заявленного в резюме
Достижения	Сигнал ∈ [0, 1] по подтверждённым достижениям (`integrations/scoring.signal`)	Влияет только на место внутри категории, не на грейд

Сумма умножается на штрафы шага 1: грейд ниже ×0,6, чужой трек ×0,45.

Сигнал достижений (подробности источников — раздел 5). Иерархии соревнований нет: вес определяется только результатом кандидата — победитель 0,8, призёр 0,6, финалист 0,35, участник отбора 0,05. Несколько соревнований складываются с убывающей отдачей: k-е по силе входит с множителем $0{,}5^k$. Поэтому десяток онлайн-явок не сравняется с одним призовым местом. Рейтинг Codeforces — отдельное слагаемое: 1200 и ниже — 0, 3000 и выше — 1. Учитываются только подтверждённые достижения. Спортивный разряд и дисциплина ФСП показываются в карточке как факты, на вес не влияют.

LLM-реранк (необязательный). Если модель включена и в потребности есть навыки, топ-30 (`RERANK_K`) отправляется в модель. В запрос уходят только структурные поля: публичный id, грейд, трек, θ, подтверждённые и заявленные навыки. ФИО и текста потребности в запросе нет. Оценка модели подмешивается с долей 0,3 (`LLM_BLEND`) и с теми же штрафами за грейд и трек. При сбое модели подборка считается без неё.

Шаг 4. Взаимный интерес

Подборка, в которой все работодатели зовут одних и тех же сильных кандидатов, а те отказывают по зарплате, бесполезна обеим сторонам. Поэтому итог учитывает оценку того, что кандидат примет приглашение (`matching.accept_prob`):

- r = (верх вилки − минимум кандидата) / минимум, обрезается до ±0,5;
- z = −0,55 + 7·r (при r < 0 наклон вдвое круче) − 1,2, если формат не подходит, − 1,4·(1 − доля обязательных навыков, которые есть у кандидата);
- P = 1 / (1 + e^(−z)).

Примеры: вилка на 30 % выше минимума — P ≈ 0,82; верх вилки равен минимуму — ≈ 0,37; вилка на 10 % ниже минимума — ≈ 0,12. Если в потребности нет верхней границы, берётся медиана рынка.

Минимум кандидата уточняется по его отказам (`matching._effective_min`). Если кандидат отклонил приглашение с причиной «Зарплата», его эффективный минимум поднимается до верха отклонённой вилки + 5 %.

Коэффициенты подобраны вручную по макету и помечены в коде как упрощение (`# ponytail`). Следующий шаг — дообучить логистическую регрессию на реальных принятиях и отказах.

Шаг 5. Итог и нагрузка

Итог = match × (0,5 + 0,5·P) × штраф за нагрузку (`matching.final_score`).

- Вероятность принятия сдвигает итог в пределах ×0,5…×1 и не может перебить соответствие: слабый кандидат с высокой P не обгонит сильного.
- Штраф за нагрузку (`catalog.load_penalty`) — за открытые приглашения кандидата: каждое снижает итог на 15 % (0 → 1,0; 1 → 0,85; 2 → 0,7; 3 → 0,55). При четырёх, то есть заполненной ёмкости, множитель 0,3: кандидат остаётся в выдаче, но опускается вниз, а пригласить его нельзя (см. 3.5).

Порядок выдачи — по итогу, при равенстве — по публичному id, то есть детерминированно (`matching.order_key`).

Что не участвует в ранжировании: ФИО, почта, Telegram, пол, возраст, вуз, текст резюме, а также роли и софт-скиллы со слов кандидата. Они видны в карточке справочно, с пометкой «со слов кандидата, в оценке не участвуют». Это инвариант проекта (`CLAUDE.md`): функции оценки этих полей не

читают. Тест `tests/test_privacy.py` обходит все ответы, которые видит работодатель, и проверяет, что ПДн в них нет.

3.4. Объяснимость, анонимная карточка и поиск по банку

«Почему в подборке»

Для каждого кандидата `score_candidates` собирает объяснение (`explanation`), которое показывается в подборке и в карточке кандидата (`GET /api/employer/candidates/{public_id}?need_id=…`):

- **краткое «почему»** — совпавшие подтверждённые навыки и сильнейшая компетенция с перцентилем, например: «Совпадение стека Python, Kafka; базы данных и транзакции — 91-й перцентиль»;
- **пробелы человеческим языком** (`matching.gaps`): «Kafka не подтверждён тестом», «Kafka: опыт с RabbitMQ, близкий навык подтверждён» или просто название отсутствующего навыка;
- **вилка против ожиданий:** «Ожидания от … на руки — внутри вашей вилки» или «— выше вашей вилки: вероятность принятия ниже»;
- **формат и город;**
- **достижения:** лучшее подтверждённое по каждому источнику — ФСП, хакатон, CTFtime, Codeforces. Если достижений нет, показывается текст: «Внешних достижений нет. Это не снижает позицию: оценка по тесту»;
- **разбивка итога** (`breakdown`): соответствие × (0,5 + 0,5 · P) × поправка на нагрузку = итог. В API отдаются все слагаемые с весами (`parts`);
- **компетенции** с θ ± SE.

Кандидат во входящем приглашении тоже видит строку «Почему вы» — то же краткое объяснение.

Анонимная карточка

`matching.card` собирается по белому списку полей:

- публичный id (например, A-1042);
- категория, заявленный грейд, пометка экспресс-оценки, перцентиль, θ ± SE;
- подтверждённые и заявленные навыки, пробелы;
- бейджи достижений и спортивный разряд;
- минимальная зарплата, формат и город;
- нагрузка, уровни доверия.

ФИО, почты, Telegram и резюме в карточке нет. Постановщики в письменных ответах допускали показ ФИО и мест работы, но команда осознанно оставила анонимность до принятия приглашения. Это защищает от предвзятости и от сбора контактов фиктивными компаниями (`table.md`).

Фильтры без потери подборки

У подборки есть фильтры (`filters` в `GET …/picks`):

- `fmt` — подходит формат;
- `free` — у кандидата не заполнена ёмкость;
- `fsp` — есть подтверждённое достижение ФСП;
- `cf` — есть подтверждённый аккаунт Codeforces.

Фильтр только сужает уже посчитанное ранжирование. Порядок сохраняется, и у каждого кандидата остаётся его место в полной подборке (`rank`). Сняли фильтр — вернулась та же подборка. Число кандидатов без фильтров отдаётся в поле `total`.

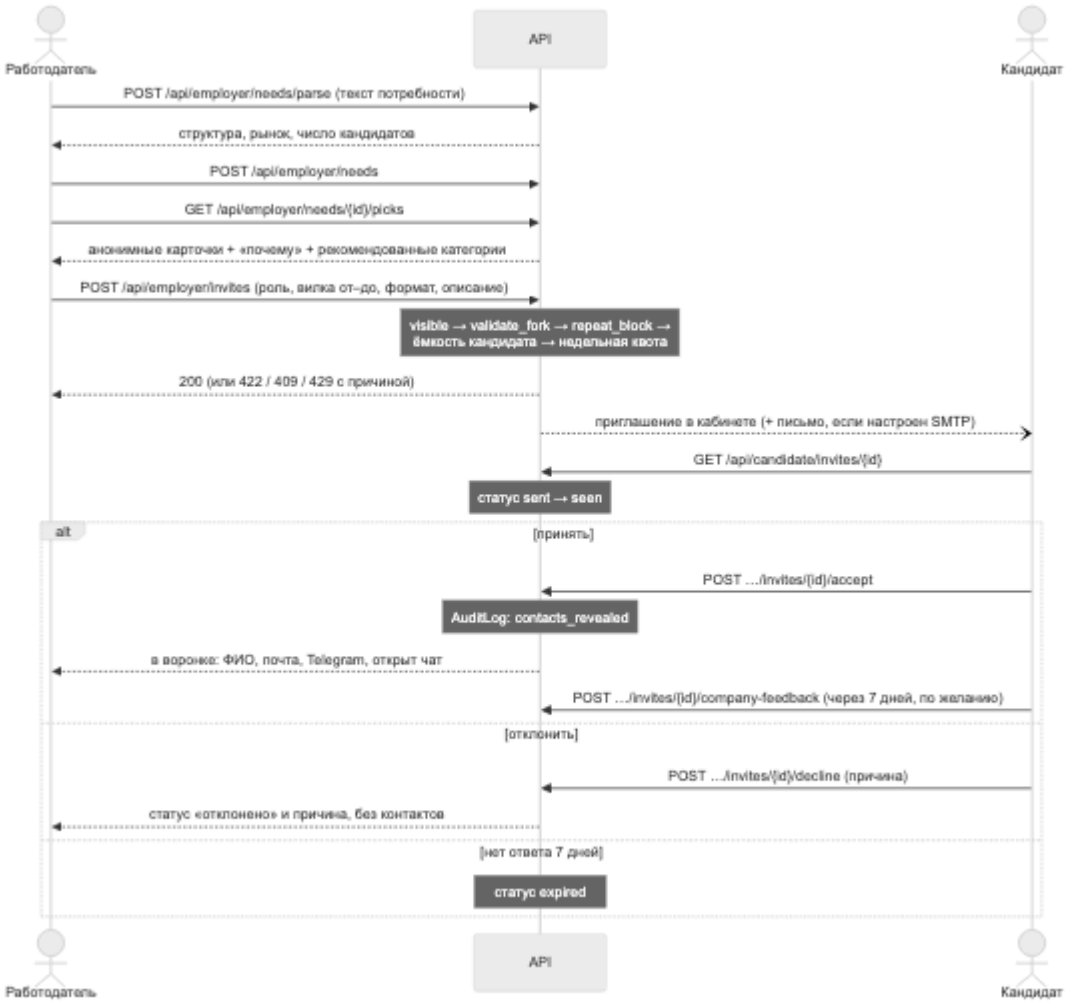
Поиск по банку

`GET /api/employer/search` ищет по всей базе без привязки к потребности. Требование ТЗ — «поиск по банку кандидатов с фильтрацией по специализации, грейду, категории, стеку и наличию достижений ФСП».

- **Фильтры:** специализация, трек, грейд (по тесту), навыки (`skills`), достижения ФСП (`fsp`), Codeforces (`cf`), хакатоны (`hackathon`), формат, город, свободный текст `q`, из которого извлекаются навыки справочника. Если `q` состоит только из ID кандидатов (`A–1042`, несколько — через запятую; любой регистр, кириллические буквы того же начертания, тире из мессенджеров), поиск идёт по ID мимо остальных фильтров (`by_id` в ответе) — работодатель копирует ID из воронки или чата; скрытых, без согласия и без теста так не найти. ID вместе с текстом — обычный текстовый поиск.
- **Ранжирование по силе подтверждённого профиля** (`matching.strength`): θ − 0,5·SE + 0,5·сигнал достижений. Если заданы навыки, добавляется среднее покрытие по ним.
- **Фасеты** по специализации и грейду считаются без собственного фильтра. Работодатель видит, сколько кандидатов будет, если сменить грейд, и уточняет запрос, не теряя остальные условия.
- **Видимость** та же, что в подборке: только кандидаты с пройденным тестом и согласием на публикацию.

3.5. Выход на контакт: приглашение-оффер

Это смысловой центр продукта. Инициатива у работодателя. Кандидат видит условия до начала общения, а контакты раскрываются только с его согласия.



Состав приглашения

Приглашение отправляется через `POST /api/employer/invites`. Привязка к потребности необязательна, как требует ТЗ («без обязательной привязки к опубликованной вакансии»).

Приглашение содержит:

- роль и описание предложения;
- вилку «от–до» в рублях на руки;
- формат («Гибрид, Москва, 2 дня в офисе»);
- необязательное задание вместо первого созвона;
- название компании и способ связи с рекрутером: имя, почта, Telegram. Контакты рекрутера — не ПДн кандидата, поэтому они видны кандидату сразу.

Кандидату показываются факты о компании (`employer.company_trust`): «почта на домене подтверждена» или «не подтверждена», а также, если не сходятся контрольные цифры ИНН, «ИНН: контрольные цифры не сходятся». Если верх вилки ниже минимума кандидата, он видит пометку об этом.

Правила вилки — `employer.validate_fork` и проверки приглашения

Ошибки блокируют отправку (422), предупреждения нет.

Правило	Тип	Зачем
Обе границы указаны	ошибка	ТЗ: вилка «от–до» обязательна в приглашении и вакансии
От ≤ до	ошибка	
Нижняя граница ≥ 25-го перцентиля рынка для специализации, грейда кандидата и города (`catalog.market`)	ошибка	Отсекает заниженные офферы «на всякий случай»
Ширина: (до − от) / от ≤ 40 % (`FORK_MAX_WIDTH`)	ошибка	Вилка «от 100 до 500 тысяч» ничего не сообщает кандидату
Верх вилки ниже минимума кандидата	предупреждение	Работодатель видит оценку: «вероятность принятия упадёт примерно до …»
Кандидат не рассматривает этот формат (проверка в обработчике приглашения, не в `validate_fork`)	предупреждение	

Отдельно, ещё до `validate_fork`, любая сумма от пользователя ограничена 10 млн ₽ в месяц (`catalog.SALARY_MAX`, тип `employer.Salary`): это проверка схемы запроса, ответ — 422 со списком ошибок pydantic «проверьте, нет ли лишних нулей» (формат — раздел 6).

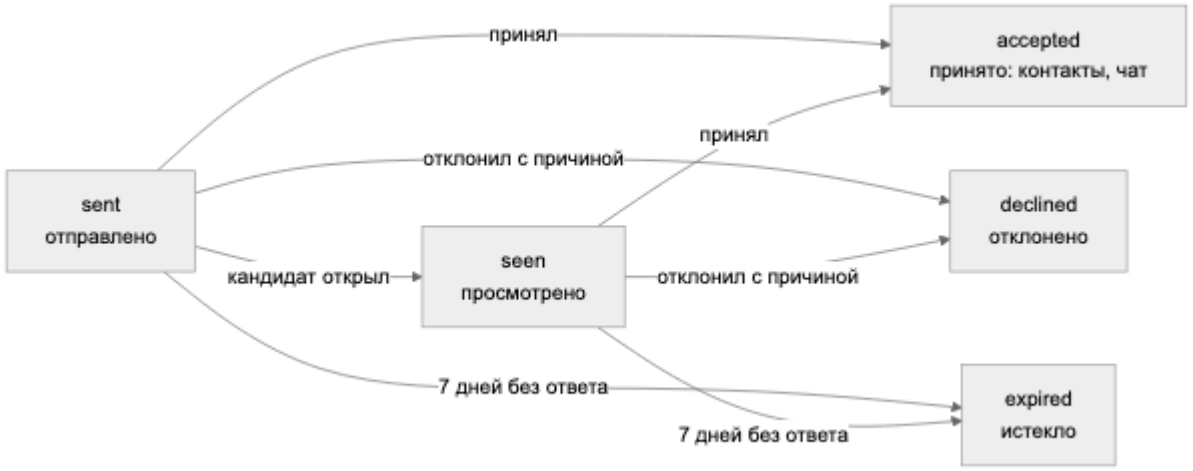
Рыночные перцентили — `catalog.MARKET_MOSCOW`, это таблица по грейду с множителями специализации и города. Для Backend в Москве: Middle — 210 / 260 / 310 тыс. ₽ (p25 / p50 / p75), Senior — 320 / 390 / 460 тыс. ₽. Таблица экспертная, реальных данных рынка в ней нет (`# ponytail` в коде).

Проверки при отправке

Проверки идут в таком порядке, под блокировкой строк компании и кандидата, поэтому параллельные запросы не обходят лимиты:

1. Кандидат видим (`visible`) — иначе 404.
2. Вилка проходит `validate_fork` — иначе 422 со списком ошибок.
3. Повторное приглашение разрешено (`employer.repeat_block`) — иначе 409 с причиной:
    - открытое приглашение от этой компании уже есть — «Кандидат уже приглашён»;
    - кандидат уже принял приглашение по этой потребности — повтор по ней навсегда закрыт. Без потребности или по другой потребности пригласить можно;
    - кандидат отклонил приглашение этой компании — повтор не раньше чем через 30 дней (`REINVITE_AFTER_DECLINE_DAYS`) по любой потребности. Дата показывается в карточке;
    - истёкшее без ответа приглашение не мешает: в карточке есть кнопка «Пригласить снова», квота тратится заново.
4. Ёмкость кандидата: не больше 4 открытых приглашений (`CANDIDATE_CAP`) — иначе 409.
5. Недельная квота компании — иначе 429 «Квота на неделю исчерпана … Обновится в понедельник».

Статусы и срок ответа



- Обе стороны видят актуальный статус: кандидат — в `GET /api/candidate/invites`, работодатель — в воронке `GET /api/employer/pipeline`.
- Срок ответа — 7 дней (`INVITE_TTL_DAYS`). Кандидат видит, сколько дней осталось. Истечение ленивое (`matching.expire_invites`): статус обновляется при обращении к приглашениям кандидата или компании.
- Отказ — с причиной из справочника: «Зарплата», «Стек», «Формат», «Сейчас не ищу», «Компания» (`DECLINE_REASONS`). Компания видит только причину, комментарий кандидата ей не показывается. Отказ по зарплате поднимает эффективный минимум кандидата в следующих подборках (шаг 4).

Раскрытие контактов

- **До принятия** работодатель видит только анонимную карточку.
- **При принятии** (`POST /api/candidate/invites/{id}/accept`) в журнал `AuditLog` пишется событие `contacts_revealed` с идентификаторами приглашения и компании, без ПДн. Работодатель видит в воронке ФИО, почту и Telegram, открывается внутренний чат.
- **Контакты снова скрываются,** если кандидат отозвал согласие на обработку, внёс компанию в «скрыть профиль от» или компанию заблокировали по отзывам. Правило одно для воронки и для чата — `chat.may_talk`. Работодатель видит причину, например «кандидат отозвал согласие».
- Принятие проверяет согласие под блокировкой строки кандидата, поэтому отзыв согласия не проскочит между проверкой и раскрытием.

После принятия

- **Отметка работодателя** — «грейд подтвердился», «не подтвердился» или «наняли» (`POST /api/employer/invites/{id}/feedback`). Первая и третья увеличивают счётчик «подтверждён работодателем» в уровнях доверия.
- **Отзыв кандидата о компании** — через 7 дней после принятия, по желанию и один раз: «всё честно», «не вышли на связь», «условия не совпали», «вакансии не было». На стенде с `DEMO=1` отзыв доступен сразу. Отзыв влияет только на квоту и блок компании, кандидату репутация компании не показывается. Подробно — в разделе 11.
- **Чат** (`routers/chat.py`) открыт, пока приглашение принято. Нужен, чтобы стороны договорились, не обмениваясь контактами вне платформы.

Ёмкость кандидата и квота работодателя

Регулятор	Значение	Где
Ёмкость кандидата	не больше 4 открытых приглашений (sent / seen)	`CANDIDATE_CAP`
Недельная квота компании	10 приглашений; сброс в понедельник 00:00 МСК	`QUOTA_BASE`, `employer.quota`
Бонус квоты	12 в неделю, если в прошлые недели принято ≥ 50 % закрытых приглашений	`QUOTA_BONUS`, `QUOTA_BONUS_AT`
Неподтверждённая компания	3 в неделю, без бонуса	`UNVERIFIED_QUOTA`
Штраф по отзывам кандидатов	3 в неделю, без бонуса	`FEEDBACK_PENALTY_QUOTA` (раздел 11)
Компания заблокирована	0	`reputation.check_block`

Квота и её остаток видны работодателю в кабинете.

3.6. Квалифицированная лотерея

Проблема. Внутри узкого порога кандидаты статистически неразличимы: интервалы θ ± SE перекрываются. Ранжировать их дальше — ложная точность, в которую легко просачиваются предубеждения и знакомства. Вдобавок все работодатели зовут одних и тех же «звёзд» из верха выдачи.

Решение — квалифицированная лотерея (threshold lottery по Frey & Osterloh): детерминированный порог, затем слепой жребий среди прошедших. Код — `services/lottery.py` и `routers/lottery.py`.

1. **Порог.** Работодатель задаёт его от своей потребности (`lottery.effective_threshold`). Специализация, трек, грейд не ниже грейда потребности − 1, обязательные навыки потребности (они должны быть подтверждены тестом), верх вилки и формат берутся из потребности. Клиент может только ужесточить порог: поднять минимальный грейд, добавить θ − SE ≥ X, добавить навыки, снизить верх вилки. Ослабление отклоняется (422).

2. **Кто входит в пул всегда:**

   - кандидат открыт;
   - действуют согласия на обработку и публикацию;
   - тест пройден;
   - компания не в «скрыть от»;
   - по умолчанию у кандидата не заполнена ёмкость.

   Исключаются кандидаты, у которых есть открытое приглашение этой компании, а также те, кто за последние 90 дней отклонил её приглашение или оставил его без ответа (`lottery.COOLDOWN_DAYS`).

3. **Заморозка (`POST /api/lottery/`).** Пул фиксируется как отсортированный список псевдонимов sha256(соль + public_id)[:16]. Соль своя у каждой лотереи и наружу не отдаётся, поэтому псевдонимы не связать с карточками и с другими лотереями. Публикуются sha256(seed) и commitment — sha256 канонического JSON с порогом, пулом, числом мест и хешем зерна.

   Ограничения:

   - пул должен быть больше числа мест — иначе жребий не нужен;
   - пул не больше 500 — иначе порог слишком мягкий;

    - мест от 1 до 50;
    - одна жеребьёвка на потребность. Перебросить жребий до удобного результата нельзя (409).

4. **Жребий (`POST /api/lottery/{id}/draw`).** Зерно seed раскрывается. Порядок — сортировка пула по sha256(seed + псевдоним), берутся первые N.

5. **Приглашение выбранных** идёт обычным потоком (`POST /api/employer/invites` с `lottery_id`), со всеми правилами вилки, ёмкости и квоты. Сама лотерея квоту не тратит. Пригласить по `lottery_id` кандидата, которого не вытянул жребий, нельзя (422).

Как проверить честность. `GET /api/lottery/{id}/public` открыт без авторизации. Он отдаёт порог, псевдонимы пула, хеш зерна, commitment, после розыгрыша — само зерно и выбранных. Любой может проверить:

- sha256(seed) совпадает с хешем, опубликованным до розыгрыша;
- commitment совпадает с порогом, пулом и числом мест;
- сортировка пула по sha256(seed + псевдоним) даёт опубликованных выбранных.

Это и есть `lottery.verify_draw`. Самопроверка модуля — `uv run python -m app.services.lottery`.

Ограничение: оператор платформы технически видит seed до розыгрыша. Следующий шаг — брать зерно из внешнего маяка случайности (drand). Это отмечено в коде.

3.7. Вакансии и отклики (дополнительный сценарий)

ТЗ называет классический отклик дополнительным сценарием: кандидат, пока не получивший приглашений, может проявить инициативу. Реализация — `routers/vacancies.py`.

- **Публикация** — `POST /api/vacancies`. Вилка обязательна и проверяется тем же `validate_fork`, что и приглашение: обе границы, не ниже p25 рынка, ширина ≤ 40 %.
- **Список вакансий** открыт всем. Залогиненному кандидату вакансии сортируются по той же формуле соответствия, что и подборка (`vacancies.fit`), и показывается пометка, если вилка ниже его минимума.
- **Отклик** — только после теста и при действующем согласии на обработку. Мягкий лимит — 10 откликов за 7 дней (`APPLICATIONS_PER_WEEK`), иначе 429 с датой, когда можно откликнуться снова. Повторный отклик на ту же вакансию — 409.
- **Статусы отклика:** sent → seen → accepted | declined. Пока нет ответа, кандидат может отозвать отклик (withdrawn).
- **Контакты — только после принятия.** До ответа компания видит анонимную карточку кандидата (как в поиске) и его заметку, в которой замаскированы ФИО, почта, телефон, Telegram и ссылки. ФИО, контакты и чат открываются, только когда компания приняла отклик («Пригласить на интервью»). Раскрытие пишется в журнал один раз. Просмотр и отказ контактов не открывают. Так фиктивная вакансия не соберёт контакты откликнувшихся.
- **Срок жизни** вакансии — 30 дней (`VACANCY_TTL_DAYS`). Истёкшая скрыта и не принимает отклики. Компания продлевает её кнопкой (`POST /api/vacancies/{id}/extend`).
- **Лимит открытых вакансий:** 2 у неподтверждённой компании, 10 у подтверждённой (`VACANCY_LIMIT_*`). Сверх лимита — 409.
- **«Отвечает на N % откликов».** Кандидат видит долю откликов за 90 дней, которые компания приняла или отклонила. Отозванные не учитываются. Доля показывается от 5 откликов, на меньшем числе она случайна.

3.8. Регуляторы рынка и зачем они нужны

В обратном найме главный риск — перекос внимания. Работодатели видят одну и ту же верхушку выдачи и засыпают сильных кандидатов приглашениями. Те перестают отвечать, а остальные кандидаты не получают ничего. Регуляторы держат рынок в равновесии.

Регулятор	Значение	Против чего
Ёмкость кандидата	4 открытых приглашения; полная ёмкость → множитель 0,3 в выдаче и 409 при отправке	Завал «звёзд»; кандидат отвечает на каждое приглашение, а не игнорирует поток
Штраф за нагрузку	−15 % за каждое открытое приглашение	Плавно разносит приглашения по выдаче ещё до заполнения ёмкости
Недельная квота	10 (12 при доле принятых ≥ 50 %, 3 у неподтверждённой компании)	Массовая рассылка «всем подряд»; бонус поощряет точные приглашения
Срок ответа	7 дней, затем «истекло»	Висящие приглашения не занимают ёмкость бесконечно
Повторное приглашение	30 дней после отказа; никогда — по уже принятой потребности	Навязчивость
Обязательная вилка	≥ p25 рынка, ширина ≤ 40 %	Кандидат видит реальные условия до общения
Взаимный интерес	итог × (0,5 + 0,5·P)	Приглашения, которые почти наверняка отклонят
Лимит откликов	10 в неделю	Отклики «вслепую» перегружают работодателей
Лотерея	порог + жребий	Ложная точность и непотизм внутри неразличимой группы

Результаты симуляции рынка (README «Результаты валидации»; синтетика, seed 42, 4 недели; методика — раздел 4):

	Без лимитов	Только квота	Квота + ёмкость + взаимный интерес
Доля принятых	6,9 %	7,6 %	11,9 %
Охват кандидатов	87,9 %	71,2 %	70,6 %
Коэффициент Джини	0,43	0,54	0,53

Полный набор регуляторов поднимает долю принятых приглашений с 6,9 % до 11,9 %. Без лимитов Джини ниже, потому что приглашений отправляется в 2,4 раза больше (5302 против 2205). При сопоставимом объёме («только квота») ёмкость снижает концентрацию.

Качество выдачи на эталонном наборе пар «потребность — кандидат» (README; эталон и метрики — раздел 4):

Версия подбора	P@10	nDCG@10
Навыки (поиск)	0,253	0,554
+ структурный реранк	0,392	0,699
+ взаимный интерес	**0,393**	**0,732**

P@10 — доля релевантных кандидатов в первой десятке. nDCG@10 — качество порядка первой десятки с учётом степени релевантности (1 — идеальный порядок). Потолок P@10 на этих данных — 0,500, потому что у части потребностей меньше 10 годных кандидатов. Три версии — это варианты `embed`, `rerank` и `full` функции `score_candidates`. Их же можно запросить в API через параметр `variant` у `GET …/picks`.

3.9. Кандидат без истории ФСП

ТЗ требует корректно обрабатывать случай, когда истории ФСП у кандидата нет. Принцип: достижения поднимают кандидата внутри категории, но их отсутствие не штрафуется.

- **Грейд и категория** определяются только тестом. Достижения на них не влияют ни в какую сторону (`integrations/scoring.py`: «Нет достижений → 0 без штрафа»).
- **В соответствии** сигнал достижений весит 0,10, тест и навыки — 0,90. Без достижений слагаемое равно нулю, а не отрицательно. Сильный по тесту кандидат без ФСП обходит более слабого с победой в соревновании. При прочих равных выше тот, у кого есть подтверждённые достижения, — так и требует ТЗ.
- **В карточке** у такого кандидата написано: «Внешних достижений нет. Это не снижает позицию: оценка по тесту».
- **Фильтры `fsp` и `cf`** включает сам работодатель. По умолчанию подборка их не применяет.
- **В синтетике** достижения есть примерно у 30 % кандидатов. Эталон и метрики раздела 4 считаются по всей популяции, включая кандидатов без достижений.

3.10. Ограничения MVP

- Рыночные перцентили и коэффициенты вероятности принятия экспертные, на реальных данных не откалиброваны (`# ponytail` в `catalog.py` и `matching.accept_prob`).
- Качество подбора измерено на синтетике с эталоном, посчитанным правилами по скрытым полям. Экспертной разметки ФСП пока нет (`table.md`, строка «Экспертная разметка „золотых“ пар»). Подробно — раздел 4.
- У кандидата одна категория. Несколько категорий постановщики назвали плюсом, это развитие: отдельный тест и кулдаун на каждую (`table.md`, строка «Несколько категорий у одного кандидата»).
- Релевантность дисциплины ФСП специализации (соревнование по ИБ важнее для специалиста по безопасности) не учитывается. План — множитель для смежных дисциплин без штрафа остальным (`table.md`, строка «Релевантность дисциплины ФСП специализации кандидата»).
- LLM-кэш реранка, подписчики чата и троттлинг живут в памяти одного процесса API (раздел 1).

4. Валидация решения

Готовых датасетов и эталонной разметки на хакатоне нет (ТЗ, п. 2), поэтому мы проверяем решение на собственной синтетике. Валидация отвечает на вопросы двух критериев оценки:

- критерий 1 (механика подбора): какая доля релевантных кандидатов попадает в топ выдачи на наборе пар «вакансия — кандидат», как механика ведёт себя на рынке;
- критерий 2 (механика тестирования): разделяет ли тест слабых и сильных, стабилен ли грейд при пересдаче, насколько дискриминативны задания, устойчив ли тест к LLM-подсказчику.

Процедура целиком в коде: генератор синтетики `backend/seed/generate.py`, эталон релевантности `backend/seed/gold.py`, расчёт метрик `backend/app/services/validation.py`. Результат прогона пишется в `backend/var/validation.json` и показывается на дашборде `/v`. Все цифры раздела взяты из README, раздел «Результаты валидации». Это один прогон на итоговом коде: seed 42, дата засева 2026-10-05.



Синтетические данные

Принцип: сначала структура, потом текст

Генератор сначала создаёт скрытую истинную структуру кандидата, затем то, что видно платформе (результат теста, заявленные навыки), и только после этого тексты: резюме и описания потребностей. Тексты собираются из шаблонов с шумом: синонимы навыков из справочника `catalog.SKILLS` и опечатки. Скрытая правда хранится в поле `Candidate.truth`. Подбор её не читает, её использует только эталон (`seed/gold.py`).

Скрытое поле	Как задаётся
Специализация и трек	по долям `SPEC_P` (Backend 24 %, Frontend 14 %, …), треки — по `TRACK_P` или равномерно
Истинный грейд	по долям `GRADE_P`: Intern 8 %, Junior 25 %, Middle 37 %, Senior 22 %, Lead 8 %
Истинный θ (уровень на шкале IRT)	равномерно в полосе грейда `THETA_BAND`: Intern −2,2…−0,5, Junior −0,5…0,5, Middle 0,5…1,5, Senior 1,5…2,3, Lead 2,3…3,0
Истинные навыки `skills_true`	ядро трека (`CORE`) плюс дополнительные навыки специализации; чем выше грейд, тем больше навыков
Признак «завышенное резюме»	у 10 % кандидатов (`inflated`)

Из скрытой правды выводится то, что видит платформа:

- наблюдаемый θ — истинный θ плюс шум N(0, SE), где SE берётся из диапазона 0,15–0,35; грейд по тесту — правилом θ − SE (`catalog.grade_from_theta`, см. раздел 2). Синтетические кандидаты тест не проходят: их результат сгенерирован сразу с этим шумом;
- подтверждённые тестом навыки — ядро трека и около 65 % остальных истинных навыков;
- заявленные навыки совпадают с истинными. У «завышенных» резюме к ним добавляются 2–4 навыка, которых у кандидата нет, а заявленный грейд выше истинного на 1–2 ступени. Ожидаемая зарплата тоже считается от завышенного грейда;
- достижения: вероятность их иметь растёт с θ (0,25 + 0,08·θ, в пределах 0,1–0,6), место в соревновании зависит от θ плюс шум. Codeforces есть у 20 % кандидатов Backend, ML / DS и Data Eng, у 4 % остальных разработчиков, у нетехнических ролей его нет совсем. Рейтинг CF коррелирует с компетенцией «Алгоритмы» (заложено ρ ≈ 0,6);
- около 9 % кандидатов зарегистрировались, но тест не прошли (функция `untest`). У них нет ни грейда, ни θ, работодателям они не видны;
- прочее: 90 % кандидатов открыты к предложениям, 95 % дали согласие на публикацию, около 3 % скрыли профиль от одной компании. Город, форматы работы и минимальная зарплата (от медианы рынка `catalog.market`) тоже генерируются.

Потребности работодателей бывают трёх типов (поле `Need.kind`). Тип задаётся номером потребности по кругу из девяти:

Тип	Доля	Что проверяет
`normal`	7 из 9	обычная потребность: специализация, трек, грейд (Junior 20 %, Middle 45 %, Senior 28 %, Lead 7 %), обязательные и желательные навыки, вилка между 25-м перцентилем и медианой рынка
`adjacent`	1 из 9	соседняя специализация, в тексте те же слова, что у Backend (Data Engineering, DevOps, Classic ML, QA Automation: Python, PostgreSQL, Kafka…). Ловушка для подбора по ключевым словам
`fork_low`	1 из 9	вилка ниже ожиданий: нижняя граница 65–80 % от 25-го перцентиля рынка. Для большинства кандидатов категории такая вилка ниже их минимума

Объёмы и детерминизм

Полный засев (`seed.generate --reset`): 1500 кандидатов (из них когорта из 30 кандидатов Backend · Python в Москве вокруг демо-потребности «Ладога Пэй») и 250 синтетических потребностей от 60 компаний. Из 250 потребностей 28 типа `adjacent`, 28 типа `fork_low` и 194 типа `normal`. Отдельно засеваются демо-персонажи, приглашения, вакансии, отклики и задания.

Синтетика детерминирована по `--seed` (по умолчанию 42) и `--today`. По умолчанию `--today` равен `DEMO_TODAY` = 2026-10-05: от полудня по Москве этого дня отсчитываются все сроки (приглашения, неделя квоты, даты тестов и достижений). Валидация считает открытые приглашения и нагрузку кандидатов на ту же дату через `matching.CLOCK`, а не по реальным часам. Поэтому цифры воспроизводятся в любой день. Тест `test_same_numbers_any_day` (`backend/tests/test_validation.py`) проверяет, что прогон «через 40 дней» и засев от другого дня недели дают те же метрики. Генераторы случайных чисел самой валидации тоже фиксированы (`seed` плюс имя эксперимента).

Набор пар «вакансия — кандидат» и эталон релевантности

Что такое набор пар

Тестовый набор пар (ТЗ, критерий 1; постановщики на QA-сессии попросили его описать) — это все пары «синтетическая потребность × допустимый кандидат». Для каждой из 250 потребностей берётся пул кандидатов, которых подбор вообще может показать работодателю (`matching.eligible`, фильтр категории, см. раздел 3):

- та же специализация;
- грейд по тесту равен грейду потребности или на один ниже. На один выше — только если минимум кандидата укладывается в верх вилки;
- кандидат открыт к предложениям, дал действующее согласие на публикацию, прошёл тест и не скрыл профиль от этой компании.

Каждой паре в пуле эталон ставит оценку 0–3. Подбор ранжирует этот же пул, и метрики сравнивают его порядок с эталоном. Потребности, в пуле которых нет ни одного кандидата с оценкой ≥ 2, из метрик исключаются: искать там нечего, и P@k для них не определена.

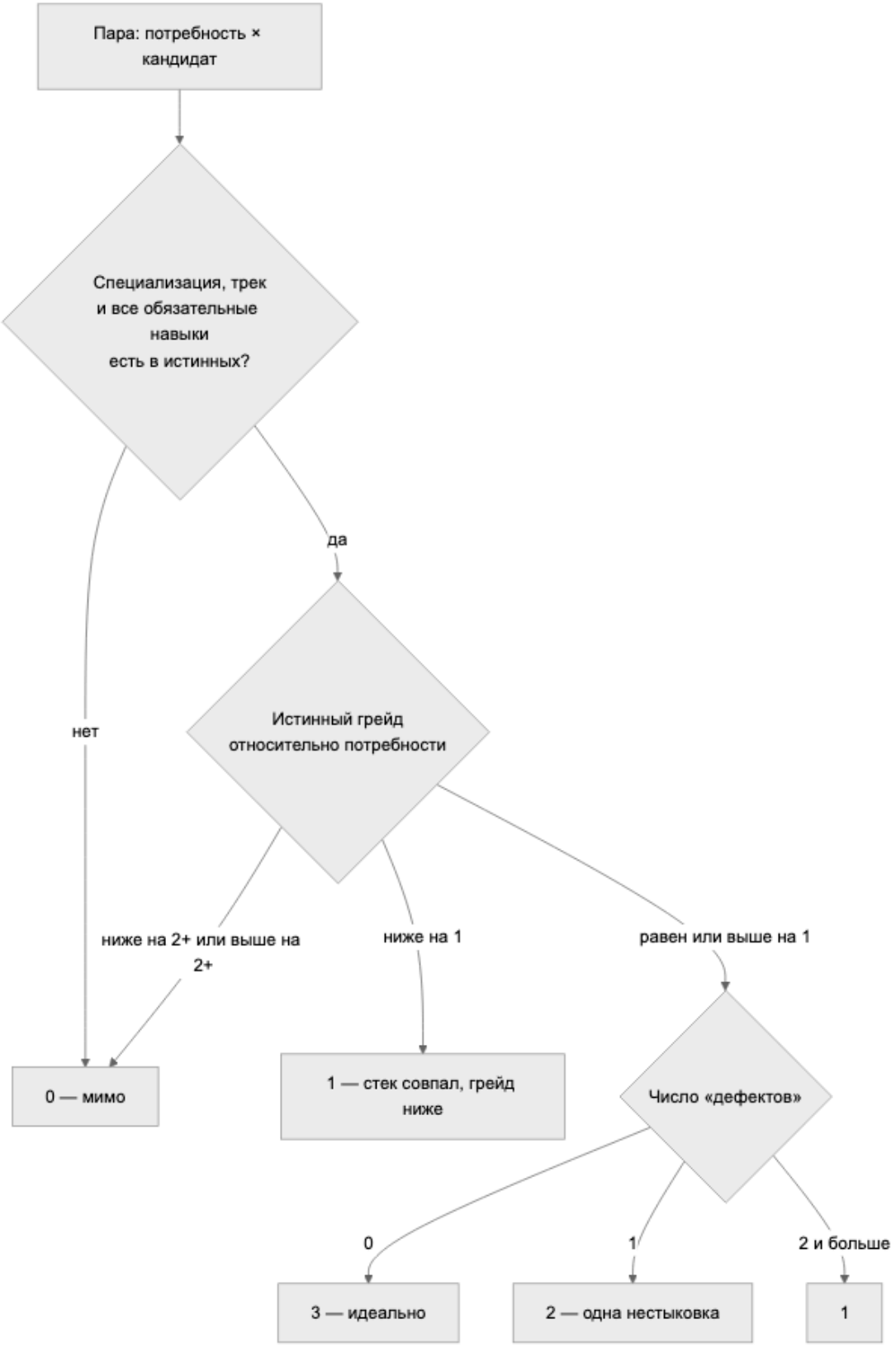
Объём набора в прогоне из README (seed 42, дата засева 2026-10-05):

	Количество
Потребностей всего	250
Оценено (в пуле есть кандидат с оценкой ≥ 2)	206
Пропущено (годных нет)	44
Размеченных пар всего	20 907
Пар в оценённых потребностях	16 567
Оценка 0 / 1 / 2 / 3	17 228 / 2 484 / 884 / 311

Счётчики потребностей пишутся в `validation.json` (`needs_evaluated`, `needs_skipped`) и в отчёт прогона; число пар и распределение оценок посчитаны тем же `seed.gold.relevance` по пулам `validation._pools`.

Как размечаются пары

Оценку ставит функция `seed.gold.relevance` по скрытой правде кандидата (`truth`: специализация, трек, истинный грейд, истинные навыки), а не по тому, что он заявил. Иначе «завышенные» резюме попадали бы в релевантные. LLM в разметке не участвует.



«Дефекты» — это каждый недостающий желательный навык, минимум кандидата выше верхней границы вилки и несовместимый формат работы или город. Офис и гибрид требуют того же города, удалёнке город не важен. Кандидат считается релевантным при оценке ≥ 2 (`REL_TH`). Правило проверяется самопроверкой модуля (`uv run python -m seed.gold`) на парах-контрпримерах: соседняя специализация → 0, нет обязательного навыка → 0, вилка ниже ожиданий → 2, вилка ниже и не тот формат → 1, гибрид в другом городе → 2, удалёнка в другом городе → 3.

Почему коэффициенты не подгоняются под эталон

Эталон считается правилами по скрытым полям той же синтетики. Если перебирать веса подбора (`W_THETA`, `W_COVER`, штрафы за грейд и трек и т. п.) до максимума P@10 на этом эталоне, подбор выучит правила разметки, а не свойства рынка. Получится замкнутый круг: высокая метрика будет означать только то, что подбор повторяет наши же правила. Поэтому в правилах репозитория (`CLAUDE.md`) прямо запрещено подгонять коэффициенты перебором по эталону. Веса задаются из содержательных соображений (см. раздел 3). Каждое изменение подбора, теста или синтетики сопровождается перезасевом отдельной базы и полным перепрогоном валидации, и все цифры README обновляются из одного прогона.

Подбор и эталон намеренно смотрят на разные данные. Эталон видит истинные навыки и истинный грейд. Подбор видит только наблюдаемое: θ ± SE, навыки, подтверждённые тестом (с полным весом), и заявленные (с весом 0,4). Расхождение между ними и есть то, что измеряет валидация.

Метрики подбора

Определения

- **P@10 (Precision@10)** — доля релевантных (оценка ≥ 2) среди первых 10 кандидатов выдачи. Знаменатель всегда 10: если выдача короче, это считается промахом. Это и есть «доля релевантных кандидатов в топе выдачи» из критерия 1.
- **nDCG@10** (нормированный дисконтированный совокупный выигрыш) учитывает и порядок, и степень релевантности. Выигрыш позиции равен 2^оценка − 1 с логарифмическим дисконтом по позиции. Нормировка идёт на идеальный порядок лучших 10 оценок среди всего допустимого пула, а не только выданных кандидатов.
- **Потолок P@10** — лучшая достижимая P@10: для каждой потребности min(10, число релевантных) / 10, усреднённое по потребностям.

Сравниваются три версии подбора (`matching.score_candidates`, параметр `variant`). Каждая следующая добавляет один этап конвейера:

Версия	Что делает
Навыки (поиск)	только отбор: косинусная близость вектора навыков потребности и кандидата (с эмбеддингами, если они есть, но в валидации LLM выключена, поэтому отбор идёт по навыкам)
+ структурный реранк	соответствие: θ с поправкой на SE, покрытие обязательных и желательных навыков, достижения; штрафы за грейд ниже и чужой трек
+ взаимный интерес	итог = соответствие × (0,5 + 0,5·P(принятия)) × поправка на нагрузку кандидата

Результаты

Версия подбора	P@10	nDCG@10
Навыки (поиск)	0,253	0,554
+ структурный реранк	0,392	0,699
+ взаимный интерес	**0,393**	**0,732**

Потолок P@10 на этих данных — 0,500: у части потребностей меньше 10 годных кандидатов. Структурный реранк поднимает P@10 с 0,253 до 0,392. Взаимный интерес почти не меняет P@10, но заметно поднимает nDCG@10 (0,699 → 0,732): внутри топа выше оказываются кандидаты, которые с большей вероятностью примут приглашение. Кандидаты на грейд выше потребности попадают в выдачу, если вилка покрывает их ожидания.

9 % синтетики тест не проходили. Работодателям они не видны и в пулы подбора не попадают: в прогоне таких кандидатов во всех пулах 0 (счётчик `untested_in_pool`). Поэтому и эталон, и потолок считаются только по прошедшим тест. Коэффициенты под эталон не подгонялись.

На дашборде `/v` дополнительно показан nDCG@10 по типам потребностей (`normal`, `adjacent`, `fork_low`). Он показывает, как подбор справляется с ловушками соседней специализации и низкой вилки.

Симуляция рынка

Точность выдачи не показывает, что станет с рынком, если все работодатели будут приглашать по ней. Симуляция (`validation.market`) прогоняет работодателей-агентов (синтетические потребности) на 4 недели:

- каждая потребность шлёт до 10 приглашений в неделю. Потребности ходят по кругу по одному приглашению, чтобы живая нагрузка кандидатов влияла на всех;
- кандидат за неделю отвечает максимум на 4 приглашения (`catalog.CANDIDATE_CAP`), начиная с лучшей вилки. Он принимает приглашение, если вилка не ниже его минимума и формат подходит, с вероятностью `matching.accept_prob`. Принятое приглашение закрывает потребность;
- сравниваются три режима: «без лимитов» (порядок по соответствию, квоты нет), «только квота» (тот же порядок плюс недельная квота компании 10 приглашений, 12 при доле принятых от 50 %) и «квота + ёмкость + взаимный интерес» (итоговый порядок продукта, живая нагрузка, у кандидата не больше 4 открытых приглашений, как в продукте).

	Без лимитов	**Только квота**	**Квота + ёмкость + взаимный интерес**
Доля принятых	6,9 %	7,6 %	11,9 %
Охват кандидатов	87,9 %	71,2 %	70,6 %
Коэффициент Джини	0,43	0,54	0,53

Охват — доля кандидатов, получивших хотя бы одно приглашение. Считается по кандидатам, которых работодатели видят (открыт, есть согласие на публикацию, тест пройден). Коэффициент Джини показывает, насколько неравномерно приглашения распределены между кандидатами: 0 — поровну, 1 — все у одного. Без лимитов Джини ниже, потому что приглашений отправляется в 2,4 раза больше (5302 против 2205). При сопоставимом объёме («только квота») ёмкость снижает концентрацию. Полный набор регуляторов (ёмкость кандидата, учёт его текущей нагрузки и порядок по взаимному интересу) поднимает долю принятых приглашений с 7,6 % до 11,9 % при почти том же охвате. Вклад каждого регулятора по отдельности симуляция не разделяет.

Валидация теста

Термины. **IRT** (Item Response Theory, модель 2PL) описывает вероятность верного ответа через уровень кандидата θ и два параметра задания: дискриминативность a (крутизна кривой) и сложность b. **EAP** — оценка θ как среднего апостериорного распределения. **SE** — стандартная ошибка этой оценки.

Грейд присваивается, если θ − SE не ниже порога грейда. Механика подробно описана в разделе 2.

Тест проверяется симуляцией: симулированные кандидаты с известным истинным θ отвечают на задания реального банка по модели 2PL, а сессия идёт тем же кодом, что в продукте (`irt.choose`, `irt.eap`, `irt.should_stop`).

Разделяющая сила: отличает ли тест один грейд от другого

200 симулированных кандидатов на каждый грейд, банки всех 12 специализаций по кругу (`validation.separation`). Истинный θ берётся равномерно из полосы грейда, как в синтетике. Сессия идёт как в продукте (полный тест, со слотом задачи на код): выбор заданий начинается у порога заявленного грейда, итог — EAP с нейтральным априором, грейд — по θ − SE. Заявленный грейд в симуляции равен истинному, завышенные заявки не моделируются, так что оценка оптимистичная. В ячейках — доля строки:

Истинный грейд → по тесту	Intern	Junior	Middle	Senior	Lead
Intern	**98,5 %**	1,5 %	0	0	0
Junior	39 %	**59,5 %**	1,5 %	0	0
Middle	0	40 %	**58,5 %**	1,5 %	0
Senior	0	1 %	56 %	**42,5 %**	0,5 %
Lead	0	0	5,5 %	65 %	**29,5 %**

Метрика	Значение
Точное совпадение грейда	57,7 %
То же, если грейд считать по самой оценке θ без запаса SE	75 %
В пределах ±1 грейда	99 %
Доля ошибок, приходящихся на соседний грейд	96,9 %
κ (каппа Коэна) с квадратичными весами	0,877
AUC оценки θ между соседними грейдами	Intern|Junior 0,966, Junior|Middle 0,948, Middle|Senior 0,919, Senior|Lead 0,914

Каппа Коэна с квадратичными весами измеряет согласие истинного и тестового грейда с поправкой на случайное совпадение. Далёкие ошибки штрафуются сильнее соседних, 1 — полное согласие. AUC — вероятность того, что у кандидата старшего грейда оценка θ выше, чем у младшего; 0,5 означает, что тест их не различает.

Ошибки почти только вниз: грейд выше истинного получают не больше 1,5 % в строке. Правило θ − SE осторожное и грейд не завышает. Senior и Lead чаще уходят на грейд ниже: при SE ≈ 0,3 правилу θ − SE нужен запас над порогом. Это осознанная защита от завышения, и кандидат может пересдать тест позже (см. раздел 2 и `table.md`).

По специализациям разница небольшая. В каждом банке 28–34 семейства заданий (Backend — 34; у специализаций без «Алгоритмов» задач на код нет — 28, так и у четырёх нетехнических: Analysis, Management, Design, SysAdmin / Support). Сессия идёт 12 заданий, SE 0,32–0,33. Точное совпадение — 49–66 %, ±1 грейд — 97,5–100 % (200 симулированных кандидатов на грейд; разброс между специализациями в пределах шума симуляции и от прогона к прогону при правке банков меняется на ±10 п. п.). У нетехнических специализаций точное совпадение 52–60 %, ±1 — 97,5–100 %.

Стабильность при пересдаче (test-retest)

200 симулированных кандидатов с θ ~ N(0, 1) проходят тест Backend дважды: на других клонах заданий и по другой траектории (`validation.stability`).

Метрика	Значение
Грейд в профиле после пересдачи не изменился	88,5 %
Сырая оценка θ − SE дала тот же грейд	76 %
Корреляция истинного и оценённого θ	0,95

Грейд в профиле стабильнее сырой оценки, потому что повышение идёт по θ − SE, а принудительного понижения нет (см. раздел 2).

Внешняя проверка: коэффициент Спирмена между компетенцией «Алгоритмы» и рейтингом Codeforces — 0,56 (n = 120, только прошедшие тест; у нетехнических ролей CF в синтетике нет). Эта связь заложена в генератор (см. «Синтетические данные»), поэтому метрика проверяет, что сквозной путь данных её не теряет. Внешней валидацией теста она не является: для неё нужны рейтинги CF у кандидатов, реально прошедших тест.

Дискриминативность заданий

Насколько задания разделяют слабых и сильных (`validation._items`):

- 1912 заданий. Параметр a: медиана 2,19 (Q1–Q3 1,84–2,29), у всех a ≥ 1,44, то есть дискриминативность «высокая» по шкале Baker (2001), где высокая начинается от 1,35;
- точечно-бисериальная корреляция ответа на задание с суммой остальных ответов (r_pb item-rest) на калибровочной выборке из 500 симулированных кандидатов: медиана 0,48, слабых заданий (r < 0,2) — 1,9 %. В адаптивной сессии у разных людей разные задания, поэтому классическая item-total там не определена. Выборка отвечает на все задания банка.

Параметры a и b заданы при составлении банка и не откалиброваны на живых ответах. Поэтому r_pb на ответах, смоделированных по той же 2PL, проверяет только согласованность банка, а не его калибровку.

LLM-читер и детекция несогласованности

Сценарий: кандидат проходит тест с подсказками языковой модели. Реальная LLM не запускается. Её профиль ответов смоделирован (`validation._cheater_p`, вероятности заданы руками):

- вопросы с вариантами без контекста модель решает с вероятностью 0,9 независимо от сложности;
- на ревью кода с тонкими багами она отвечает как человек с θ = −0,5;
- на задачах с контекстом (логи, графики) — как человек с θ = −0,3;
- задачи на код решает с вероятностью 0,95.

200 сессий на каждый тип заданий, банк Backend:

Тип заданий	Грейд
Вопросы с вариантами	Senior
Ревью кода	Intern
Задачи с контекстом	Intern

Тип заданий	Грейд
Задачи на код (песочница)	Junior
Смешанный адаптивный тест	Junior

Тест только из вопросов с вариантами LLM прошла бы на Senior. Смешанный тест, как в продукте, даёт ей Junior: ревью кода и задачи с контекстом модель решает плохо. Задачи на код LLM решает почти всегда: в модели читера — с вероятностью 0,95 независимо от сложности. Тест только из них дал бы Junior с $\theta \approx +0{,}95$, а выше его держат лишь пять семейств и большая SE. Поэтому вес задач на код умеренный: одна задача из 6–12 заданий сессии, $a = 1{,}5$–$1{,}7$ против 1,9–2,5 у сценарных заданий. Слишком быстрые и вставленные из буфера верные решения помечаются флагами для разбора. В экспресс-тесте задач на код нет.

Проверка на несогласованность (person-fit статистика l_z, Drasgow et al., 1985; флаг `inconsistent` при $l_z < -1{,}96$, `irt.suspicion_flags`) ловит 30 % читеров при 2 % ложных тревог у 200 честных симулированных кандидатов. Без задач на код на той же синтетике было 45 % и 0 %. Задача на код, которую читер решает, по сложности совместима с его средним θ и вытесняет задание, на котором его профиль ответов расходится сильнее (`table.md`, ━). Флаг не меняет грейд, а только отправляет попытку на разбор. Устойчивость теста к утечке заданий (параметрические клоны) описана в разделе 2.

Как воспроизвести

Нужен Postgres (`docker compose up -d db`). Валидацию запускают на отдельной базе, чтобы не трогать демо-данные (команды из `CLAUDE.md`):

```
cd backend
DATABASE_URL=postgresql+psycopg://fsp:fsp@localhost:5432/fsp_valid uv run python -m seed.generate --reset
DATABASE_URL=postgresql+psycopg://fsp:fsp@localhost:5432/fsp_valid uv run python -m app.services.validation
```

Валидация печатает отчёт в консоль и пишет `backend/var/validation.json`. Параметры: `--k 10`, `--seed 42`, `--today 2026-10-05|now`. Дата `--today` должна совпадать с датой засева. Прогон на 1500 кандидатах занимает около 40 с. Он ничего не пишет в базу, кроме засева банка заданий, если банк пуст. На время прогона LLM-реранк и эмбеддинги выключены: метрики детерминированы и меряют структурный реранк.

На развёрнутом стенде в Docker:

```
docker compose exec api python -m seed.generate --reset --today now
docker compose exec api python -m app.services.validation --today now
```

Дашборд `/v` (http://localhost:8000/v) открывается без входа по прямой ссылке. Он показывает последний прогон: качество выдачи по версиям и по типам потребностей, симуляцию рынка, LLM-читера, стабильность, матрицу разделения грейдов, AUC соседних грейдов, разбивку по специализациям и распределения a и r_{pb}. API:

- `GET /api/validation` — последний `validation.json`, 404, если прогона ещё не было;
- `POST /api/validation/run?k=10&seed=42&today=now` — пересчёт. При `DEMO=1` он доступен без входа: на `/v` это кнопка «Запустить прогон» / «Пересчитать». Вне демо пересчитывать может только организатор. Без синтетики в базе запрос вернёт 409, второй одновременный прогон — тоже 409.

Метрики и сама процедура покрыты тестами `backend/tests/test_validation.py`: метрики на ручных примерах, фиксированные часы, форма результата, одинаковые цифры в любой день. Правило эталона проверяет самопроверка `uv run python -m seed.gold`.

Ограничения

- **Только синтетика.** Распределения навыков, зарплат и достижений заданы генератором, а не измерены на рынке. Цифры показывают, что механика работает на модели рынка, а не на реальных кандидатах и вакансиях.
- **Эталон написан той же командой.** Правила эталона и подбора смотрят на разные поля (скрытые и наблюдаемые), но опираются на общие допущения о предметной области: грейд ниже на ступень — частичная релевантность, вилка и формат — «дефекты». Подгонку коэффициентов под эталон мы запретили, но общее смещение допущений этим не убирается.
- **Нет экспертной разметки.** Разметки экспертов ФСП пока нет (`table.md`, «Экспертная разметка „золотых“ пар с ФСП (каппа Коэна)»). План: эксперты ФСП размечают выборку пар «потребность — кандидат» по той же шкале 0–3, считается каппа Коэна между экспертами и между экспертами и правилом эталона. Поля `gold_experts` и `kappa` уже есть в `validation.json`, на `/v` сейчас выводится «разметка экспертов: нет».
- **Экспертные a и b.** Параметры заданий заданы при составлении банка и не откалиброваны. Калибровка (MML/EM) — когда наберётся около 300 ответов на семейство, в том числе по пилотным заданиям (`table.md`). Разброс сложности клонов внутри семейства (0,05) тоже задан, а не измерен.
- **Симуляции оптимистичны.** В матрице разделения заявленный грейд равен истинному. Модель читера — руками заданные вероятности, а не доли верных ответов реальной LLM. Синтетические кандидаты подбора получают θ сразу с шумом, а не через симулированную сессию.
- **Симуляция рынка использует ту же модель принятия, что и подбор.** Кандидаты-агенты принимают приглашения с вероятностью `matching.accept_prob`, и тот же `accept_prob` подбор использует как «взаимный интерес». Поэтому прирост доли принятых отчасти заложен конструкцией. Коэффициенты `accept_prob` подобраны руками; план — дообучить логистическую регрессию на реальных принятиях и отказах.
- **Категоризация по специализации и стеку отдельно не измеряется.** Валидация проверяет грейд (матрица разделения, пересдача). Специализацию и трек кандидат выбирает сам, а тест специфичен для специализации (свой банк заданий). Ошибка выбора специализации кандидатом в синтетике не моделируется (заявленная специализация совпадает со скрытой) и не измеряется: кандидат с неверной специализацией попадает не в тот пул. Потребности типа `adjacent` проверяют другое — что подбор не путает соседние специализации по ключевым словам. Точность подтверждённых тестом навыков (`skills_confirmed`) не измерялась — это следующий шаг вместе с экспертной разметкой.
- **LLM не входит в измерение.** Валидация идёт без модели, поэтому вклад LLM-реранка и эмбеддингов в качество выдачи не измерен (см. раздел 10).

5. Интеграция с реестром ФСП

ТЗ не даёт ни API, ни структуры данных реестра ФСП: команда сама решает, какие сведения о достижениях получать и как показывать их работодателю. Обязательно одно — корректно обрабатывать кандидата без истории ФСП. В этом разделе описано, что реализовано в MVP, какие данные мы берём и почему, и что понадобится от ФСП для боевой интеграции.

Коротко о подходе. Любой источник достижений — реестр ФСП, протокол хакатона, Codeforces, CTFtime — сводится к одной модели: **событие** (соревнование) и **строки протокола** (кто из кандидатов какое место занял). Запись в базу у всех источников идёт через две общие функции, `common.upsert_event` и `common.upsert_achievement` (`backend/app/integrations/common.py`), а вес считается только функциями `backend/app/integrations/scoring.py`. Реестр ФСП подключается как ещё один такой источник. Для этого не нужно менять подбор: меняется только то, откуда приходят строки.

Принципы

Принцип	Как реализовано
Достижения влияют только на место внутри категории, но не на грейд	Категорию и грейд задаёт только тест (см. раздел 2). Достижения входят в оценку соответствия с весом 0,10 и в силу профиля при поиске по банку, но кандидата в другую категорию не переносят (см. ниже «Влияние на ранжирование»)
Нет достижений — нет штрафа	Сигнал кандидата без достижений равен 0. Отрицательного слагаемого в формуле нет, поэтому оценка такого кандидата определяется тестом и навыками. Работодатель видит в карточке: «Внешних достижений нет. Это не снижает позицию: оценка по тесту»
В ранжирование идут только подтверждённые данные	В сигнал входят только достижения с `verified = true` (`matching.achievement_signal`). Подтверждают их доверенные источники: протокол организатора, ФСП ID, официальное API Codeforces после проверки владения. Заявленное кандидатом участие в CTF-команде без подтверждения организатором в сигнал не входит
Иерархии соревнований нет	По итогам QA-сессии с постановщиками все соревнования считаются равно важными. Вес зависит только от результата кандидата в соревновании, уровень события хранится как факт и на вес не влияет
Отдельное согласие на внешние данные	Без действующего согласия `consent_external` (152-ФЗ) ничего из внешних источников не записывается, а отзыв согласия удаляет всё полученное (см. «Согласие на внешние данные»)

Модель данных

Схема — `backend/app/models.py`.

Событие (`Event`, уникально по паре `source` + `external_id`):

Поле	Смысл	Влияет на вес
`source`	`fsp`, `hackathon`, `ctftime` или `codeforces`	нет
`external_id`	идентификатор события в источнике: номер в реестре ФСП, id события CTFtime, id контеста Codeforces	нет
`title`, `date`, `url`, `tags`	название, дата, ссылка на протокол, направления	нет
`level`	`international`, `national`, `regional` или `local`, только описательно	нет
`discipline`	дисциплина ФСП из справочника `catalog.DISCIPLINES`: продуктовое программирование, алгоритмическое программирование, программирование систем информационной безопасности, программирование беспилотных авиационных систем, программирование робототехники (синонимы вроде «БАС» и «кибербезопасность» распознаются)	нет
`participants`	число участников	косвенно: нужно, чтобы определить «финалиста» по месту, если результат в протоколе не указан

Достижение (`Achievement`, одно на пару «кандидат — событие»):

Поле	Смысл
`place`, `team`, `role`	место, команда, роль в команде
`result`	результат: `winner` (победитель), `prize` (призёр), `finalist` (финалист), `participant` (участник отбора)
`sport_rank`	спортивный разряд или звание по протоколу (`catalog.SPORT_RANKS`: ЗМС, МС, КМС, 1–3 разряд, 1–3 юношеский разряд); на вес не влияет
`weight`	вес $\in [0, 1]$
`verified`	подтверждено доверенным источником

Почему именно эти данные

- **Результат и место** — главный сигнал: они показывают, насколько кандидат выделился среди других участников того же соревнования. Именно это работодатель хочет знать о «спортивном» опыте.
- **Число участников** нужно, чтобы отличить финалиста от простого участника, когда организатор не указал результат явно: финалист — место не ниже `participants // 10`, то есть топ-10 % таблицы (`scoring.FINALIST_SHARE`).
- **Дисциплина и разряд** — факты для работодателя: он видит направление и квалификацию по ЕВСК. В вес они не входят, потому что дисциплины не ранжируются друг относительно друга.
- **Уровень и дата** хранятся для карточки. Уровень не входит в вес по решению постановщиков, дата в вес тоже не входит (давность результата пока не учитывается).
- **Команда и роль** — контекст: что кандидат делал в команде (backend, ML, капитан).

ФИО и контакты участников из протокола мы не храним: строка сопоставляется с кандидатом по почте входа или ФСП ID, после чего остаются только идентификатор кандидата и поля выше.

Вес и сводный сигнал

Вес одного соревнования — `scoring.place_weight`: результат из протокола важнее вывода по месту.

Результат	Как определяется, если в протоколе пусто	Вес
победитель	1-е место	0,8
призёр	2–3-е место	0,6
финалист	место в пределах первых 10 % участников	0,35
участник отбора	всё остальное: место ниже, нет места или ниже 3-го места при неизвестном числе участников	0,05

У победы вес 0,8, а не 1,0: остаётся запас под повторяемость результата и рейтинг, и сигнал не насыщается от одной строки протокола.

Сводный сигнал кандидата — `scoring.signal` (через `scoring.combine`). Соревнования сортируются по весу, k-е по силе (k = 0, 1, …) входит с множителем $0{,}5^k$, дальше вклады объединяются по правилу noisy-OR: $1 - \prod(1 - w_k \cdot 0{,}5^k)$. Число соревнований учитывается с убывающей отдачей, поэтому онлайн-явками профиль не набить. Сколько ни набирай участий с весом 0,05, их вклад не превысит ≈ 0,1 — меньше одного выхода в финал (0,35). Одна победа и два призовых места дают ≈ 0,88.

Рейтинг Codeforces — отдельное слагаемое без дисконта: он уже обобщает сотни раундов. Вес рейтинга — `(рейтинг − 1200) / 1800`, ограниченный отрезком [0, 1]: 1200 — стартовый рейтинг новичка, 3000 — legendary grandmaster (`scoring.rating_weight`). Итог: `1 − (1 − сигнал соревнований)·(1 − вес рейтинга)`. Самопроверка модуля — `uv run python -m app.integrations.scoring`, тесты — `backend/tests/test_integrations.py::test_weights_sane`.

Влияние на ранжирование и показ работодателю

Подборка по потребности (`services/matching.py`, подробно в разделе 3). Сначала фильтр категории: специализация и грейд по тесту. Только внутри отобранного пула считается соответствие:

`match = (0,45·θ-оценка + 0,45·покрытие навыков + 0,10·сигнал достижений) × штрафы за грейд и трек`

(`W_THETA`, `W_COVER`, `W_ACH`). Поэтому достижения поднимают кандидата внутри категории, но не переводят его в другую категорию и не меняют грейд. Кандидату без достижений вклад 0, без штрафа.

Поиск по банку (`GET /api/employer/search`). Сила профиля — `θ − 0,5·SE + 0,5·сигнал` плюс покрытие навыков, если они заданы (`matching.strength`). Здесь θ — оценка уровня кандидата по тесту в модели IRT, SE — её стандартная ошибка.

Что видит работодатель (анонимная карточка `matching.card`, только подтверждённые достижения):

- `badges` — лучшее достижение по каждому источнику (ФСП, хакатон, CTF, Codeforces): название, место, результат, дисциплина, разряд; на экране — бейджи с подсказкой;

- `fsp` — лучшее подтверждённое достижение ФСП (строка), `sport_rank` — высший разряд по протоколам ФСП, `cf` — рейтинг Codeforces;
- в объяснении оценки (`explanation.parts`) — вклад достижений `achievement` и веса формулы, чтобы было видно, сколько дал этот сигнал;
- фильтры: в подборке — «есть ФСП» и «есть Codeforces» (`filters=fsp,cf`), в поиске по банку — `fsp`, `cf`, `hackathon` (требование ТЗ о фильтре «наличие подтверждённых достижений ФСП»);
- если достижений нет — фраза «Внешних достижений нет. Это не снижает позицию: оценка по тесту».

Кандидат в своём профиле (`GET /api/achievements`) видит все свои достижения по источникам, их веса, отметку «подтверждено» и сводный сигнал. Сигнал там считается так же, как в подборе, — только по подтверждённым.

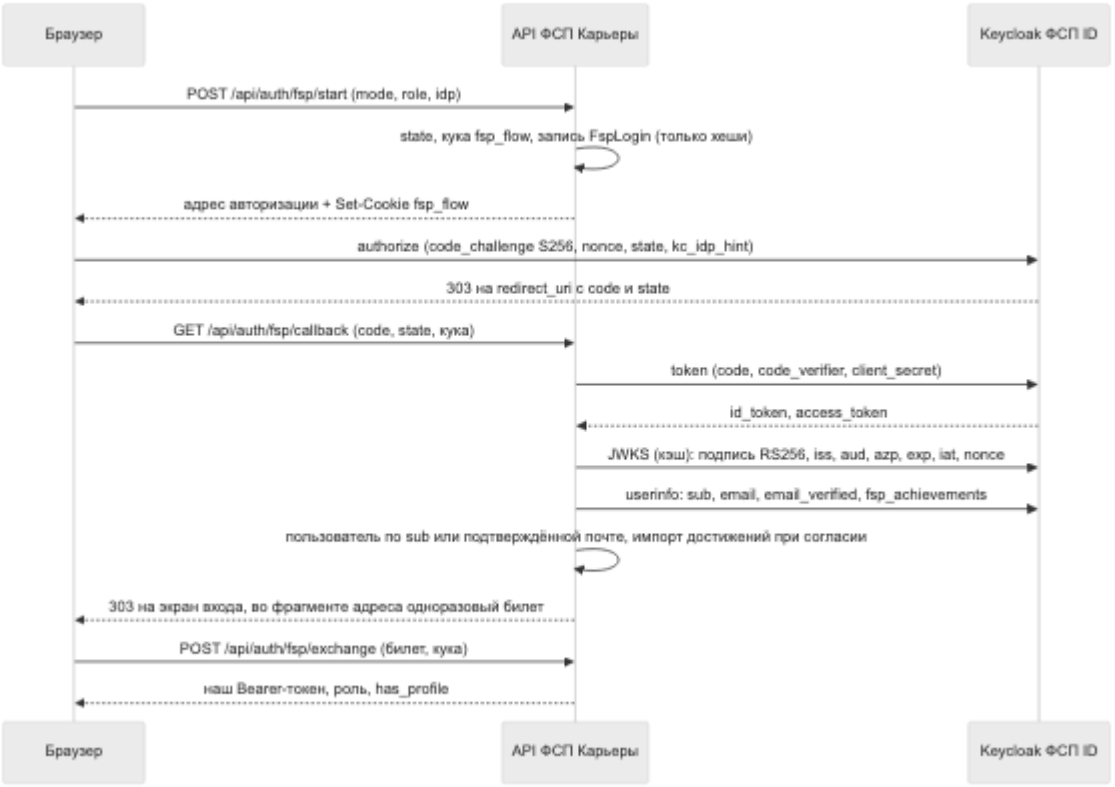
Каналы поступления данных



Канал	Источник (`source`)	Кто подтверждает	Где в коде
ФСП ID	`fsp`	реестр ФСП через Keycloak (claim)	`integrations/fsp_id.py`, `routers/fsp_id.py`
Импорт протоколов	`fsp`, `hackathon`, `ctftime`	организатор (`ORGANIZER_TOKEN` или роль `admin`)	`integrations/hackathons.py`, `routers/achievements.py`, страница `/org`
Codeforces	`codeforces`	официальное API после проверки владения	`integrations/codeforces.py`
CTFtime	`ctftime`	только организатор: импорт протокола с `source=ctftime`	`integrations/ctftime.py`

ФСП ID: вход через Keycloak по OpenID Connect

ТЗ говорит, что ФСП ID будет работать на базе Keycloak. Мы реализовали настоящий клиент OpenID Connect (OIDC): Authorization Code + PKCE S256. Боевой Keycloak подключается только конфигурацией, код менять не нужно. Боевого Keycloak на хакатоне нет, поэтому в том же приложении работает мок-провайдер с тем же протоколом (о нём ниже).



Протокол (`backend/app/integrations/fsp_id.py`):

- Настройки берутся из `{issuer}/.well-known/openid-configuration` (discovery). Проверяется, что `issuer` в документе совпадает с настроенным, все эндпоинты идут по https и провайдер поддерживает PKCE S256. Документ discovery и ключи JWKS кэшируются на час. Если пришёл неизвестный `kid`, ключи перечитываются (ротация), но не чаще раза в 60 с.
- id_token проверяется так: алгоритм фиксирован (RS256, никаких `none` и HS256), подпись — по JWKS, обязательны `exp`, `iat`, `iss`, `aud`, `sub`, `nonce`; `aud` — наш `client_id`, `azp` проверяется, когда он есть или аудиторий несколько; допуск по часам — 30 с. В userinfo `sub` должен совпасть с `sub` из id_token.
- `state` одноразовый и привязан к браузеру кукой `fsp_flow` (HttpOnly, SameSite=Lax, путь `/api/auth/fsp`): чужая ссылка с `code` и `state` не войдёт и не привяжет чужой ФСП ID (защита от login CSRF). Поток живёт 10 минут.
- PKCE-верификатор и `nonce` не хранятся: они выводятся как HMAC(`SECRET_KEY`, state), так что по дампу базы их не восстановить. `state`, кука и билет хранятся в `fsp_logins` только как sha256.
- Наш токен в адрес не попадает. Callback кладёт во фрагмент адреса одноразовый билет на 60 секунд, фронт меняет его на токен через `POST /exchange` с той же кукой.
- Почта принимается только с `email_verified = true`.

Какой аккаунт входит (`routers/fsp_id.py::_login_user`): сначала ищется аккаунт по ФСП ID (`sub`), затем по подтверждённой почте. Существующий аккаунт с той же почтой связывается автоматически, только если в нём нет ни профиля кандидата, ни компании. Аккаунт с данными привязывается из настроек кандидата после входа по коду: доверять одному `email_verified` провайдера, когда в аккаунте есть что угнать, мы не стали. Новый аккаунт создаётся только по подтверждённой почте, с ролью из `/start` (`candidate` по умолчанию или `employer`). Роль меняется по тем же правилам, что при входе по коду: только пока нет ни профиля, ни компании, а организатора (`admin`) в другую роль не переключить.

Достижения импортируются только кандидату. Компании вход через Keycloak статус «почта подтверждена» не даёт: его ставит только наш код на почту (`users.email_confirmed`, подробнее — раздел 9).

Привязка и отвязка. В настройках кандидата есть кнопки «Связать с ФСП ID» (`POST /start` с `mode=link`), «Обновить достижения» (повторный проход через ФСП ID; при живой сессии Keycloak пароль вводить не нужно) и «Отвязать» (`POST /api/auth/fsp/unlink`). Каждая привязка и отвязка пишется в журнал (`fsp_id_linked`, `fsp_id_unlinked`) без персональных данных. После отвязки подтверждённые достижения остаются; удалить их можно отзывом согласия на внешние данные.

Сторонние входы через брокер Keycloak. Яндекс ID, корпоративный SSO и т. п. настраиваются в Keycloak как Identity Providers. У нас их перечисляет `FSP_ID_IDPS=alias:Подпись,…`, `/start` принимает `idp` и добавляет в адрес авторизации `kc_idp_hint=<alias>`: Keycloak сразу уводит пользователя к этому провайдеру. Неизвестный alias — 422. Список провайдеров фронт берёт из `/api/meta` (`fsp_idps`).

Конфигурация (`backend/app/config.py`):

Переменная	Назначение
`FSP_ID_ISSUER`	`https://<host>/realms/<realm>`, только https
`FSP_ID_CLIENT_ID`, `FSP_ID_CLIENT_SECRET`	confidential-клиент в Keycloak (по умолчанию `client_id` — `fsp-career`)
`PUBLIC_URL`, `FSP_ID_REDIRECT_URI`	redirect URI: по умолчанию `PUBLIC_URL/api/auth/fsp/callback`; у боевого issuer — только https, иначе кнопка ФСП ID выключена
`FSP_ID_SCOPE`	по умолчанию `openid email profile`
`FSP_ID_IDPS`	провайдеры брокера для `kc_idp_hint`
`FSP_ID_MOCK`	встроенный мок; по умолчанию включён при `DEMO=1`, рядом с боевым `FSP_ID_ISSUER` выключается

Мок-провайдер (`routers/mock_fsp_id.py`, путь `/mock-fsp-id`, включён при `DEMO=1` или `FSP_ID_MOCK=1`, иначе 404). Он повторяет протокол realm Keycloak: discovery, страница authorize с выбором демо-аккаунта вместо пароля, token (authorization_code + PKCE S256, код одноразовый и живёт 60 с, проверяются `client_id` и `redirect_uri`), userinfo с claim `fsp_achievements`, JWKS с RS256-ключом, который генерируется при старте процесса. Демо-аккаунты синтетические: кандидаты Дмитрий Орлов (`FSP-1042`, есть в сиде) и Анна Соколова (`FSP-2077`, на платформе её ещё нет), сотрудники компаний Мария Ковалёва (`KC-ladoga-0001`, «Ладога Пэй» из сида) и Игорь Белов (`KC-seversoft-0001`, новая компания). Мок понимает `kc_idp_hint` и кладёт в id_token claim `identity_provider`, как protocol mapper Keycloak. Поток проверяется тестами `backend/tests/test_fsp_id.py` и e2e `frontend/e2e/09-fsp-id.spec.js`.

Формат claim `fsp_achievements`. Это массив в userinfo, каждая запись — в формате строки протокола импорта. Пример записи (поля как в моке, `date` и `result` добавлены для полноты):

```
{
  "fsp_achievements": [
    {
      "event": {
        "external_id": "cup-2025-prod",
        "title": "Финал Кубка России по спортивному программированию 2025, продуктовое
        направление",
        "level": "national",
        "discipline": "продуктовое программирование",
        "date": "2025-11-20",
        "participants": 600,
        "url": null,
        "tags": ["product"]
      },
      "place": 12,
      "team": "Команда «Сапсан»",
      "role": "backend",
      "result": "финалист",
      "rank": "1 разряд"
    }
  ]
}
```

Каждая запись импортируется через `hackathons.import_protocol` с `source = 'fsp'` и `fsp_id = sub`. Берётся не больше 200 записей. Битая запись (неизвестная дисциплина, результат или разряд, не то значение) пропускается, в лог попадает только тип ошибки. Если у кандидата нет профиля или действующего согласия на внешние данные, ничего не пишется, и фронт показывает «Достижения ФСП подтянем после согласия на внешние данные». В Keycloak этот claim должен формировать protocol mapper из атрибута пользователя, который правит реестр ФСП, а не сам пользователь: мы считаем claim подтверждённым.

Импорт протоколов организатором

`POST /api/achievements/import` — для организаторов хакатонов и соревнований ФСП. Он же подтверждает места CTFtime. Интерфейс к нему — страница `/org` (ссылка «Организатору: импорт протоколов» на экране валидации `/v`).

- **Доступ:** `Authorization: Bearer <ORGANIZER_TOKEN>` (переменная окружения) или токен пользователя с ролью `admin`. На странице токен хранится только в памяти вкладки.
- **Формат:** JSON `{event: {...}, rows: [...]}` или multipart: поля события и CSV-файл в поле `file` (заголовок `email,fsp_id,team,place,role,result,rank`, разделитель «,» или «;», UTF-8, BOM допустим). Не больше 2 МБ и 5000 строк.
- **Событие:** `source` (`hackathon`, `fsp`, `ctftime`), `external_id` и `title` обязательны; `level`, `discipline`, `date` (ISO), `participants`, `url`, `tags` — по желанию.
- **Сопоставление** — только по подтверждённым идентификаторам входа: `User.email` (подтверждён кодом) или `User.fsp_id`. Почту из анкеты (`Candidate.email`) мы не используем: её кандидат правит сам и мог бы вписать чужую почту из протокола.
- **Идемпотентность:** событие ищется по паре (`source`, `external_id`), достижение — по паре (кандидат, событие). Протокол авторитетен: повторный импорт перезаписывает строку целиком, а поля события без значения не трогает.
- **Ответ:** `{event_id, matched, created, updated, no_consent, unmatched: [{row, email, fsp_id, reason}]}`. Причины в `unmatched`: «нет email и fsp_id», «кандидат не найден», «нет согласия на внешние данные». Страница `/org` показывает только счётчики и номера строк с причиной, без

почт и ФСП ID участников. Каждый импорт пишется в журнал `protocol_imported` (id события и счётчики).
- **Ошибки:** 401/403 — нет прав; 413 — файл больше 2 МБ; 422 — формат (текст ошибки по-русски, с перечнем допустимых значений).

Пример строки CSV из демо-примера страницы `/org` (кнопка «Демо: подставить пример» при `DEMO=1`):

```
email,fsp_id,team,place,role,result,rank
,FSP-1042,Команда «Сапсан»,2,капитан,призёр,КМС
,FSP-0000,Команда «Ветер»,7,,финалист,
```

Вторая строка в демо специально не привязывается: такого ФСП ID на платформе нет. Сценарий проверяется e2e `frontend/e2e/12-organizer.spec.js`.

Хакатоны идут тем же путём, с `source = hackathon`. Пока реестр ФСП не подключён, `source = fsp` в импорте — его заглушка на том же формате.

Codeforces

Владение подтверждается кодом (`integrations/codeforces.py`, `routers/achievements.py`):

1. `POST /api/achievements/codeforces/link {handle}` выдаёт код вида `FSP-7Q2K`. Открытый код есть только в этом ответе, в базе хранится его sha256, код живёт 24 часа.
2. Кандидат вписывает код в поле «Организация» (или «Имя», «Фамилия») профиля Codeforces.
3. `POST /api/achievements/codeforces/verify` проверяет код через `user.info`, не больше 5 попыток на один код (дальше 429). Один хэндл может подтвердить только один кандидат: это закреплено уникальным индексом в БД.
4. Дальше из `user.info` берётся рейтинг, а из `user.rating` — места в раундах. В достижения попадают до 5 лучших мест в пределах топ-100 общего зачёта: 1-е место — победитель, 2–3-е — призёр, остальные — финалист. Деления на Div. нет, иерархии соревнований тоже.

`POST /codeforces/refresh` обновляет данные не чаще раза в час. Запросы к API идут с паузой не меньше 2 с (троттлинг в памяти процесса), при сбое API — 502, база при этом не меняется. Подтверждённый рейтинг Codeforces также сдвигает стартовую точку адаптивного теста (см. раздел 2), но на итоговый грейд не влияет.

CTFtime

У команды CTFtime нет поля, через которое участник мог бы надёжно доказать членство, поэтому (`integrations/ctftime.py`):

- `POST /api/achievements/ctftime/link {team_id, role}` подтягивает рейтинг команды и до 5 лучших мест за текущий и прошлый год, с общим сроком синхронизации 40 с;
- такие достижения записываются с `verified = false` и вдвое меньшим весом, в сводный сигнал и подбор они не входят;
- подтверждает их организатор: импорт протокола с `source = ctftime` и `external_id` = id события CTFtime находит то же событие и ставит `verified = true` и полный вес. При следующих обновлениях подтверждение не теряется. Вес события по версии CTFtime в наш вес не входит, он хранится только как факт.

Согласие на внешние данные

Согласие отдельное, по 152-ФЗ. Кандидат даёт его при регистрации (поле `consent_external` в `POST /api/candidate/profile`) или в настройках (`POST /api/achievements/consent`). Согласие действует, только если его версия совпадает с текущей `CONSENT_VERSION` и есть действующее согласие на обработку данных (`matching.consent_ok`).

- Без согласия Codeforces и CTFtime отвечают 403. Строки протокола такого кандидата не привязываются: в отчёте импорта они попадают в `no_consent` и `unmatched`. Claim ФСП ID не записывается.
- Отзыв согласия удаляет все достижения внешних источников (`codeforces`, `ctftime`, `hackathon`, `fsp`) и все привязки внешних профилей. Повторный импорт их не вернёт без нового согласия.
- Каждое изменение пишется в `ConsentEvent` с версией текста и в `AuditLog` (`consent_changed`).
- Гонки закрыты блокировкой строки кандидата. После внешнего HTTP-запроса и перед записью импорт и синхронизация берут `SELECT … FOR UPDATE` и перепроверяют согласие (`consent.lock_candidate`), поэтому параллельный отзыв не оставит в базе данных, полученных после него.

Целевая схема интеграции с реальным реестром

Клиент OIDC, формат протокола и путь записи в MVP уже такие, какими должны быть в боевой интеграции. От ФСП нужны две вещи: Keycloak с атрибутом достижений и способ передавать протоколы.



Что нужно от ФСП.

1. **Realm Keycloak:** confidential-клиент для ФСП Карьеры (client_id и secret), разрешённый redirect URI `https://<стенд>/api/auth/fsp/callback`, поддержка PKCE S256, scope `openid email profile`, подтверждённая почта (`email_verified`) и стабильный `sub` — это и есть ФСП ID, ключ сопоставления.
2. **Protocol mapper** `fsp_achievements` в userinfo — из атрибута пользователя, который пользователь не может редактировать. Формат записи — как в примере выше.
3. **Передача протоколов пакетом** — для кандидатов, которые ещё не входили через ФСП ID, и для обновления без входа. Это либо вызов нашего `POST /api/achievements/import` реестром при публикации протокола (вебхук), либо периодическая выгрузка в том же формате. Сейчас у нас есть только сам эндпоинт импорта и доступ по `ORGANIZER_TOKEN`. Ни приёма подписанных вебхуков, ни фоновой выгрузки из реестра нет — это концепция.

Сопоставление полей реестра с нашей моделью (имена полей реестра условные: структура не предоставлена):

Данные реестра	Наше поле	Обязательность
идентификатор соревнования или этапа	`Event.external_id (source = fsp)`	да, должен быть стабильным

Данные реестра	Наше поле	Обязательность
название	`Event.title`	да
дата	`Event.date`	желательно
уровень (международный … локальный)	`Event.level`	по желанию
дисциплина ФСП	`Event.discipline`	желательно
число участников (команд)	`Event.participants`	желательно: без него «финалиста» по месту не определить
ссылка на официальный протокол	`Event.url`	желательно
ФСП ID участника	`fsp_id` строки → `User.fsp_id`	да (или подтверждённая почта)
место	`Achievement.place`	желательно
результат (победитель, призёр, финалист, участник)	`Achievement.result`	желательно: точнее, чем вывод по месту
спортивный разряд или звание	`Achievement.sport_rank`	по желанию
команда, роль в команде	`Achievement.team`, `Achievement.role`	по желанию

ФИО, дата рождения, контакты и прочие персональные данные участников из реестра не нужны: сопоставление идёт по ФСП ID.

Что доработать перед боевым стендом

Из `table.md`:

- ☐ **ФСП ID: доводка для боевого стенда.** Сейчас claim `fsp_achievements` считается доверенным и перезаписывает общие поля события `source = fsp` (название, уровень, число участников — от последнего влияет вес у всех участников события). При подключении реестра `upsert_event` для событий из claim должен только создавать событие и заполнять пустые поля, а авторитетным источником полей события станет протокол реестра. Кроме того: одноразовый билет лежит в заголовке `Location` редиректа 303 (живёт минуту, привязан к куке); нужен лимит на `POST /api/auth/fsp/start` по IP на прокси; индекс по `fsp_logins.expires_at`, если строк станет много; `downgrade` ревизии `0004_fsp_logins` упадёт, если в `users.fsp_id` уже есть UUID-идентификаторы Keycloak.
- ☐ Привязка Keycloak к существующему аккаунту компании из настроек: сейчас привязка из настроек есть только у кандидата.
- ☐ Список доверенных провайдеров и флаг доверия по claim `identity_provider`: сейчас клиент этот claim не использует.
- ☐ Импорт с `source = ctftime` без тегов в форме затирает теги события (`Security`, `CTF`) пустым списком.
- ➖ Refresh-токены провайдера не храним, фоновой синхронизации достижений нет: обновление — при входе и кнопкой «Обновить достижения».
- ➖ Один поток входа на браузер: кука `fsp_flow` одна, второй вход в соседней вкладке делает первый устаревшим.

Ограничения MVP

- Боевого Keycloak и реестра ФСП на хакатоне нет: поток проверен на встроенном мок-провайдере и тестах, достижения в демо синтетические (в синтетике достижения есть примерно у 30 % кандидатов, Codeforces — примерно у 9 %, только у разработчиков; см. README «Как устроено»).
- Веса результатов (0,8 / 0,6 / 0,35 / 0,05) — экспертное решение команды, а не калибровка по данным найма. Экспертной разметки ФСП пока нет.
- Codeforces-раунды в топ-100 считаются «финалистом» независимо от дивизиона (`—` в `table.md`: это топ-1–5 % раунда с тысячами участников, а иерархию соревнований по итогам обсуждения с постановщиками не вводим).

6. Описание API

Бэкенд — FastAPI (`backend/app/main.py`, роутеры в `backend/app/routers/`). Фронт ходит в тот же API через `fetch`, отдельного приватного API у него нет: всё, что умеет интерфейс, доступно и по HTTP. Все методы — под префиксом `/api`, тела запросов и ответов — JSON (кроме загрузки файлов, PDF-профиля и потока чата).

Спецификация OpenAPI

Где	Что
`http://<стенд>/docs`	живой Swagger UI: методы, схемы запросов, кнопка «Try it out»
`http://<стенд>/openapi.json`	спецификация OpenAPI 3.1, генерируется FastAPI из кода
`docs/openapi.json`	та же спецификация, выгруженная из кода для приложения к документации

Выгрузка без базы и без сервера: `cd backend && DEMO=0 uv run python -c "import json; from app.main import app; print(json.dumps(app.openapi(), ensure_ascii=False, indent=2))" > ../docs/openapi.json`. В спецификации 93 операции, заголовок «ФСП Карьера API», версия 0.1.0. В схему не входят служебные адреса: мок-провайдер ФСП ID `/mock-fsp-id/*` и страницы фронта (`/`, `/candidate…`, `/employer…`, `/v`, `/org`).

Что в OpenAPI пока неполно. В спецификации описаны **запросы** (тела — Pydantic-модели с типами и ограничениями, параметры пути и строки запроса), общие ответы с ошибками 401, 403, 404, 409, 413, 422, 429 и 503 (`ErrorResponse`) объявлены глобально для всех методов (`FastAPI(..., responses=…)` в `backend/app/main.py`), краткий справочник — `docs/api.md`. Модели успешных ответов (`response_model`, `backend/app/api_schemas.py`) пока есть у шести ключевых методов обратного найма: потребности (создание и список), подборка, карточка кандидата, поиск по банку и приглашение; остальные возвращают словари без схемы ответа `200`. Поэтому ниже для каждой группы методов перечислены ключевые поля ответа и специфичные ошибки, а для ключевых методов приведены примеры. Источник — код роутеров. Модели ответов в OpenAPI — ближайшая доработка (`table.md`).

Ещё одна особенность: аутентификация не объявлена в спецификации как `securityScheme`, а `Authorization` у каждого метода — обычный заголовок-параметр. В Swagger UI токен вписывается в поле `authorization` метода в виде `Bearer <токен>`; общей кнопки «Authorize» нет.

Аутентификация

Вход выдаёт **Bearer-токен сессии**, который передаётся в заголовке `Authorization: Bearer <токен>`. Токен — 32 случайных байта (`secrets.token_urlsafe`), в базе хранится только его sha256, срок жизни — 30 дней. Новый вход отзывает прежний токен, `POST /api/auth/logout` отзывает текущий (`backend/app/deps.py`).

Способ	Методы	Когда доступен
Код на почту	`POST /api/auth/email {email, role}` → письмо с 6-значным кодом (живёт 10 минут, новый — не чаще раза в минуту, 5 попыток); `POST /api/auth/verify {email, otp}` → `{token, role, has_profile}`	всегда; без SMTP вне демо — 503. Код хранится как HMAC с серверным ключом `SECRET_KEY`. Пользователь создаётся только после подтверждения почты, роль выбирает владелец адреса
ФСП ID (Keycloak, OpenID Connect)	`POST /api/auth/fsp/start` → провайдер → `GET /api/auth/fsp/callback` → `POST /api/auth/fsp/exchange {ticket}` → `{token, role, has_profile}`	если настроен `FSP_ID_*` или включён мок; подробно — раздел 5
Демо-вход за персонажа	`POST /api/auth/demo/{role}` (`candidate` — Дмитрий Орлов, `employer` — «Ладога Пэй»)	только при `DEMO=1`, иначе 404. В демо `POST /api/auth/email` ещё и возвращает код в ответе (`otp`). Общий демо-токен вне `DEMO=1` не действует
Токен организатора	`Authorization: Bearer <ORGANIZER_TOKEN>`	только для `POST /api/achievements/import` и `POST /api/validation/run` (вне демо); альтернатива — пользователь с ролью `admin`

Роли. `candidate` (кандидат), `employer` (компания), `admin` (организатор). Роль проверяется зависимостями FastAPI: `current_user` — любой вошедший; `current_candidate` — создан профиль кандидата, иначе 403; `current_company` — зарегистрирована компания, иначе 403; `active_company` — компания не заблокирована по отзывам кандидатов, иначе 403. Кандидат видит только свои приглашения, отклики и решения: чужой объект отвечает 404, а не 403, чтобы не раскрывать, что он существует.

Общие коды ошибок

Ошибки приходят как JSON `{"detail": …}`. Текст `detail` — по-русски и рассчитан на показ пользователю (фронт показывает его как есть).

Код	Что значит в этом API	Формат `detail`
401	Нет токена, токен неверен или истёк («Нужен вход»); неверный или истёкший код входа; устаревший билет ФСП ID; неверный токен организатора	строка
403	Роль не та или нет профиля («Профиль кандидата не создан», «Компания не зарегистрирована»); компания заблокирована до ручной проверки; нет действующего согласия (на обработку данных — для принятия приглашения, отклика, решения задания и чата; на внешние данные — для Codeforces и CTFtime); кандидат без теста пытается откликнуться или решить задание; импорт протоколов не организатором	строка

Код	Что значит в этом API	Формат `detail`
404	Объект не найден или не виден этому пользователю. Сюда же относятся: кандидат без теста или скрывший профиль от компании («Кандидат не найден» — существование не раскрывается); демо-методы вне `DEMO=1`; нет активной сессии теста	строка или пусто
409	Конфликт состояния: приглашение уже закрыто или истекло; повторное приглашение раньше срока; ёмкость кандидата заполнена (4 открытых приглашения); ответ не на текущее задание теста; тест не завершён или уже завершён; вакансия закрыта, истекла или превышен лимит открытых; решение уже отправлено; прогон валидации уже идёт; привязка внешнего профиля изменилась во время проверки	строка
413	Тело запроса больше 6 МБ (все методы); резюме больше 5 МБ; протокол больше 2 МБ	строка
422	Ошибка входных данных. Два вида: (1) **валидация FastAPI/Pydantic** — тип, обязательное поле, длина, сумма больше 10 млн ₽: `detail` — массив `[{type, loc, msg, input, …}]` (у суммы `msg` по-русски, у остальных — стандартный английский текст Pydantic); (2) **правила домена** — значение не из справочника (специализация, трек, грейд, формат, причина отказа), неверный ИНН, неверный формат протокола: `detail` — строка с перечнем допустимых значений. У приглашения и вакансии нарушение правил (вилка, формат, справочники) — `detail = {"errors": [...], "warnings": [...]}`	массив, строка или объект
429	Лимит частоты: новый код почты раньше чем через 60 с; больше 5 проверок кода Codeforces; недельная квота приглашений компании исчерпана; больше 10 откликов кандидата за 7 дней; больше 20 сообщений чата в минуту; запуски задачи на код на примерах (не больше 20 удачных и не чаще раза в 3 с); решение уже проверяется	строка

Код	Что значит в этом API	Формат `detail`
502	Внешний сервис ответил ошибкой или недоступен: Codeforces, CTFtime, провайдер ФСП ID при `start`, SMTP при отправке кода. База при этом не меняется	строка
503	Не настроено или временно недоступно: отправка писем (`SMTP_HOST`), вход через ФСП ID, банк заданий пуст, песочница задач на код занята или недоступна (задание остаётся текущим или помечается «не засчитано»), нет файла фронта	строка

Код 500 для ожидаемых ситуаций не используется: сбой LLM уходит в фолбэк на правила (раздел 10), сбой внешних API превращается в 502, сбой песочницы — в 503.

Методы по группам

Обозначения в колонке «Кто»: «—» — без входа; «вход» — любой вошедший; «кандидат» — пользователь с профилем кандидата; «компания» — пользователь с зарегистрированной компанией; «компания (активная)» — то же, но компания не заблокирована; «организатор» — `ORGANIZER_TOKEN` или роль `admin`. Ошибки 401 и 403 из-за роли, общие для всех защищённых методов, в колонке «Ошибки» не повторяются.

Вход (`auth`)

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/auth/email`	—	Отправить код. Запрос `{email, role: candidate | employer}` → `{sent, otp}` (`otp` — только в демо)	422, 429, 502, 503
POST	`/api/auth/verify`	—	Проверить код → `{token, role, has_profile}`	401
POST	`/api/auth/demo/{role}`	—	Демо-вход за персонажа → `{token, role}`	404 (не демо или нет сида)
POST	`/api/auth/logout`	вход	Отозвать токен → `{ok}`	—
GET	`/api/auth/me`	вход	`{id, email, role, fsp_id}`	—

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/auth/fsp/start`	— (для `mode=link` — кандидат)	Начать вход или привязку ФСП ID: `{mode: login | link, role, idp}` → `{url}` адреса авторизации, ставит куку `fsp_flow`	401, 403, 422 (неизвестный `idp`), 502, 503
GET	`/api/auth/fsp/callback`	браузер с кукой `fsp_flow`	Возврат от провайдера: 303 на экран входа или настроек; во фрагменте адреса `fsp=login|linked|error`, одноразовый билет `t`, число импортированных достижений `ach`, код отказа `e`	ошибки — кодом `e` во фрагменте
POST	`/api/auth/fsp/exchange`	браузер с кукой	Билет → `{token, role, has_profile}`	401
POST	`/api/auth/fsp/unlink`	вход	Отвязать ФСП ID → `{fsp_id: null}`	—

Кандидат (`candidate`)

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/candidate/profile`	вход с ролью кандидата	Создать или обновить профиль: ФИО, контакты, специализация, трек, заявленный грейд, минимум зарплаты, форматы, город, навыки, согласия (`consent_152`, `consent_publish`, `consent_external`), текст резюме. Ответ — как `GET /me` плюс `test_reset` (смена специализации сбрасывает результат теста)	403, 422 (справочники, нет согласия на обработку)

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
GET	`/api/candidate/me`	кандидат	Профиль: категория, θ ± SE, компетенции, навыки, достижения, согласия и их версии, `visible` (видят ли работодатели), открытые приглашения и ёмкость	—
DELETE	`/api/candidate/me`	кандидат	Удалить аккаунт со всеми данными → `{deleted: true}`	—
GET	`/api/candidate/me/export`	кандидат	Выгрузка всех своих данных одним JSON (152-ФЗ)	—
POST	`/api/candidate/cv`	вход	Загрузить PDF-резюме (multipart, поле `file`, до 5 МБ) → распознанные поля для формы и подсказка грейда	413, 422 (не PDF, скан без OCR)
GET	`/api/candidate/profile.pdf`	кандидат	Стандартизированный PDF-профиль	—
PATCH	`/api/candidate/pdf`	кандидат	`{experience}` — показывать ли места работы в PDF	—
PATCH	`/api/candidate/status`	кандидат	`{status: open | off}` — ищу работу или нет	422
PATCH	`/api/candidate/notif`	кандидат	`{mail, tg}` — уведомления	—
PATCH	`/api/candidate/hidden-from`	кандидат	`{inns}` — скрыть профиль от компаний по ИНН	422
PATCH	`/api/candidate/consents`	кандидат	`{consent_152, consent_publish}` — дать или отозвать согласия	422
GET	`/api/candidate/invites`	кандидат	Входящие приглашения → `{items, open, cap}`; у каждого компания, должность, вилка `from–to`, формат, срок ответа, статус	—

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
GET	`/api/candidate/invites/{id}`	кандидат	Карточка приглашения (переводит статус в «просмотрено»): проверка вилки против минимума, почему кандидат подошёл, контакт рекрутера	404
POST	`/api/candidate/invites/{id}/accept`	кандидат	Принять приглашение: контакты кандидата открываются компании, раскрытие пишется в журнал	403 (нет согласия), 404, 409
POST	`/api/candidate/invites/{id}/decline`	кандидат	`{reason, comment}` — отказ с причиной из справочника	404, 409, 422
POST	`/api/candidate/invites/{id}/company-feedback`	кандидат	`{value}` — отзыв о компании по принятому приглашению (защита от фиктивных приглашений)	404, 409, 422

Тест (`quest`)

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/quest/start`	кандидат	Начать тест или продолжить незаконченный: `{mode: full | express, target_grade, story, invitation_id}` → состояние сессии с текущим заданием (без ключа ответа), `resumed`, `cooldown`	404 (приглашение), 409, 422, 503 (банк пуст)
GET	`/api/quest/current`	кандидат	Состояние активной сессии	404

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/quest/answer`	кандидат	Ответ на текущее задание: `{item_id, response}` или для задачи на код `{item_id, lang, code, pasted}`; `ms` — время по часам клиента → `{saved, …следующее задание}`. Правильность до конца теста не показывается	404, 409, 422, 429, 503 (песочница)
POST	`/api/quest/run`	кандидат	Запустить решение задачи на код на открытых примерах → `{status, passed, total, results, stderr, runs_left}`; на оценку не влияет	404, 409, 422, 429, 503
POST	`/api/quest/finish`	кандидат	Завершить: θ ± SE, грейд, категория, компетенции, перцентиль, флаги, `applied` (изменился ли профиль), `note`, `lower_offer`	404, 409
POST	`/api/quest/accept-lower`	кандидат	`{session_id}` — добровольно принять грейд последней попытки, если он ниже текущего	409
GET	`/api/quest/result`	кандидат	Текущий результат и условия повышения (`upgrade`: следующий грейд, порог θ − SE, дата окончания кулдауна)	404
GET	`/api/quest/history`	кандидат	История попыток без ответов и ключей	—
POST	`/api/quest/plan`	компания	План теста под потребность: `{need_id}` или структура `{spec, track, grade, must, nice}` или `{text}` → веса компетенций, диапазон сложности, примеры заданий	404, 422
POST	`/api/quest/plan/preview`	—	То же по тексту вакансии для публичного демо	404 вне демо, 422

Работодатель (`employer`)

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/employer/company`	вход с ролью компании	Профиль компании: ИНН, название, описание, сайт, рекрутер и контакты	403, 409 (ИНН занят или менять поздно), 422
GET	`/api/employer/company`	компания	Профиль, автоматические проверки (контрольная сумма ИНН, почта на домене сайта), квота	—
POST	`/api/employer/needs/parse`	компания	Текст потребности → структура (специализация, трек, грейд, обязательные и желательные навыки, вилка, формат, город) + рынок и число кандидатов в категории. Ничего не сохраняет	422
POST	`/api/employer/needs/category`	компания	Категория, поправленная вручную → рынок и число кандидатов	422
POST	`/api/employer/needs`	компания	Сохранить потребность → `{id, spec, track, grade, must, nice, from, to, format, city, …}`	422
GET	`/api/employer/needs`	компания	Все потребности компании	—
GET	`/api/employer/needs/{id}/picks`	компания	Подборка: `filters` (`fmt`, `free`, `fsp`, `cf`), `variant` (`embed`, `rerank`, `full`), `limit` → `{need, total, items, categories, market, quota, cap}`; у каждого кандидата анонимная карточка, оценка и объяснение	404, 422

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
GET	`/api/employer/candidates/{public_id}`	компания	Анонимная карточка кандидата: по `need_id` — с разбором оценки (`explanation`, `breakdown`); `invite_block` — можно ли пригласить снова	404
GET	`/api/employer/search`	компания	Поиск по всему банку: `spec`, `track`, `grade`, `skills`, `fsp`, `cf`, `hackathon`, `format`, `city`, `q` (навыки из текста; если в `q` только ID кандидатов вроде `A-1042` — поиск по ID мимо остальных фильтров, `by_id: true`), `page`, `limit` → `{items, total, page, facets, …}`; ранжирование по силе подтверждённого профиля	422
POST	`/api/employer/invites`	компания (активная)	Отправить приглашение (обязательная вилка) → `{invite, warnings, quota}`	404, 409, 422 (вилка, формат, лотерея), 429 (квота)
GET	`/api/employer/pipeline`	компания	Статусы всех приглашений; контакты — только у принятых	—
POST	`/api/employer/invites/{id}/feedback`	компания (активная)	`{value}` — отметка по принятому приглашению: грейд подтвердился, не подтвердился, наняли	404, 409, 422
GET	`/api/employer/quota`	компания	`{used, max, left, resets_at, accepted_share, next_max, reputation}`	—

Лотерея (`lottery`)

Квалифицированная лотерея — способ дать шанс всем, кто прошёл порог, а не только верху выдачи (см. раздел 3).

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/lottery/`	компания (активная)	`{need_id, threshold, draw_count}` → пул замораживается, публикуется `seed_hash`: `{id, pool_size, seed_hash, commitment, threshold}`	404, 409 (лотерея уже есть), 422
POST	`/api/lottery/{id}/draw`	компания (активная)	Бросить жребий, раскрыть seed	404, 409
GET	`/api/lottery/{id}`	компания	Состояние лотереи и выбранные кандидаты	404
GET	`/api/lottery/{id}/verify`	компания	Проверка результата по `pool_ids` и seed	404, 409 (жребий не брошен)
GET	`/api/lottery/{id}/public`	—	Обезличенные данные для независимой проверки	404

Вакансии и отклики (`vacancies`)

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/vacancies`	компания (активная)	Опубликовать вакансию на 30 дней (обязательная вилка)	409 (лимит открытых), 422
PATCH	`/api/vacancies/{id}`	компания (активная)	Изменить вакансию → вакансия и `warnings`	404, 422
POST	`/api/vacancies/{id}/close`	компания	Закрыть	404
POST	`/api/vacancies/{id}/extend`	компания (активная)	Продлить на 30 дней	404, 409
GET	`/api/vacancies/mine`	компания	Свои вакансии и лимит открытых	—
GET	`/api/vacancies/{id}/applications`	компания	Отклики: анонимная карточка, контакты — только после принятия отклика	404
POST	`/api/vacancies/applications/{id}/status`	компания (активная)	`{status: seen | accepted | declined, reason}`	404, 409, 422

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
GET	`/api/vacancies`	— (кандидату — с сортировкой по соответствию)	Открытые вакансии: фильтры `spec`, `track`, `grade`, `format`, `city`, `q`, `salary_min`, `limit` → `{items, total, sort}`	422
GET	`/api/vacancies/{id}`	—	Карточка вакансии	404
POST	`/api/vacancies/{id}/apply`	кандидат	`{note}` — откликнуться	403 (нет согласия или теста), 404, 409, 429
GET	`/api/vacancies/applications/mine`	кандидат	Свои отклики и статусы	—
POST	`/api/vacancies/applications/{id}/withdraw`	кандидат	Отозвать отклик без ответа	404, 409

Интеграция с ATS (`ats`)

Ключи API выпускает компания в кабинете (Bearer-сессия работодателя). Входящий API `/api/ats/v1` авторизуется заголовком `X-Api-Key` и проверяет области доступа ключа. Подробно — раздел 11.3.

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
GET	`/api/employer/api-keys`	компания	Свои ключи: `id`, `name`, `prefix`, `scopes`, `created_at`, `last_used_at`, `revoked_at` (без самого ключа)	403
POST	`/api/employer/api-keys`	компания (активная)	`{name, scopes}` → то же и `key`: полный ключ только в этом ответе. Не больше 10 активных ключей	409, 422
DELETE	`/api/employer/api-keys/{id}`	компания	Отозвать ключ	404
PUT	`/api/ats/v1/vacancies/{external_id}`	ключ, `vacancies:write`	Создать или заменить вакансию по внешнему id; тело как у `POST /api/vacancies` → вакансия, `warnings`, `external_id`	401, 403, 409, 422

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/ats/v1/vacancies/{external_id}/close`, `/extend`	ключ, `vacancies:write`	Закрыть, продлить; закрыть можно и заблокированной компании (сужает показ), продлить нельзя	401, 403, 404, 409
POST	`/api/ats/v1/applications/{id}/status`	ключ, `applications:write`	`{status, reason}`; повтор того же статуса (для `declined` — с той же причиной) — 200, другая причина — 409	401, 403, 404, 409, 422
POST	`/api/ats/v1/invites/{id}/feedback`	ключ, `invites:write`	Отметка по принятому приглашению; повтор — 200	401, 403, 404, 409
GET	`/api/ats/v1/events`	ключ, `applications:read` или `invites:read`	`after` (курсор — строка `<txid>.<id>` из `next`, `0` — с начала; иначе 422; отдаются только события завершённых транзакций), `limit` 1..500 → `{items: [{id, type, at, data}], next: "<txid>.<id>"}`; `data.candidate`: в откликах — анонимная карточка, в приглашениях — только `{public_id}`; контакты в `data` — только когда их видел бы кабинет, первая выдача пишет `contacts_revealed`; перед чтением истёкшие приглашения компании переводятся в `expired`	401, 403, 422

В демо (`ATS_MOCK`) страница `/mock-ats` показывает ленту и отвечает через этот API.

Задания от работодателей (`tasks`)

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/tasks`	компания (активная)	Создать задание: текст, специализация, компетенция, вид (`solve` или `approach`), минуты, публичность, тип ответа и эталон для автопроверки	404, 422
GET	`/api/tasks/mine`	компания	Свои задания	—
GET	`/api/tasks/{id}/submissions`	компания	Решения задания	404
POST	`/api/tasks/submissions/{id}/review`	компания (активная)	`{review: strong | ok | weak, comment}`	404, 422
GET	`/api/tasks/showcase`	кандидат	Витрина доступных заданий	—
GET	`/api/tasks/submissions/mine`	кандидат	Свои решения	—
GET	`/api/tasks/for-invitation/{id}`	кандидат	Задание, приложенное к приглашению	404
GET	`/api/tasks/{id}`	кандидат	Задание (эталон не отдаётся)	404
POST	`/api/tasks/{id}/submit`	кандидат	`{answer, choices, invitation_id}` — отправить решение (одна попытка)	403, 404, 409, 422

Достижения и интеграции (`achievements`)

Подробно — раздел 5.

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
GET	`/api/achievements`	кандидат	Достижения по источникам (`fsp`, `hackathons`, `codeforces`, `ctftime`), высший разряд, сводный сигнал	—
POST	`/api/achievements/consent`	кандидат	`{value}` — согласие на внешние данные; отзыв удаляет внешние достижения	403 (нет согласия на обработку)

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/achievements/codeforces/link`	кандидат	`{handle}` → код подтверждения (только в этом ответе) и инструкция	403, 409, 422
POST	`/api/achievements/codeforces/verify`	кандидат	Проверить код в профиле Codeforces, подтянуть рейтинг и места	400 (код не найден или истёк), 403, 404, 409, 429, 502
POST	`/api/achievements/codeforces/refresh`	кандидат	Обновить (не чаще раза в час)	403, 404, 409, 502
POST	`/api/achievements/ctftime/link`	кандидат	`{team_id, role}` — привязать команду (места без подтверждения)	403, 409, 502
POST	`/api/achievements/ctftime/refresh`	кандидат	Обновить	403, 404, 409, 502
POST	`/api/achievements/import`	организатор	Импорт протокола: JSON `{event, rows}` или multipart с CSV → `{event_id, matched, created, updated, no_consent, unmatched}`	401, 403, 413, 422
GET	`/api/achievements/events`	—	Список событий и число кандидатов платформы в каждом (`source` — фильтр)	—

Это единственный набор методов, где встречается 400: неверный или истёкший код подтверждения Codeforces.

Чат (`chat`)

Чат доступен, пока приглашение принято кандидатом или отклик принят компанией (`peer_id` — id компании у кандидата и публичный id кандидата у компании).

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
GET	`/api/chat/contacts`	вход	Список диалогов	403
GET	`/api/chat/unread`	вход	`{unread_count}`	403
GET	`/api/chat/stream`	вход	Server-Sent Events: строка `data: new` — есть новое, клиент перечитывает диалог	403
GET	`/api/chat/{peer_id}`	вход	История: `limit`, `before_id` или `after_id`	403, 404, 422

Метод	Путь	Кто	Назначение и ключевые поля	Ошибки
POST	`/api/chat/{peer_id}`	вход	`{text}` — сообщение до 2000 символов	403, 404, 422, 429
POST	`/api/chat/{peer_id}/read`	вход	Отметить прочитанным	403, 404

Служебные

Метод	Путь	Кто	Назначение	Ошибки
GET	`/api/health`	—	`{ok: true}` — проверка живости	—
GET	`/api/meta`	—	Справочники для форм: специализации, треки, грейды и пороги, форматы, причины отказа, компетенции, навыки; флаги `code_tasks`, `fsp_id`, `fsp_idps`, `demo`, `ats_mock`, `mail`; сюжеты теста	—
GET	`/api/validation`	—	Результат последнего прогона валидации (`validation.json`, см. раздел 4)	404 (прогона не было)
POST	`/api/validation/run`	при `DEMO=1` — без входа, иначе организатор	Пересчитать валидацию: `k`, `seed`, `today`	401, 403, 409 (нет синтетики или прогон уже идёт), 422

Примеры ключевых методов

Значения иллюстративные (синтетические данные, длинные тексты сокращены «…»), имена полей — точно как в коде.

Старт теста и ответ на задание

```
POST /api/quest/start
Authorization: Bearer <токен кандидата>
Content-Type: application/json

{"mode": "full", "target_grade": "Middle"}
```

```
{
  "session_id": 812,
  "quest_title": "Где Рей?",
  "mode": "full",
  "story": true,
  "target_grade": "Middle",
  "focus": null,
  "done": false,
  "progress": {"n": 1, "total": 13},
  "item": {
    "id": 4471,
    "kind": "choice",
    "competency": "Диагностика инцидентов",
    "scene_title": "…",
    "time": "02:14",
    "story": "…",
    "prompt": "В чём причина?",
    "panel": {"type": "logs", "lines": ["…"]},
    "options": ["…", "…", "…", "…"],
    "time_limit_s": 240
  },
  "time_limit_s": 240,
  "remaining_s": 240,
  "deadline_s": 270,
  "submitted": false,
  "resumed": false,
  "cooldown": {"active": false, "until": null, "note": null}
}
```


Ключа ответа в задании нет. `total` — до 12 рабочих заданий плюс пилотное. `deadline_s` больше лимита на 30 с — запас на сеть.

```
POST /api/quest/answer
Authorization: Bearer <токен кандидата>
Content-Type: application/json

{"item_id": 4471, "response": 2, "ms": 41250}
```

Ответ — `{"saved": true, …}` с тем же состоянием сессии и следующим заданием (`progress.n` = 2). Верный ли ответ, не сообщается. Ответ не на текущее задание — `409 {"detail": "Ответ не на текущее задание"}`, индекс варианта вне диапазона — `422 {"detail": "Некорректный ответ"}`.

Подборка под потребность

```
GET /api/employer/needs/17/picks?filters=fsp&limit=10
Authorization: Bearer <токен компании>
```

```
{
  "need": {"id": 17, "spec": "Backend", "track": "Python", "grade": "Middle",
           "must": ["Python", "PostgreSQL"], "nice": ["Kafka"], "from": 230000, "to": 290000,
           "format": "Гибрид", "city": "Москва", "parsed_by": "rules", "text": "…", "created_at":
         "…"},
  "total": 64,
  "filters": ["fsp"],
  "items": [
    {
      "rank": 1,
      "id": "A-1042", "public_id": "A-1042",
      "spec": "Backend", "track": "Python", "grade": "Middle", "grade_claimed": "Middle",
      "pct": 84, "theta": "0.92 ± 0.31",
      "skills": ["Python", "PostgreSQL"], "claimed": ["Kafka"],
      "gaps": ["Kafka не подтверждён тестом"],
      "fsp": "Финал Кубка России по спортивному программированию 2025, продуктовое направление",
      "badges": [{"source": "fsp", "title": "…", "place": 12, "result": "finalist",
                  "discipline": "продуктовое программирование", "rank": "1 разряд"}],
      "sport_rank": "1 разряд", "cf": null,
      "salary": 250000, "format": "…", "fmtOk": true, "inFork": true,
      "load": 0, "busy": false,
      "match": 0.712, "accept": 0.81, "score": 0.644,
      "express": false,
      "trust": {"test": true, "review": false, "employer_confirmed": 0},
      "invited": null,
      "why": "Совпадение стека Python, PostgreSQL; диагностика инцидентов — 84-й перцентиль"
    }
  ],
  "categories": [{"spec": "Backend", "track": "Python", "grade": "Middle", "count": 64, "current":
        true, "…": "…"}],
  "market": {"p25": 210000, "p50": 260000, "p75": 310000},
  "quota": {"used": 6, "max": 10, "left": 4, "…": "…"},
  "cap": 4
}
```


В карточке нет ФИО, почты, Telegram и резюме: это белый список полей `matching.card`. `score = match × (0,5 + 0,5·accept) × штраф за нагрузку` (раздел 3). Разбор оценки по слагаемым — в `GET /api/employer/candidates/A-1042?need_id=17` (`breakdown`: θ-оценка, покрытие, достижения, штрафы и веса).

Приглашение

```
POST /api/employer/invites
Authorization: Bearer <токен компании>
Content-Type: application/json

{"candidate_id": "A-1042", "need_id": 17, "role": "Python-разработчик в платёжный шлюз",
 "salary_from": 230000, "salary_to": 290000, "format": "Гибрид, Москва, 2 дня в офисе",
 "description": "…"}
```

```
{
  "invite": {"id": 5310, "cid": "A-1042", "need_id": 17, "lottery_id": null,
             "role": "Python-разработчик в платёжный шлюз", "from": 230000, "to": 290000,
             "format": "Гибрид, Москва, 2 дня в офисе", "status": "sent", "reason": null,
             "when": "9 окт", "sent_at": "2026-10-09T09:12:00Z", "expires_at": "2026-10-
        16T09:12:00Z",
             "contact": null, "fb": null, "feedback": null},
  "warnings": [],
  "quota": {"used": 7, "max": 10, "left": 3, "resets_at": "…", "accepted_share": 0.5, "next_max":
        12,
            "paid_accepts": 3, "reputation": {"…": "…"}}
}
```


`contact: null`: до принятия компания контактов не видит. Нарушение правил вилки — 422 с объектом:


```
{"detail": {"errors": ["Нижняя граница ниже 25-го перцентиля рынка для Middle (210 000 ₽ на
        руки)."], "warnings": []}}
```


Другие отказы: открытое приглашение этой же компании — `409 "Кандидат уже приглашён"`; у кандидата 4 открытых приглашения — `409 "У кандидата максимум открытых приглашений"`; квота исчерпана — `429 "Квота на неделю исчерпана (10 из 10). Обновится в понедельник"`.

Принятие приглашения кандидатом

```
POST /api/candidate/invites/5310/accept
Authorization: Bearer <токен кандидата>
```


```
{
  "id": 5310, "company": "Ладога Пэй", "role": "Python-разработчик в платёжный шлюз",
  "from": 230000, "to": 290000, "format": "Гибрид, Москва, 2 дня в офисе", "desc": "…",
  "status": "accepted", "due": null, "below_min": false,
  "domain_confirmed": true, "inn_checksum_valid": true,
  "fork_check": {"ok": true, "min": 250000, "text": "Вилка покрывает ваш минимум 250 000 ₽"},
  "why": "Совпадение стека Python, PostgreSQL; …",
  "recruiter": {"name": "…", "email": "…", "tg": "…"},
  "company_knows": "contacts",
  "company_sees": {"name": "…", "email": "…", "tg": "…"},
  "public_id": "A-1042", "load": {"open": 1, "cap": 4},
  "chat_peer": 3, "feedback_open": false, "company_feedback": null,
  "sent_at": "…", "expires_at": "…"
}
```


После принятия кандидат видит, какие его контакты открылись компании (`company_sees`), а в журнал пишется `contacts_revealed`. Ошибки: срок ответа истёк или приглашение уже закрыто — `409`; согласие на обработку отозвано или устарело — `403` (контакты не раскрываются).

Ограничения

- Схемы успешных ответов в спецификации OpenAPI есть только у шести ключевых методов (см. выше), у остальных — ближайшая доработка; общие ошибки описаны у всех.
- Аутентификация не объявлена как `securityScheme`, у каждого метода — заголовок-параметр `authorization`.
- Версия API одна (`0.1.0`), версионирования путей нет — для MVP с единственным клиентом этого достаточно.

- Лимиты частоты — только прикладные (код почты, квота, чат, отклики). Общий лимит по IP предполагается на обратном прокси перед стендом (`table.md`).

7. Используемые библиотеки и компоненты

Раздел перечисляет всё, из чего собирается и чем проверяется решение: библиотеки бэкенда, инструменты тестирования фронта, контейнерные образы, необязательные модели и CI. Версии бэкенда — точные, из файла блокировки `backend/uv.lock` (сборка образа идёт с `uv sync --frozen`, поэтому ставятся ровно они). Лицензии библиотек — из метаданных установленных пакетов (`License-Expression` / `License` в `importlib.metadata`), лицензии образов и моделей — по данным их правообладателей.

ТЗ (п. 3.5) рекомендует открытые лицензии (MIT, Apache-2.0, BSD и аналогичные) или ПО из реестра отечественного ПО. Всё перечисленное ниже — открытое ПО; отступления от «MIT / Apache / BSD» разобраны в конце раздела.

7.1. Бэкенд (Python)

Версия Python — 3.12: `requires-python = ">=3.12"` в `backend/pyproject.toml`, базовый образ `python:3.12-slim` в `backend/Dockerfile`, CI гоняет тесты на 3.12. Менеджер зависимостей — `uv` (в образ копируется из `ghcr.io/astral-sh/uv:0.10`).

Прямые зависимости (работа приложения)

Компонент	Ограничение в `pyproject.toml`	Версия в `uv.lock`	Лицензия	Назначение
fastapi	≥ 0.115	0.142.2	MIT	HTTP API и автоматическая спецификация OpenAPI (`/docs`)
uvicorn[standard]	≥ 0.32	0.54.0	BSD-3-Clause	ASGI-сервер, один процесс (см. раздел 8)
sqlalchemy	≥ 2.0	2.1.3	MIT	ORM и запросы к PostgreSQL (синхронный режим)
psycopg[binary]	≥ 3.2	3.3.6	LGPL-3.0-only	драйвер PostgreSQL (бинарная сборка с libpq)
alembic	≥ 1.20.0	1.20.0	MIT	миграции схемы БД, накатываются при старте API
httpx	≥ 0.27	0.28.1	BSD-3-Clause	HTTP-клиент: Codeforces, CTFtime, ФСП ID (OIDC), LLM, песочница
pyjwt[crypto]	≥ 2.9	2.15.1	MIT	проверка подписи id_token ФСП ID по JWKS (RS256), подпись токенов мок-провайдера
pypdf	≥ 5.0	6.19.0	BSD-3-Clause	извлечение текста и встроенных картинок из PDF-резюме

Компонент	Ограничение в `pyproject.toml`	Версия в `uv.lock`	Лицензия	Назначение
fpdf2	≥ 2.8	2.8.9	LGPL-3.0-only	генерация PDF-профиля кандидата
pillow	≥ 10	12.3.0	MIT-CMU (HPND)	декодирование картинок страниц скана (через `pypdf` `page.images`) и уменьшение до 1600 px перед OCR (`services/ocr.py`)
numpy	≥ 2.0	2.5.3	BSD-3-Clause (с вложенными 0BSD, MIT, Zlib, CC0-1.0)	IRT-расчёты, метрики валидации, синтетика
python-multipart	≥ 0.0.12	0.0.32	Apache-2.0	приём файлов (резюме, протоколы) в FastAPI
regex	≥ 2024.9	2026.9.29	Apache-2.0 AND CNRI-Python	регулярные выражения с таймаутом сверки: автопроверка ответа на задание работодателя по шаблону (`services/autocheck.py`)

Существенные транзитивные зависимости

Ставятся автоматически, перечислены, потому что на них опирается заметная часть функциональности.

Компонент	Версия	Лицензия	Откуда и зачем
starlette	1.7.0	BSD-3-Clause	основа FastAPI: маршрутизация, статика, SSE (`StreamingResponse`)
pydantic	2.13.5	MIT	валидация входных данных API (через FastAPI)
pydantic-core	2.46.5	MIT	ядро pydantic
cryptography	50.0.2	Apache-2.0 OR BSD-3-Clause	RSA для JWT (через `pyjwt[crypto]`)
uvloop	0.23.0	MIT / Apache-2.0	цикл событий uvicorn (`[standard]`, Linux)
httptools	0.8.0	MIT	разбор HTTP в uvicorn (`[standard]`)
websockets	17.2	BSD-3-Clause	из `uvicorn[standard]`, приложением не используется
watchfiles	1.3.0	MIT	из `uvicorn[standard]` (перезапуск при разработке)
python-dotenv	1.2.4	BSD-3-Clause	из `uvicorn[standard]`

Компонент	Версия	Лицензия	Откуда и зачем
pyyaml	6.0.3	MIT	из `uvicorn[standard]`
mako	1.4.3	MIT	шаблоны ревизий Alembic
fonttools	4.66.1	MIT	работа со шрифтами в fpdf2
defusedxml	0.7.1	PSF	безопасный разбор XML в fpdf2
greenlet	3.5.6	MIT AND PSF-2.0	зависимость SQLAlchemy
anyio / h11 / httpcore	4.15.1 / 0.16.0 / 1.0.9	MIT / MIT / BSD-3-Clause	сетевой стек httpx и uvicorn
certifi	2026.7.22	MPL-2.0	корневые сертификаты для HTTPS-запросов httpx

Всего в `uv.lock` 79 пакетов (вместе с dev-зависимостями).

Отправка писем (`app/mail.py`, `smtplib`), HMAC и хеши (`hmac`, `hashlib`), CSV/JSON протоколов, маскировка ПДн (`re`) — стандартная библиотека Python, отдельных пакетов нет.

Зависимости для разработки и тестов (`[dependency-groups] dev`)

В боевой образ не попадают (`uv sync --frozen --no-dev`).

Компонент	Ограничение	Версия в `uv.lock`	Лицензия	Назначение
pytest	≥ 8.0	9.1.1	MIT	тесты бэкенда (`backend/tests/`)
pytest-cov	≥ 7.1.0	7.1.0	MIT	покрытие тестами, порог 80 % в `backend/test.sh`
coverage	(через pytest-cov)	7.16.2	Apache-2.0	сбор и отчёт покрытия
locust	≥ 2.46.7	2.46.7	MIT	нагрузочный прогон `backend/loadtest/locustfile.py`

Locust тянет за собой gevent 26.9.0 (MIT), flask 3.1.3 (BSD-3-Clause) и др. — только для нагрузочного прогона.

7.2. Фронтенд

Фронт — один файл `frontend/index.html` с встроенными стилями и скриптом на чистом JavaScript: без сборщика, без фреймворков и без npm-зависимостей во время работы. Внешних загрузок нет: в `index.html` нет `<script src>` и `<link>` на CDN или Google Fonts, все ресурсы лежат в `frontend/assets/` и отдаются тем же сервером.

Ресурс	Лицензия	Где
Шрифт JetBrains Mono (Regular, Medium, SemiBold, Bold, woff2)	SIL Open Font License 1.1 (`OFL.txt` рядом)	`frontend/assets/fonts/`
Логотип, иконки, иллюстрации (`logo.svg`, `mark.png`, `icon-*.png` и т. п.)	фирменный стиль и логотип — из брендбука ФСП, предоставленного организатором (ТЗ, п. 4)	`frontend/assets/`

Ресурс	Лицензия	Где
Рей — CSS-арт в `index.html` (`.ray*`)	нарисован для проекта, стороннего кода нет	`frontend/index.html`

`frontend/package.json` нужен только для e2e-тестов (Playwright) и замера покрытия JS; в образ `api` попадают лишь `index.html` и `assets/` (`.dockerignore` отсекает `node_modules`, `e2e`, отчёты). Требование к Node — `>=22` (`engines`).

Компонент (devDependencies)	Ограничение	Версия в `package-lock.json`	Лицензия	Назначение
@playwright/test	^1.63.0	1.63.0	Apache-2.0	e2e-сценарии кандидата и работодателя в Chromium (`frontend/e2e/`)
monocart-coverage-reports	^2.13.0	2.13.0	MIT	V8-покрытие inline-скрипта `index.html`, порог 80 % строк

Транзитивные пакеты (22 в `package-lock.json`): playwright и playwright-core 1.63.0 (Apache-2.0), istanbul-lib-* (BSD-3-Clause), acorn (MIT), semver и signal-exit (ISC), foreground-child (BlueOak-1.0.0) и другие под MIT. Браузер Chromium ставится командой `npx playwright install chromium`.

7.3. Шрифты PDF-профиля (бэкенд)

Ресурс	Лицензия	Где
JetBrains Mono (Regular, Bold, ttf)	SIL OFL 1.1 (`OFL.txt`)	`backend/app/assets/fonts/`
DejaVu Sans — запасные глифы (арабица, грузинский, часть эмодзи)	лицензия Bitstream Vera / Arev, изменения DejaVu — public domain (`DejaVu-LICENSE.txt`)	`backend/app/assets/fonts/`

7.4. Инфраструктура: контейнеры

Сервисы описаны в `docker-compose.yml`, подробности запуска — в разделе 8.

Компонент	Версия (тег)	Лицензия	Где используется
PostgreSQL	`postgres:16-alpine`	PostgreSQL License	сервис `db`; та же версия — service-контейнер в CI
Python (образ API)	`python:3.12-slim` (Debian)	PSF License	`backend/Dockerfile`, сервис `api`
uv	`ghcr.io/astral-sh/uv:0.10`	MIT / Apache-2.0	установка зависимостей при сборке образа `api`
nsjail	ветка/тег `3.4` репозитория `github.com/google/nsjail`, собирается из исходников	Apache-2.0	`sandbox/Dockerfile`, этап `nsjail`: изоляция каждого запуска кода кандидата
Debian	`debian:bookworm-slim`	свободные лицензии Debian	этап сборки nsjail

Компонент	Версия (тег)	Лицензия	Где используется
Node.js	`node:22-bookworm-slim` (Node 22)	MIT	сервис `sandbox`: исполнение TypeScript (`--experimental-strip-types`)
Python в песочнице	`python3` из Debian bookworm (3.11)	PSF License	сервис `sandbox`: HTTP-сервер песочницы на stdlib и исполнение решений на Python
libprotobuf32, libnl-route-3-200	пакеты Debian bookworm	BSD-3-Clause / LGPL-2.1	библиотеки времени выполнения nsjail

Сервер песочницы (`sandbox/server.py`, `harness.py`, `harness.mts`) написан на стандартной библиотеке Python и Node без сторонних пакетов. Тестовый этап образа песочницы (`--target test`) добавляет `python3-pytest` из Debian.

Теги образов не закреплены по дайджесту, а nsjail клонируется по тегу `3.4` без фиксации коммита: пересборка через полгода может взять более новый патч-релиз базового образа. Для воспроизводимости боевой сборки теги стоит закрепить (`image@sha256:…`) — это доводка за рамками MVP.

7.5. Необязательные модели (LLM, эмбеддинги, OCR)

По умолчанию (`LLM_PROVIDER=none`, `OCR_URL` пустой) ни одна модель не используется и не скачивается — работают правила (подробно — раздел 10). Если провайдер включён, имена моделей по умолчанию заданы пресетами в `backend/app/config.py`; клиент — OpenAI-совместимый HTTP через httpx (`backend/app/services/llm.py`), отдельной SDK-библиотеки нет.

Провайдер / назначение	Модель по умолчанию	Лицензия модели	Где задаётся
`ollama`, чат (разбор текста, реранк)	`mistral:7b`	Apache-2.0	`LLM_MODEL`
`ollama`, эмбеддинги	`bge-m3`	MIT	`EMBED_MODEL`
`yandexgpt`, чат / эмбеддинги	`yandexgpt-lite/latest` / `text-search-doc/latest`	облачный сервис Yandex AI Studio (по договору)	`LLM_MODEL` / `EMBED_MODEL`
`gigachat`, чат / эмбеддинги	`GigaChat` / `Embeddings`	облачный сервис Сбера (по договору)	`LLM_MODEL` / `EMBED_MODEL`
`openai_compatible` (vLLM и т. п.)	не задана — указывается явно	по выбранной модели	`LLM_BASE_URL`, `LLM_MODEL`, `EMBED_MODEL`
OCR сканов резюме (свой vLLM)	`Qwen/Qwen2.5-VL-7B-Instruct`	Apache-2.0	`OCR_URL`, `OCR_MODEL`

Ollama и vLLM в `docker-compose.yml` не входят: их поднимают отдельно (Ollama — на хосте, адрес `OLLAMA_URL`; vLLM — свой сервер с GPU).

7.6. CI (GitHub Actions)

Файл `.github/workflows/ci.yml`; запускается на каждый PR и на пуш в `main`, e2e — только вручную.

Компонент	Версия	Где
Раннер	`ubuntu-latest`	все задачи
actions/checkout	v4	все задачи
astral-sh/setup-uv	v6, Python 3.12	`backend`, `frontend-e2e`
actions/setup-node	v4, Node 22	`frontend-syntax`, `frontend-e2e`
actions/upload-artifact	v4	отчёты покрытия и Playwright
PostgreSQL (service-контейнер)	`postgres:16-alpine`	`backend`, `frontend-e2e`
Docker / Docker Compose раннера	версии раннера GitHub	`sandbox` (тесты против настоящего nsjail, privileged), `frontend-e2e` (песочница для e2e)

7.7. Лицензионная чистота и открытые вопросы

- **Лицензия самого проекта не указана**: файла `LICENSE` в корне репозитория нет. Код открыт (публичный репозиторий `github.com/Little-Suns/fsp-lct`, без компиляции и обфускации — как требует п. 3.5), но условия использования формально не заданы. Это открытый хвост: нужно выбрать лицензию (например, MIT или Apache-2.0, совместимые со всеми зависимостями) и положить `LICENSE` в корень.
- **LGPL-3.0: psycopg и fpdf2.** Это не «MIT / Apache / BSD», но LGPL разрешает использовать библиотеку в приложении под любой лицензией, если библиотека подключается как отдельный модуль и её можно заменить: обе ставятся из PyPI как есть, без изменений и без статической сборки в наш код. Ограничений на открытие нашего кода это не накладывает. Если потребуется строго пермиссивный набор: альтернатива psycopg — psycopg2 тоже LGPL, поэтому реальная замена — asyncpg (Apache-2.0) с переходом SQLAlchemy на async; fpdf2 можно заменить на reportlab (BSD).
- **Прочие нестандартные лицензии** — пермиссивные: MIT-CMU (Pillow, разновидность HPND), PSF (defusedxml, Python), MPL-2.0 (certifi — набор сертификатов, файловый copyleft не затрагивает наш код), CNRI-Python (часть кода `regex`), ISC и BlueOak-1.0.0 (транзитивные пакеты тестов фронта), SIL OFL 1.1 (шрифты).
- **Отечественные облачные LLM** (YandexGPT, GigaChat) — внешние сервисы, а не встраиваемое ПО; подключаются только по желанию и по договору с провайдером. По умолчанию выключены.

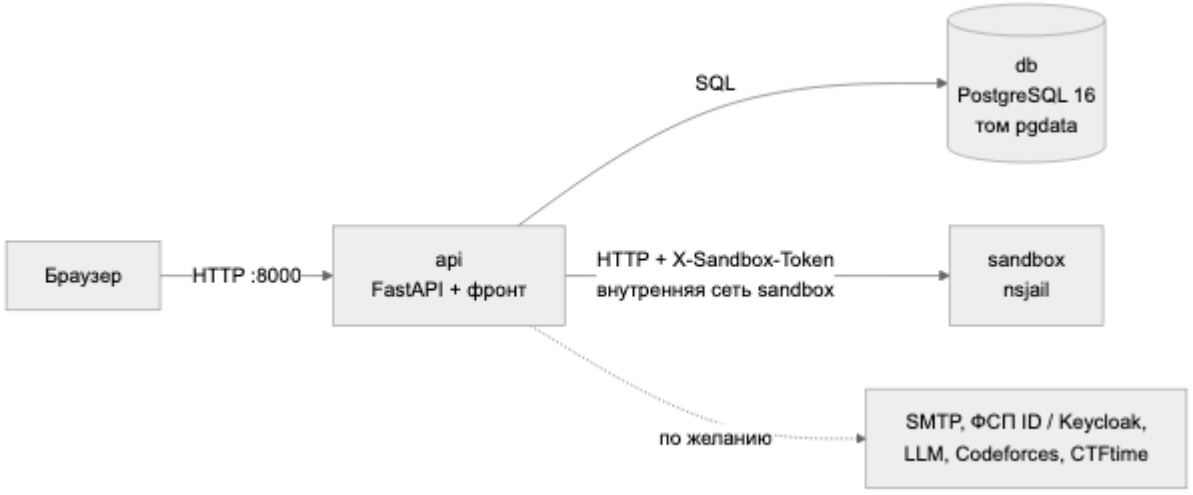
8. Сборка, развёртывание и локальный запуск

Раздел — пошаговая инструкция, по которой эксперт может сам поднять решение: сначала быстрый старт стенда в Docker (раздел 8.3), затем настройки, боевое развёртывание, запуск без Docker, тесты и типовые проблемы. Состав компонентов и их версии — в разделе 7, архитектура — в разделе 1.

8.1. Что разворачивается

Решение собирается в три контейнера (`docker-compose.yml` в корне репозитория):

Сервис	Образ	Что делает	Порты
`db`	`postgres:16-alpine`	PostgreSQL, данные в именованном томе `pgdata`	`127.0.0.1:5432` — только для локальной отладки
`api`	собирается из `backend/Dockerfile` (`python:3.12-slim`)	FastAPI-приложение: API, раздача фронта (`index.html` и `assets/` лежат в образе), миграции и засев при старте	`8000`
`sandbox`	собирается из `sandbox/Dockerfile` (nsjail + Python 3.11 + Node 22)	исполнение кода кандидатов в задачах на код; privileged, только во внутренней сети	наружу не публикуется



Фронт отдельно не собирается: это один HTML-файл без сборщика, его отдаёт тот же процесс API.

8.2. Требования

Машина

ТЗ (п. 3.1) просит обычную рабочую машину или недорогой VPS. Замеры команды (`table.md`, строки «Нагрузочная проверка» и «Ресурсы стенда»; README «Нагрузка и ресурсы»), полный засев — 1500 кандидатов, 250 потребностей, `DEMO=1`, LLM выключена:

Компонент	В простое	50 одновременных пользователей	Ограничение в compose
`api`	119 МиБ, < 1 % CPU	до 253 МиБ, в среднем 60 % одного ядра (пик 115 %)	нет (без пауз — до 390 МиБ, ~1 ядро)
`db`	34 МиБ	до 61 МиБ, 13 % ядра (пик 33 %)	нет
`sandbox`	занят только во время проверки решений	—	4 CPU (`SANDBOX_CPUS`), 3 ГБ (`mem_limit`), 512 процессов

Образы на диске: `api` — 538 МБ, `postgres:16-alpine` — 411 МБ (плюс образ песочницы и слои сборки). GPU не нужен.

Оценка по этим цифрам:

- **без песочницы** (тест без задач на код) — хватает 1–2 CPU и 1 ГБ RAM: стек `db` + `api` под нагрузкой занимает меньше 0,5 ГБ;
- **с песочницей** — рекомендуем 4 CPU и 4 ГБ RAM: каждый одновременный прогон кода занимает 1 CPU и до 256 МБ, по умолчанию их 4 (`SANDBOX_WORKERS`). На машине с меньшим числом CPU уменьшите `SANDBOX_CPUS` и `SANDBOX_WORKERS` (раздел 8.9);
- **диск** — около 5 ГБ под образы, сборку и базу.

Цифры сняты на машине разработчика (12 ядер); на VPS отдельной проверки не было, поэтому это оценка, а не гарантия.

Программное обеспечение

Для чего	Что нужно
Запуск стенда	Docker Engine (или Docker Desktop) и Docker Compose v2 (`docker compose …`); git; `openssl` — сгенерировать секреты
Песочница задач на код	privileged-контейнеры и cgroup v2 на хосте (современный Linux, Docker Desktop на macOS/Windows). Rootless Docker и Podman без privileged не подойдут — тогда тест идёт без задач на код
Разработка бэкенда	Python 3.12 (CI проверяет на 3.12), uv (в образе — 0.10), PostgreSQL 16 (проще всего `docker compose up -d db`)
E2E-тесты фронта	Node.js 22 и npm; Chromium ставит Playwright

8.3. Быстрый старт (стенд для проверки)

```
git clone https://github.com/Little-Suns/fsp-lct.git
cd fsp-lct
cp .env.example .env                                  # образец окружения, все переменные с пояснениями
export SANDBOX_TOKEN=$(openssl rand -hex 32)          # общий секрет API ↔ песочница; без него тест
        идёт без задач на код
export ORGANIZER_TOKEN=$(openssl rand -hex 16)        # вход на /org (импорт протоколов); выведите и
        сохраните: echo $ORGANIZER_TOKEN
DEMO=1 docker compose up -d --build                   # стенд для жюри; без DEMO=1 — боевой режим
docker compose logs -f api                            # дождаться «Application startup complete» (~1
        мин на первом старте)
```

Переменные из командной строки (`DEMO=1`, `SANDBOX_TOKEN`, `ORGANIZER_TOKEN`) важнее значений в `.env`. Чтобы не задавать их каждый раз, впишите `DEMO=1`, `SANDBOX_TOKEN=…` и постоянный `SECRET_KEY=…` (`openssl rand -hex 32`) прямо в `.env`.

Проверка, что API поднялся: `curl http://localhost:8000/api/health` → `{"ok": true}`.

Что открыть

Адрес	Что там
http://localhost:8000	главная: входы «Кандидатам» и «Компаниям», при `DEMO=1` — строка тестовых входов
http://localhost:8000/docs	интерактивная документация API (Swagger, OpenAPI) — см. раздел 6
http://localhost:8000/v	дашборд валидации (без входа, по прямой ссылке) — см. раздел 4
http://localhost:8000/org	импорт протоколов организатором (вход по `ORGANIZER_TOKEN`) — см. раздел 5

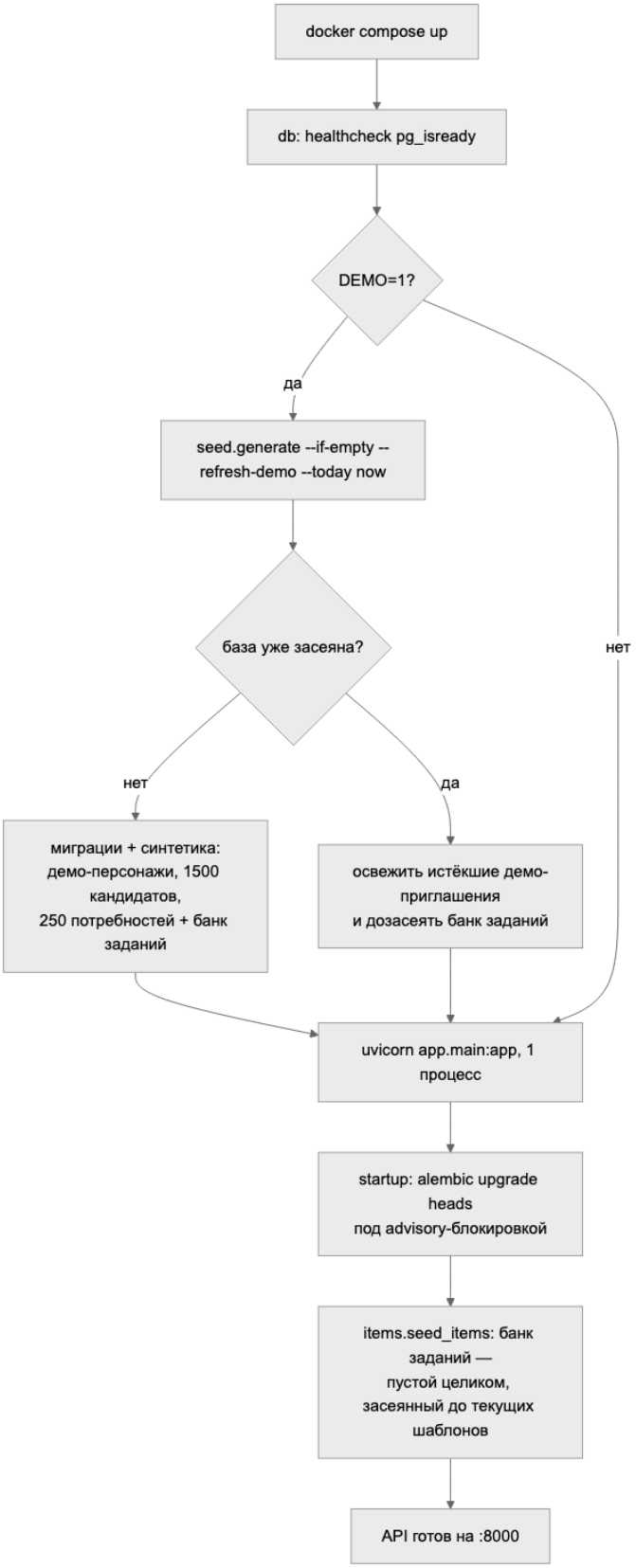
Демо-персонажи и вход

При `DEMO=1` вход идёт по почте, а код подтверждения приходит прямо в ответе сервера и подставляется сам; адреса показаны на главной.

Почта	Кто	Что показывает
`d.orlov@example.ru`	Дмитрий Орлов, кандидат А-1042	кабинет кандидата: открытые приглашения, тест, профиль
`m.kovaleva@ladoga-pay.example`	«Ладога Пэй», рекрутер Мария Ковалёва	кабинет подтверждённой компании: подборка, поиск, приглашения
`newco.hr@gmail.com`	«Новый Стартап»	компания, не прошедшая проверку (квота 3 в неделю, плашка для кандидата)

Можно и зарегистрироваться заново: `/candidate/signup` или `/employer/signup` — код также придёт в ответе. Кнопка «Войти через ФСП ID» при `DEMO=1` ведёт на встроенный мок-провайдер `/mock-fsp-id` со списком демо-аккаунтов (раздел 5). Оба кабинета работают в одном браузере в соседних вкладках.

Что происходит при старте



1. **Миграции.** API при каждом старте накатывает все ревизии Alembic (`alembic upgrade heads`, `app.db.init_db`) под advisory-блокировкой Postgres. База, созданная до перехода на Alembic, сначала помечается ревизией `0001_baseline`, данные сохраняются. Если модель изменили без ревизии или в базе ревизия от другой ветки — API не стартует и пишет в лог, что делать.

2. **Банк заданий теста** засевается при каждом старте в любом режиме (`app.main.startup` → `items.seed_items`, своя advisory-блокировка): пустой — целиком (~1940 заданий, ~1 с), засеянный — приводится к текущим шаблонам, без изменений ничего не пишет. Поэтому на новом стенде тест сразу работает, без ручного засева.
3. **Только при `DEMO=1`**: перед запуском API выполняется `python -m seed.generate --if-empty --refresh-demo --today now`.
    - Пустая база засевается синтетикой от текущей даты (около минуты до ответа API).
    - Засеянная база не пересеивается (признак — есть демо-кандидат Орлов). Демо-приглашения, истёкшие без ответа («Ладога Пэй» и «Кипрей» — Орлову, «Ладога Пэй» — J-4410), снова открываются с теми же сроками, что в свежем засеве (`--refresh-demo`); принятые, отклонённые и ещё открытые не трогаются.
4. **Песочница** стартует параллельно; её здоровье проверяет healthcheck (пробный прогон Python и TypeScript в nsjail). API сам опрашивает `/health` песочницы раз в 30 с и выдаёт задачи на код, только если она отвечает.

Полезные команды на стенде

```
docker compose exec api python -m seed.generate --reset --today now     # пересоздать стенд с нуля: стирает всю базу
docker compose exec api python -m app.services.validation --today now   # пересчитать метрики для /v
docker compose exec api python -m app.services.items                    # вручную дозасеять банк заданий
docker compose exec api python -m app.services.reputation blocked       # компании под блоком
docker compose exec api python -m app.services.reputation unblock 7704123450   # снять блок с демо-компании «Ладога Пэй»
docker compose up -d --build api                                        # пересобрать api после правки кода или фронта
docker compose down                                                     # остановить; данные остаются в томе pgdata
docker compose down -v                                                  # крайняя мера: удалить и том с базой
```

Валидацию на демо-стенде можно запустить и кнопкой «Запустить прогон» на `/v` (около 40 с; второй одновременный прогон — 409; без синтетики в базе — 409 с подсказкой про засев). Через API — `POST /api/validation/run?today=now`, вне демо — только организатору. Результат пишется в `backend/var/validation.json` внутри контейнера и после пересборки образа пропадает — прогон нужно повторить.

8.4. Песочница задач на код

Задача на код в тесте появляется, только если песочница настроена и здорова. Требования:

- **`SANDBOX_TOKEN`** — общий секрет API и песочницы (заголовок `X-Sandbox-Token`). Без него контейнер `sandbox` завершается с сообщением и после трёх перезапусков (`restart: on-failure:3`) остаётся остановленным — так задумано; API при этом просто не выдаёт задач на код, остальной тест работает.
- **privileged и cgroup v2.** nsjail нужны пространства имён и запись в cgroup внутри контейнера, поэтому сервис запущен с `privileged: true` и `cgroup: private`. Корень контейнера — только на чтение, запись — в tmpfs `/run/sbx` (256 МБ).
- **Сеть.** Песочница живёт только во внутренней сети compose `sandbox` (`internal: true`): портов на хост нет, выхода в интернет нет, обращается к ней только `api` по адресу `http://sandbox:8080`.

- **Ресурсы.** `SANDBOX_CPUS` (по умолчанию 4) — потолок CPU контейнера, `SANDBOX_WORKERS` (по умолчанию 4) — одновременных прогонов, каждый занимает 1 CPU и до 256 МБ. `SANDBOX_CPUS` не может быть больше числа CPU хоста — иначе Docker не создаст контейнер.

Модель безопасности песочницы (seccomp, лимиты, изоляция) — раздел 9 и README «Песочница». Ядро у песочницы общее с хостом, поэтому при боевой нагрузке её стоит вынести на отдельную ВМ (или в gVisor/Firecracker) — тогда privileged ограничен этой ВМ.

8.5. Переменные окружения

Compose читает `.env` из корня сам; `.env` в `.gitignore`, секреты в репозиторий не попадают. Пустое значение в `.env` означает значение по умолчанию. `DATABASE_URL` задаёт compose (`postgresql+psycopg://fsp:fsp@db:5432/fsp`), в `.env` его указывать не нужно. Источник правды — `backend/app/config.py` и `docker-compose.yml`.

Режим и секреты

Переменная	По умолчанию	Обязательна	Назначение
`DEMO`	`0`	нет	`1` — стенд жюри: засев пустой базы синтетикой, вход за демо-персонажей, код почты в ответе, мок ФСП ID, отзыв о компании сразу после принятия, кнопка пересчёта на `/v` без входа. `0` — боевой режим
`SECRET_KEY`	случайный на каждый старт процесса	**да, для боевого стенда**	«перец» для HMAC кодов входа и демо-токенов. Без постоянного значения коды и входы сгорают при перезапуске. Генерация: `openssl rand -hex 32`
`PUBLIC_URL`	`http://localhost:8000`	да, если стенд за доменом	адрес стенда в браузере: из него собираются `redirect_uri` ФСП ID, issuer мок-провайдера и ссылки в письмах
`ORGANIZER_TOKEN`	пусто	нет	токен организатора для импорта протоколов (`/org`, `POST /api/achievements/import`) и пересчёта валидации вне демо; пусто — только пользователь с ролью `admin`. В `.env.example` отсутствует, но compose его передаёт — допишите строку в `.env` сами
`WEB_CONCURRENCY`	`1` (задан в `Dockerfile` и compose)	—	число процессов uvicorn; **менять нельзя**, см. раздел 8.6

Песочница

Переменная	По умолчанию	Обязательна	Назначение
`SANDBOX_TOKEN`	пусто	для задач на код	общий секрет API ↔ песочница (`openssl rand -hex 32`); пусто — тест без задач на код
`SANDBOX_WORKERS`	`4`	нет	одновременных прогонов в песочнице (каждый — 1 CPU, до 256 МБ)
`SANDBOX_CPUS`	`4`	нет	лимит CPU контейнера `sandbox`; не больше числа CPU хоста
`SANDBOX_URL`	`http://sandbox:8080` (задан в compose)	—	адрес песочницы для API; вне compose пусто — задачи на код не выдаются

Почта (коды входа и уведомления о приглашениях)

Переменная	По умолчанию	Обязательна	Назначение
`SMTP_HOST`	пусто	**да при `DEMO=0`**	SMTP-сервер. Без него и вне демо вход по почте отвечает 503; при `DEMO=1` код приходит в ответе API
`SMTP_PORT`	`587`	нет	порт; 587 — STARTTLS с проверкой сертификата, 465 — SMTP over SSL
`SMTP_USER`, `SMTP_PASSWORD`	пусто	по требованию сервера	учётные данные SMTP
`SMTP_FROM`	`no-reply@fsp-career.local`	нет	адрес отправителя

ФСП ID (Keycloak, OpenID Connect)

Подробно про поток входа и настройку Keycloak — раздел 5.

Переменная	По умолчанию	Обязательна	Назначение
`FSP_ID_ISSUER`	пусто	для боевого входа	issuer realm Keycloak: `https://<host>/realms/<realm>` (только https). Пусто + `DEMO=1` — встроенный мок `/mock-fsp-id`
`FSP_ID_CLIENT_ID`	`fsp-career`	нет	клиент в Keycloak
`FSP_ID_CLIENT_SECRET`	пусто (для мока — случайный на процесс)	для боевого входа	секрет confidential-клиента
`FSP_ID_MOCK`	пусто (= по `DEMO`)	нет	`1` — включить мок без `DEMO`; рядом с боевым `FSP_ID_ISSUER` мок выключен всегда

Переменная	По умолчанию	Обязательна	Назначение
`ATS_MOCK`	пусто (= по `DEMO`)	нет	`1` — включить демо-приёмник Mock ATS `/mock-ats` без `DEMO`; `0` — выключить при `DEMO=1`
`FSP_ID_IDPS`	пусто (в моке — `yandex:Яндекс ID,corp:Корпоративный SSO`)	нет	сторонние входы через брокер Keycloak: `alias:Подпись,alias2:Подпись2`

LLM и OCR (необязательны)

По умолчанию всё работает на правилах; подробности, пресеты провайдеров и фолбэки — раздел 10.

Переменная	По умолчанию	Назначение
`LLM_PROVIDER`	`none`	`none`, `ollama`, `openai_compatible`, `yandexgpt`, `gigachat`
`OLLAMA_URL`	`http://host.docker.internal:11434/v1`	адрес Ollama на хосте (для `ollama`)
`LLM_BASE_URL`, `LLM_MODEL`, `EMBED_MODEL`	пусто — пресет провайдера	свой адрес и модели; `EMBED_MODEL=none` — без эмбеддингов
`LLM_TIMEOUT`	`60`	таймаут запроса к модели, с
`LLM_API_KEY`	пусто	Bearer для `openai_compatible`, API-ключ сервисного аккаунта для `yandexgpt`
`YANDEX_FOLDER_ID`	пусто	каталог Yandex Cloud (для `yandexgpt`)
`GIGACHAT_AUTH_KEY`, `GIGACHAT_SCOPE`	пусто, `GIGACHAT_API_PERS`	ключ авторизации и scope GigaChat
`LLM_CA_BUNDLE`	пусто	путь внутри контейнера к PEM «Russian Trusted Root CA» (для GigaChat; файл нужно смонтировать)
`OCR_URL`	пусто — OCR выключен	адрес своего vLLM с vision-моделью (только self-hosted: скан уходит в модель с ПДн)
`OCR_MODEL`	`Qwen/Qwen2.5-VL-7B-Instruct`	имя модели OCR
`OCR_API_KEY`	пусто	Bearer, если vLLM запущен с `--api-key`
`OCR_TIMEOUT`	`120`	общий срок распознавания документа, с

Переменные только для запуска без compose

`backend/app/config.py` читает ещё несколько переменных, которые `docker-compose.yml` в контейнер не передаёт (чтобы задать их в Docker, добавьте строку в `environment` сервиса `api`): `DATABASE_URL` (по умолчанию `postgresql+psycopg://fsp:fsp@localhost:5432/fsp`), `FRONTEND_DIR` (каталог фронта; в образе — `/srv/frontend`), `SMTP_STARTTLS` (`1`), `FSP_ID_REDIRECT_URI` (по умолчанию `PUBLIC_URL/api/auth/fsp/callback`), `FSP_ID_SCOPE` (`openid email profile`), `GIGACHAT_OAUTH_URL`, `SANDBOX_TIMEOUT` (90 с), `SANDBOX_API_SLOTS` (8 одновременных обращений API к песочнице).

8.6. Боевое развёртывание

Отличия боевого стенда от демо:

1. **`DEMO=0`** (значение по умолчанию). Демо-входы, код в ответе, мок ФСП ID и пересчёт валидации без входа выключены. База не засевается синтетикой — только банк заданий теста.

2. **Постоянный `SECRET_KEY`** (`openssl rand -hex 32`, хранить вне репозитория). Иначе после каждого перезапуска выданные коды входа перестают сходиться.

3. **SMTP** (`SMTP_HOST` и др.). Без него вход по почте отвечает 503. С SMTP кандидат получает и письмо о новом приглашении — со ссылкой по `PUBLIC_URL`.

4. **`PUBLIC_URL` на https-домен стенда.** Из него строятся redirect URI ФСП ID и ссылки в письмах; без https кнопка ФСП ID выключена.

5. **ФСП ID** — только конфигурацией: `FSP_ID_ISSUER`, `FSP_ID_CLIENT_ID`, `FSP_ID_CLIENT_SECRET`; в Keycloak — confidential-клиент с redirect URI `PUBLIC_URL/api/auth/fsp/callback`. Требования к настройке брокера (Trust Email, автопривязка) — раздел 5 и README «Безопасность настройки брокера».

6. **`ORGANIZER_TOKEN`** — если протоколы импортирует организатор без отдельной учётки admin.

7. **Постоянный том БД.** Данные Postgres лежат в именованном томе `pgdata`; не удаляйте его (`docker compose down -v`). Пароль базы в compose — учебный (`fsp`/`fsp`), а порт опубликован только на `127.0.0.1`; для боевого стенда смените пароль (в `db` и в `DATABASE_URL` сервиса `api`) или уберите публикацию порта.

8. **HTTPS-прокси перед `api`** (nginx, Caddy, Traefik или встроенный в панель): TLS, домен → порт 8000. Учтите:

    - SSE чата (`GET /api/chat/stream`) — долгое соединение: буферизацию на прокси выключить (API шлёт `X-Accel-Buffering: no`, nginx его понимает), таймаут чтения — не меньше нескольких минут;
    - размер тела: API отвечает 413 на `Content-Length` больше 6 МБ; запросы без длины (chunked) режет только прокси — задайте `client_max_body_size` ≈ 6 МБ;
    - лимиты частоты по IP (отправка кода входа, пересчёт валидации, «запуск на примерах» в задачах на код) в приложении не сделаны и ставятся на прокси (`table.md`, строки «Rate limit на отправку кода» и про песочницу).

9. **Один процесс uvicorn** (`WEB_CONCURRENCY=1`, не менять). В памяти процесса живут: подписчики SSE чата (`services/chat.Hub`), троттлинг запросов к Codeforces и CTFtime, одна проверка кода на сессию (`quest._one_check`), кэш `/health` песочницы, мок ФСП ID и случайный `SECRET_KEY`, если он не задан. Второй воркер ломает доставку сообщений и ограничения. Запаса одного процесса хватает: 50 одновременных пользователей — 0 ошибок, ~29 запросов/с, p95 подборки 170 мс; предел одного процесса — ~72 запроса/с (README «Нагрузка и ресурсы», `backend/loadtest/`). Масштабирование на несколько процессов потребует вынести это состояние в Redis/Postgres (`table.md`).

10. **Резервные копии.** Встроенного бэкапа нет. Минимальный вариант — регулярный дамп по cron:

```
docker compose exec -T db pg_dump -U fsp -Fc fsp > fsp-$(date +%F).dump          # копия
docker compose exec -T db pg_restore -U fsp -d fsp --clean --if-exists < fsp-2026-10-09.dump
        # восстановление
```

Перед восстановлением остановите `api` (`docker compose stop api`), после — запустите: миграции при старте доведут схему до текущей версии.

11. **Песочница** — на отдельной ВМ при реальной нагрузке (раздел 8.4).

Опыт развёртывания в Dokploy

Стенд команды развёрнут в Dokploy как Docker Compose из git. Что из этого следует (README «Запуск», `table.md`):

- При каждом деплое Dokploy заново клонирует репозиторий и удаляет прежний каталог, поэтому в `docker-compose.yml` нет монтирований из каталога репозитория: база — в томе `pgdata`, фронт — внутри образа `api`. Изменили фронт — нужен редеплой (пересборка образа).
- Домен назначается в Dokploy на порт 8000 сервиса `api`. `docker-compose.override.yml` Dokploy не подхватывает (он запускает compose-файл явно через `-f`) — не копируйте override на стенд.
- Переменные окружения (`DEMO`, `SECRET_KEY`, `SANDBOX_TOKEN`, `PUBLIC_URL` …) задаются в интерфейсе Dokploy.
- При `DEMO=1` за доменом мок ФСП ID ходит к самому себе по HTTP через `PUBLIC_URL`, как к настоящему Keycloak, поэтому домен стенда должен быть доступен изнутри контейнера `api`.
- Банк заданий засевается при каждом старте API, отдельный шаг засева на новом стенде не нужен. Любая сумма зарплаты ограничена 10 млн ₽ (`catalog.SALARY_MAX`), сверх неё — 422.

8.7. Локальная разработка без Docker

Нужны Python 3.12, uv и PostgreSQL 16. Проще всего базу взять из compose, а API запустить на хосте:

```
docker compose up -d db                                  # только Postgres на 127.0.0.1:5432 (fsp/fsp,
        база fsp)
cd backend
uv sync                                                  # зависимости, включая dev (pytest, locust)
uv run python -m seed.generate --reset                   # схема + синтетика (дата засева по умолчанию
        2026-10-05)
DEMO=1 uv run uvicorn app.main:app --port 8000           # API + фронт на http://localhost:8000
```

- Свой Postgres: `DATABASE_URL=postgresql+psycopg://<user>:<pass>@<host>:<port>/<db>` перед командами.
- Фронт раздаётся из `frontend/` рабочей копии (`FRONTEND_DIR` по умолчанию указывает туда) — правки `index.html` видны после обновления страницы.
- Для демонстрации интерфейса «как сегодня» засевайте от текущей даты: `seed.generate --reset --today now`. Без `--today` сроки приглашений отсчитываются от 2026-10-05 и к реальному дню могут истечь — цифры валидации при этом воспроизводимы.
- Миграции вручную тем же путём, что при старте: `uv run python -c "from app.db import init_db; init_db()"`; текущая ревизия — `uv run alembic current`. Изменение схемы — только вместе с ревизией Alembic (`CLAUDE.md`).
- Задачи на код локально: поднять песочницу отдельно и передать адрес — `docker build --target runtime -t fsp-sandbox sandbox && docker run -d --rm --name sbx --privileged -p 127.0.0.1:8091:8080 -e SANDBOX_TOKEN=dev fsp-sandbox`, затем `SANDBOX_URL=http://127.0.0.1:8091`

`SANDBOX_TOKEN=dev` перед `uvicorn`.

Валидация

Метрики подбора и теста (раздел 4) считаются на отдельной базе, чтобы не трогать рабочую:

```
cd backend
DATABASE_URL=postgresql+psycopg://fsp:fsp@localhost:5432/fsp_valid uv run python -m seed.generate --reset
DATABASE_URL=postgresql+psycopg://fsp:fsp@localhost:5432/fsp_valid uv run python -m app.services.validation
```

База `fsp_valid` должна существовать (`createdb`, или её создаёт `./backend/test.sh`). Синтетика детерминирована по `--seed` (42) и `--today` (2026-10-05), поэтому цифры совпадают с README «Результаты валидации» в любой день. Отчёт печатается в консоль и пишется в `backend/var/validation.json` — его показывает `/v`.

8.8. Тесты

Что	Команда	Условия
Бэкенд, все наборы + покрытие	`./backend/test.sh`	Postgres на `localhost:5432` (`docker compose up -d db`); другой сервер — `PG=postgresql+psycopg://fsp:fsp@localhost:5434 ./backend/test.sh`. Скрипт пересоздаёт свои базы `fsp_*` (демо-базу `fsp` не трогает), гоняет каждый набор в своей базе с `DEMO=1` и падает при покрытии ниже 80 % (`COV_MIN`)
Один набор	`cd backend && DEMO=1 DATABASE_URL=postgresql+psycopg://fsp:fsp@localhost:5432/fsp_auth uv run pytest -q tests/test_auth.py`	база набора должна существовать; `-m 'not slow'` пропускает наборы с засевом
Самопроверки модулей	`cd backend && uv run python -m app.services.irt` (и `app.services.lottery`, `app.services.parsing`, `app.services.autocheck`, `app.integrations.scoring`, `seed.gold`)	без базы
E2E фронта	`cd frontend && npm ci && npx playwright install chromium && npm test`	Node 22, Postgres на 5432. Playwright сам поднимает бэкенд на порту 8010 (`E2E_PORT`) с базой `fsp_e2e` и пересевом на каждый прогон; порог покрытия JS — 80 % строк, отчёт `frontend/coverage/index.html`
E2E с задачей на код	песочница как в 8.7, затем `SANDBOX_URL=http://127.0.0.1:8091 SANDBOX_TOKEN=dev npm test`	без `SANDBOX_URL` сценарий `09-code` пропускается

Что	Команда	Условия
Песочница против настоящего nsjail	`docker build --target test -t fsp-sandbox-test sandbox && docker run --rm --privileged -v "$PWD/backend/app/services/code_items.py:/srv/code_items.py:ro" fsp-sandbox-test`	из корня репозитория, Docker с privileged
Нагрузка	`cd backend && uv run locust -f loadtest/locustfile.py --headless -u 50 -r 10 -t 2m --host http://127.0.0.1:8000 --csv var/load`	API с `DEMO=1` на полном засеве
Живые внешние API и модели	`LIVE=1` (Codeforces, CTFtime), `LIVE_LLM=1 … tests/test_llm_live.py`	только вручную; в обычных тестах внешние API замоканы

CI (`.github/workflows/ci.yml`) на каждый PR гоняет бэкенд-тесты с покрытием, самопроверки, тесты песочницы против nsjail и проверку синтаксиса JS; e2e — вручную (Actions → CI → Run workflow).

8.9. Типовые проблемы и решения

Симптом	Причина	Что сделать
`docker compose up`: порт 8000 или 5432 занят (`port is already allocated`)	на хосте уже работает другой сервис или локальный Postgres	остановить его; либо поменять публикацию в `docker-compose.yml` (`ports` у `db` нужен только для отладки — строку можно убрать; у `api` — например `"8080:8000"`). В `docker-compose.override.yml` порты не заменяются, а добавляются к основным, поэтому правьте основной файл или используйте `!override`. При другом порту `api` задайте `PUBLIC_URL` на новый адрес; мок ФСП ID в этом случае может не работать, потому что контейнер ходит к себе по `PUBLIC_URL`
Тесты или e2e не подключаются к базе	порт 5432 занят чужим Postgres без пользователя `fsp`	поднять базу compose на свободном порту и передать `PG=…:<порт>` (test.sh, e2e `server.sh`)
Контейнер `sandbox` не создаётся: ошибка про `cpus` / range of CPUs	на хосте меньше 4 CPU	в `.env`: `SANDBOX_CPUS=<число CPU>`, `SANDBOX_WORKERS` — не больше него
`sandbox` остановлен, в тесте нет задач на код	не задан `SANDBOX_TOKEN` (контейнер выходит по задумке)	задать `SANDBOX_TOKEN` и `docker compose up -d`
В логе `sandbox`: «/sys/fs/cgroup недоступен на запись — контейнеру нужен privileged»	Docker без privileged, cgroup v1 или rootless	использовать Docker с cgroup v2 и privileged; иначе работать без задач на код
Первый ответ API приходит через минуту	при `DEMO=1` идёт засев пустой базы	дождаться «Application startup complete» в `docker compose logs -f api`

Симптом	Причина	Что сделать
Стенд пустой: нет демо-персонажей и подборок	запущен без `DEMO=1` (боевой режим синтетику не сеет) или переменная не дошла до compose	`DEMO=1 docker compose up -d` (засев пустой базы) или `docker compose exec api python -m seed.generate --reset --today now`
`POST /api/quest/start` → 503 «Банк заданий пуст»	старая версия без засева банка при старте	обновить код и перезапустить `api`; вручную — `docker compose exec api python -m app.services.items`
`GET /` → 503 «Фронт недоступен»	в образе или в смонтированном каталоге нет `index.html`	пересобрать образ (`docker compose up -d --build api`); с override — не пересоздавать каталог `frontend/` под работающим контейнером
API не стартует, в логе — про ревизию Alembic	модель изменена без ревизии или в базе ревизия другой ветки	следовать подсказке в логе; для стенда проще всего `seed.generate --reset`
Вход по почте → 503	`DEMO=0` и не задан `SMTP_HOST`	задать SMTP или поднять стенд с `DEMO=1`
Коды входа и демо-входы перестают работать после перезапуска	не задан `SECRET_KEY`, ключ случайный на процесс	задать постоянный `SECRET_KEY` в `.env`
У демо-кандидата нет открытых приглашений	демо-приглашения истекли (срок 7 дней от засева)	перезапустить `api` (`docker compose restart api` — `--refresh-demo` снова откроет их) или «Пригласить снова» в карточке кандидата; показать «как в макете» — `seed.generate --reset --today now`
«Ладога Пэй» не может приглашать, вакансии скрыты	компанию заблокировали отзывы «вакансии не было» (в демо отзыв доступен сразу после принятия)	`docker compose exec api python -m app.services.reputation unblock 7704123450`
`/v` пустой	валидация ещё не запускалась или образ пересобран	кнопка «Запустить прогон» на `/v` (при `DEMO=1`) или `docker compose exec api python -m app.services.validation --today now`
Кнопка «Войти через ФСП ID» неактивна	боевой `FSP_ID_ISSUER` при `PUBLIC_URL` не на https	задать https-адрес в `PUBLIC_URL` (или `FSP_ID_REDIRECT_URI`)
`LLM_PROVIDER=ollama`, но разбор идёт по правилам	Ollama недоступна из контейнера или модели не скачаны	`ollama serve` на хосте, `ollama pull mistral:7b bge-m3`; адрес — `OLLAMA_URL`. Сбой модели не ломает работу — после ошибки модель не вызывается минуту, работают правила

9. Безопасность и 152-ФЗ

ТЗ (п. 3.5) требует соблюдать 152-ФЗ «О персональных данных», получать согласие пользователя на обработку и публикацию профиля, разграничивать права по ролям, защищать от несанкционированного доступа, безопасно хранить пароли и токены. Раздел описывает, как это сделано в коде и где проходят границы MVP.

Термины:

- **ПДн** — персональные данные: ФИО, почта, Telegram, текст резюме кандидата, контакты рекрутера.
- **HMAC** — хеш с секретным ключом. Без ключа его нельзя ни подделать, ни перебрать.
- **Перец** — серверный секретный ключ `SECRET_KEY`, который хранится вне базы.

9.1. Согласия кандидата

Три раздельных согласия

Цель	Флаг и версия в `Candidate`	Что разрешает
Обработка ПДн (`processing`)	`consent_152`, `consent_version`, дата `consent_152_at`	хранить и обрабатывать профиль; без неё профиль не создаётся (422)
Публикация карточки (`publish`)	`consent_publish`, `consent_publish_version`	показывать обезличенную карточку работодателям в подборке, поиске и лотерее
Внешние данные (`external`)	`consent_external`, `consent_external_version`	привязывать Codeforces и CTFtime, принимать достижения из протоколов ФСП, хакатонов и из ФСП ID

Согласия даются отдельными галочками при регистрации, а меняются в настройках:

- `PATCH /api/candidate/consents` — обработка и публикация;
- `POST /api/achievements/consent` — внешние данные.

Публикация и внешние данные зависят от обработки. Включить их без действующего согласия на обработку нельзя (422 или 403).

Версия текста

Текущая редакция текста согласия — `catalog.CONSENT_VERSION` (сейчас `"2026-10"`). Согласие действует, только если и флаг включён, и версия этой цели совпадает с текущей. Проверка — одна функция `matching.consent_ok(c, purpose)`.

Если текст согласия сменили, у всех кандидатов `consent_ok` становится ложным. Они пропадают из выдачи и не могут менять свои ПДн, пока не подтвердят согласие заново (`OUTDATED` в `routers/candidate.py`). У каждой цели своя версия: повторное согласие на обработку не «оживляет» публикацию, данную на старую редакцию.

Журнал согласий

Каждое изменение по любой цели пишется в `ConsentEvent`: кандидат, цель, выдано или отозвано, версия текста, время. Это бывает при создании профиля, изменении, повторном согласии на новую редакцию и отзыве. Таблица только дополняется и не имеет внешнего ключа на кандидата: доказательство согласия переживает удаление аккаунта. Дополнительно в `AuditLog` пишется `consent_changed` со снимком флагов и версий до и после (без ПДн).

Отзыв согласия

Отзыв согласия на обработку (`candidate._withdraw_processing`):

- снимает публикацию;
- отзывает согласие на внешние данные и удаляет внешние аккаунты и достижения всех внешних источников (Codeforces, CTFtime, хакатоны, ФСП — `integrations/consent.py`);
- переводит профиль в статус «скрыт».

После отзыва запрещена любая запись ПДн (`_require_152`). Разрешены только действия, сужающие обработку: смена согласий, удаление аккаунта, скрытие профиля, отказ от приглашения, отзыв отклика, исключение мест работы из PDF. Чтение своих данных тоже остаётся: `/me`, выгрузка, PDF-профиль. Работодатель перестаёт видеть контакты и диалог (`chat.may_talk` требует действующего согласия).

Сами данные профиля при отзыве не стираются. Полное удаление — отдельным действием «Удалить аккаунт» (9.3).

Отзыв и принятие приглашения сериализуются блокировкой строки кандидата (`SELECT … FOR UPDATE`, `consent.lock_candidate`). Поэтому отзыв не может «проскочить» между проверкой согласия и раскрытием контактов.

9.2. Что видит работодатель

До принятия приглашения или отклика

Работодатель видит только анонимную карточку `matching.card`. Она собирается по белому списку полей:

- анонимный код `public_id` (вида `A-1042`);
- специализация, трек, грейд по тесту, заявленный грейд;
- перцентиль, θ ± SE (θ — оценка уровня по IRT, SE — её погрешность, раздел 2), компетенции;
- подтверждённые тестом и заявленные навыки, роли и софт-скиллы со слов кандидата;
- достижения (ФСП, рейтинг Codeforces, бейджи, разряд);
- ожидания по зарплате, формат работы, город;
- нагрузка открытыми приглашениями;
- оценка соответствия и её объяснение;
- сигналы доверия (тест, подтверждения работодателей, решённые задания).

ФИО, почты, Telegram и резюме в карточке нет.

Как это защищено:

- **Поиск по банку не читает ПДн из БД.** Колонки ФИО, почты, Telegram, резюме, эмбеддинга и `truth` отложены с `raiseload` (`employer.SEARCH_SKIP`): случайное обращение к ним — ошибка, а не утечка.
- **Видимость кандидата.** Кандидат виден работодателю только при выполнении `matching.visible`: профиль открыт, тест пройден, действует согласие на публикацию (а значит, и на обработку), компания не внесена в его список «скрыть профиль от» (по ИНН). Иначе — 404 в карточке, нет в подборке, поиске, лотерее, и приглашение не отправить.
- **Заметка к отклику.** Кандидат мог по просьбе фиктивной вакансии вписать контакты в заметку. До принятия отклика почта, телефон, Telegram, ссылки и слова ФИО самого кандидата в заметке маскируются (`vacancies._note`, `parsing.mask_contacts`).
- **Лотерея.** Публичная часть оперирует псевдонимами `sha256(salt + public_id)`: без соли псевдоним не связать с кандидатом и с другими лотереями.
- **В ранжировании нет ФИО, пола, возраста и вуза** (раздел 3).

После согласия кандидата

Контакты (ФИО, почта, Telegram) открываются работодателю по одному из двух оснований (ТЗ, п. 2.2):

Основание	Где открываются	Условие
Кандидат принял приглашение	статусы приглашений, `GET /api/employer/pipeline` (`employer._row`)	`status = accepted` и `chat.may_talk`
Компания приняла отклик кандидата	отклики на вакансию (`vacancies._shown`)	`status = accepted` и `chat.may_talk`

`may_talk` — одно правило для контактов и чата. Оно требует трёх условий: согласие на обработку действует, компании нет в «скрыть от», компания не заблокирована по отзывам. Если кандидат позже отзовёт согласие или скроет профиль от компании, контакты и диалог закрываются снова, а работодатель видит причину без подробностей.

Отклик без ответа контакты не открывает. Это сознательное отступление от буквы ТЗ («…или откликнулся сам»): иначе фиктивная вакансия собирала бы контакты всех откликнувшихся (раздел 11). Отказ компании и просмотр отклика контакты тоже не открывают.

Сам кандидат ещё до ответа видит название компании, вилку и контакт рекрутера. Контакт рекрутера — часть приглашения, а не ПДн кандидата.

Журнал раскрытий

Каждое раскрытие контактов пишется в `AuditLog` с действием `contacts_revealed`:

- по приглашению — в момент принятия (`candidate.accept`), с `invitation_id` и `company_id`;
- по отклику — один раз на отклик (`vacancies._reveal`, условный `UPDATE … WHERE contacts_revealed_at IS NULL`): повторные просмотры запись не дублируют.

`AuditLog` только дополняется и не имеет внешних ключей, поэтому запись переживает удаление аккаунта. Другие действия в журнале: `data_exported`, `account_deleted`, `consent_changed`, `test_reset`, `grade_lowered`, `role_changed`, `fsp_id_linked`, `fsp_id_unlinked`, `protocol_imported`, `company_blocked`, `company_unblocked`. В `detail` пишутся только идентификаторы и основания, без ПДн.



9.3. Права субъекта ПДн: выгрузка и удаление

- **Выгрузка своих данных** (`GET /api/candidate/me/export`, ст. 14 152-ФЗ) — один JSON со всеми данными кандидата: аккаунт, профиль, вектор эмбеддинга, журнал согласий, записи `AuditLog` о нём, достижения, внешние аккаунты, приглашения, отклики, переписка, решения заданий, сессии теста. Не выгружаются только секреты: хеши токена и кода, код подтверждения внешнего профиля. Ответ помечается `Cache-Control: no-store`. Факт выгрузки пишется в журнал (`data_exported`) до снимка, поэтому сам попадает в выгрузку.
- **Удаление аккаунта** (`DELETE /api/candidate/me`). Удаляются кандидат и пользователь. Каскадом уходят достижения, внешние аккаунты, приглашения, отклики, сообщения чата, решения заданий и сессии теста. В журнале остаётся `account_deleted` только с идентификаторами. `public_id` удалённого переносится в `RetiredPublicId` и повторно не выдаётся: иначе старая лотерея или воронка «узнала» бы в новом человеке прежнего.
- **Резюме до согласия не хранится.** PDF, загруженный до создания профиля, сервер разбирает и возвращает текстом (`no-store`), но не сохраняет. Резюме записывается только вместе с согласием на обработку в `POST /api/candidate/profile`.
- Кандидат может скрыть профиль целиком (`PATCH /api/candidate/status`), скрыть его от конкретных компаний по ИНН (`PATCH /api/candidate/hidden-from`) и исключить распознанные места работы из PDF-профиля (`PATCH /api/candidate/pdf`).

9.4. Хранение секретов

Секрет	Как хранится	Где
Токен сессии (Bearer)	в БД — только sha256 (`deps.hash_secret`); открытое значение отдаётся клиенту один раз; срок 30 дней; новый вход отзывает прежний токен, выход стирает	`users.token`, `deps.issue_token`
Код входа (6 цифр)	HMAC-SHA256 с перцем `SECRET_KEY` и адресом (`deps.otp_hash`): шестизначный код по голому sha256 перебирается за секунды; срок 10 минут, 5 попыток; попытка списывается атомарно до сравнения, верный код удаляется атомарно; новый код — не чаще раза в 60 с на адрес	`login_challenges.code_hash`
Код подтверждения Codeforces	sha256, срок 24 часа, не больше 5 проверок на код, после успешной проверки стирается	`external_accounts.verify_code`

Секрет	Как хранится	Где
Поток входа через ФСП ID	`state`, кука браузера `fsp_flow` (HttpOnly, SameSite=Lax) и одноразовый билет на обмен — только sha256; PKCE-верификатор и nonce не хранятся вовсе, а выводятся HMAC от `state`; билет живёт 60 с	`fsp_logins`
Ключ API для ATS	`fsp_` + 32 случайных байта; в БД — только sha256 (`deps.hash_secret`) и начало ключа для списка; полный ключ показывается один раз при выпуске; отзыв — `revoked_at`; не больше 10 активных на компанию	`api_keys`
Общий демо-токен	HMAC адреса демо-персонажа на `SECRET_KEY`; при `DEMO=0` отвергается в `deps.current_user`	—
Пароли	паролей нет: вход по коду на почту или через ФСП ID	—
Ключи и пароли интеграций	только из окружения: `SECRET_KEY`, `SMTP_PASSWORD`, `FSP_ID_CLIENT_SECRET`, `LLM_API_KEY`, `GIGACHAT_AUTH_KEY`, `ORGANIZER_TOKEN`, `SANDBOX_TOKEN`	`backend/app/config.py`, `.env.example`

Секретные токены (организатора, песочницы, nonce ФСП ID, хеши лотереи) сравниваются за постоянное время: `secrets.compare_digest` и `hmac.compare_digest`.

Если `SECRET_KEY` не задан, ключ генерируется случайно при старте процесса. Коды входа и демо-токены тогда сгорают при перезапуске. На стенде ключ задаётся постоянным.

В тестах токен выдаётся через `deps.issue_token`. Утечка дампа БД не даёт войти ни по токену, ни по коду.

Интеграция с ATS (раздел 11.3) не расширяет права работодателя: события в ленте `/api/ats/v1/events` собираются при чтении по правилам кабинета. Анонимная карточка отдаётся только при действующем согласии кандидата и если компания не скрыта; контакты — только после принятия отклика или приглашения. Выпуск и отзыв ключа пишутся в `AuditLog` (`api_key_issued`, `api_key_revoked`, в detail — идентификаторы, без ПДн). Хранение данных, выгруженных в ATS, — ответственность работодателя как оператора ПДн.

9.5. Роли и разграничение доступа

Аутентификация — Bearer-токен в заголовке `Authorization`. Права проверяют зависимости FastAPI в `backend/app/deps.py`:

Зависимость	Что проверяет	Ответ при отказе
`current_user`	токен есть, его хеш найден, срок не истёк; общий демо-токен вне `DEMO=1` не действует	401
`current_candidate`	у пользователя есть профиль кандидата	403

Зависимость	Что проверяет	Ответ при отказе
`current_company`	у пользователя есть компания	403
`active_company`	компания не заблокирована по отзывам — для действий, доходящих до кандидатов: приглашения, вакансии, задания, лотерея, ответы на отклики	403
`achievements.organizer`	`ORGANIZER_TOKEN` из окружения или пользователь с ролью `admin` — для импорта протоколов и (вне демо) пересчёта валидации	401 / 403
Ключ API (`X-Api-Key`, `/api/ats/v1`)	ключ найден по sha256 и не отозван; у него есть нужная область (`vacancies:write`, `applications:read`, `applications:write`, `invites:read`, `invites:write`); записывающие запросы заблокированной компании отклоняются, как у `active_company`	401 / 403
`mock_ats`	демо-приёмник ATS включён (`ATS_MOCK=1`, по умолчанию вместе с `DEMO=1`)	404
`mock_fsp_id._enabled`	мок-провайдер ФСП ID включён (`DEMO=1` или `FSP_ID_MOCK=1`)	404

Дополнительно:

- **Роль при создании профиля и компании.** Профиль кандидата создаёт только пользователь с ролью `candidate`, компанию — только `employer`. Роль закрепляется при первом входе: пользователь создаётся только после подтверждения кода, так что роль выбирает владелец адреса. Роль меняется, лишь пока нет ни профиля, ни компании.
- **Владелец записи.** Каждый объект проверяется на принадлежность: чужое приглашение, отклик, потребность, вакансия, задание, лотерея — 404. Кандидат видит только свои приглашения и отклики (ТЗ, п. 2.1).
- **Неподтверждённая компания** работает сразу, но с урезанной квотой (3 приглашения в неделю) и лимитом 2 открытых вакансий. Кандидат видит отдельные факты, а не общее «компания проверена»: «почта на домене подтверждена / не подтверждена» и, если не сходится, «ИНН: контрольные цифры не сходятся» (`employer.company_trust`). Подтверждение — контрольная сумма ИНН плюс почта входа на домене сайта компании, подтверждённая нашим кодом. Почте из Keycloak и сторонних IdP («Trust Email») для статуса компании не доверяем.
- **Защита от предзахвата аккаунта через ФСП ID.** Если ФСП ID привязали к аккаунту, почту которого владелец не подтверждал кодом, при первом входе владельца по коду привязка сбрасывается (`fsp_id_unlinked` в журнале). Подробности OIDC-клиента (проверка подписи id_token по JWKS, iss/aud/azp/exp/nonce, PKCE S256, только https у боевого issuer) — раздел 5.
- **Заголовки безопасности** на каждый ответ (`main.security_headers`):
  - `X-Frame-Options: DENY` и `Content-Security-Policy: frame-ancestors 'none'` — чужой сайт не встроит кабинет в невидимый фрейм и не подставит под клик кнопку «Принять и открыть мои контакты»;
  - `X-Content-Type-Options: nosniff`;
  - `Referrer-Policy: same-origin`.

- **Размер запроса.** Тело больше 6 МБ отвергается (413, `main.limit_body`). Файл резюме — до 5 МБ, протокол организатора — до 2 МБ.
- **Наружу отдаются только страница и `/assets`:** `node_modules`, e2e-сценарии и отчёты тестов из каталога фронта не видны.
- **Фронт экранирует все строки из API** (`esc()`). E2E-сценарии включают проверку XSS.
- **Проверка в тестах.** `backend/tests/test_privacy.py` обходит все ответы компании до и после принятия и проверяет отсутствие ПДн. Тест `test_search_does_not_load_pii_columns` проверяет, что поиск не читает колонки ПДн.

9.6. ПДн не в логах и не в LLM

Логи. В логи и `AuditLog.detail` пишутся только идентификаторы, типы ошибок и коды ответов, без самих данных:

- сбой письма — тип исключения без адреса (`mail.py`);
- сбой OCR, LLM, песочницы — тип ошибки и HTTP-код;
- PDF-профиль — `public_id` и число незнакомых символов.

LLM. Модель необязательна (раздел 10). Если она включена, в неё уходит только:

Вызов	Что отправляется
Разбор потребности и резюме (`parsing.parse_need`, `parse_cv`)	текст после `parsing.mask_pii`: ФИО, почта, телефон, Telegram, ссылки на профили, адрес, дата рождения, гражданство заменяются метками. Контакты для формы кандидата распознаются правилами локально, до маскировки
Реранк подборки (`matching._llm_scores`)	только структурные поля: `public_id`, грейд, трек, θ, навыки; без ФИО и без текста потребности
Эмбеддинги	строка «специализация · трек · грейд. Навыки: …»

Маскировка сделана на правилах, а не на NER. Редкое имя вне шапки резюме может пройти (`table.md`, строка «Маскировка ПДн — правила, не NER»). Для облачной модели в бою планируется подключить NER (Natasha).

OCR — исключение: картинку скана не замаскировать. Поэтому `OCR_URL` допускается только self-hosted (свой vLLM), не облачный API. В README и в `config.py` это сказано прямо.

Внешние песочницы (e2b и подобные) отклонены по той же причине: код кандидата не должен уходить за пределы нашей инфраструктуры.

9.7. Песочница для кода кандидатов

Задачи на код в тесте исполняют недоверенный код. Модель безопасности (`sandbox/`, nsjail 3.4, подробно — README «Песочница»):

- **Сеть.** Отдельный контейнер `sandbox` во внутренней сети compose (`internal: true`): портов на хост нет, выхода наружу нет. Говорит с ним только `api` по общему секрету `SANDBOX_TOKEN` (заголовок `X-Sandbox-Token`, сравнение за постоянное время).

- **Изоляция каждого теста.** Каждый тест — отдельный запуск nsjail со своими пространствами имён (пустая сеть, pid, mount, user, uts, ipc, cgroup). Пользователь uid/gid 65534 без capabilities и с `no_new_privs`, окружение очищено. Корень собран только из нужного на чтение. Решение лежит в `/task` на чтение, запись разрешена только в tmpfs `/tmp` на 8 МБ. Корень самого контейнера тоже только на чтение.
- **Лимиты на тест:** 3 с по часам, RLIMIT_CPU, 256 МБ в cgroup v2 без свопа, 32 процесса и потока, 1 CPU, файлы и вывод до 8 МБ, 64 дескриптора.
- **Лимиты на запрос:** код до 20 КБ, до 50 тестов. Одновременно 4 прогона, очередь до 32 запросов, дальше 503.
- **seccomp.** Процесс убивается на ptrace, mount, unshare/setns, bpf, perf_event_open, userfaultfd, keyctl, загрузке модулей ядра, kexec, clone с новыми пространствами имён и др. На `socket` возвращается EPERM.
- **Утечки ответов.** Ожидаемые ответы внутрь nsjail не попадают: решение получает только аргументы, сравнение идёт снаружи. Журнал nsjail пишется в отдельный файл, поэтому подделать «сбой песочницы» через stderr нельзя.
- **Обход оценки.** Отклонённое песочницей решение (4xx) засчитывается как неверный ответ, а не «пропуск». Нарочно неприемлемым кодом задачу не обойти.
- **Проверено тестами против настоящего nsjail** (CI job `sandbox`): сеть наружу и к соседним сервисам, fork-бомба, бесконечный цикл, 2 ГБ памяти, запись вне `/tmp`, чтение `/etc/passwd` и секретов из окружения, `os.system` и `child_process`, ptrace/unshare/mount через ctypes, подделка строки результата.

Ограничения песочницы:

- nsjail работает в privileged-контейнере: без этого в Docker нет пространств имён и cgroup.
- Ядро общее с хостом, поэтому уязвимость ядра даёт выход наружу. seccomp и непривилегированный uid сужают поверхность атаки, но виртуализацию не заменяют.
- Политика seccomp — запретительный список поверх сброса capabilities, а не белый список.
- При боевой нагрузке песочницу следует вынести на отдельную ВМ или gVisor/Firecracker (`table.md`,).

9.8. Демо-режим — только явно

`DEMO=1` нужен только стенду жюри. По умолчанию (`DEMO=0` в `config.py`, `docker-compose.yml` и `.env.example`) режим выключен. Только в демо:

- код входа возвращается прямо в ответе API, а не письмом. Вне демо без SMTP вход по почте отвечает 503;
- работает вход за демо-персонажей (`POST /api/auth/demo/{role}`) с общим демо-токеном. Вне демо эндпоинт отвечает 404, а уже выданный демо-токен отвергается;
- включён мок-провайдер ФСП ID `/mock-fsp-id`, если не задан боевой `FSP_ID_ISSUER`. Рядом с боевым issuer мок выключен всегда, чтобы не принять боевой секрет клиента;
- пересчёт валидации на `/v` открыт без входа (вне демо — только организатору);
- включён демо-приёмник ATS `/mock-ats` (`ATS_MOCK`, по умолчанию вместе с `DEMO`): страница опрашивает ленту событий по ключу API, который вводит пользователь; ключ хранится только в `sessionStorage` вкладки;
- отзыв о компании доступен сразу после принятия, а не через 7 дней.

Письма демо-персонажам не уходят: их адреса не настоящие. Все данные стенда синтетические, реальные ПДн в репозиторий, сид и тесты не попадают.

9.9. Известные ограничения

Перечислены осознанные упрощения MVP из `table.md` (➖ — отложено с причиной, ⬜ — план). Это продакшен-доводка, к сдаче она не требуется.

Ограничение	Статус	Как закрывается
Токен — Bearer в `localStorage`, а не HttpOnly-cookie; полного CSP нет (только запрет фреймов)	➖	cookie-сессии + CSRF + CSP с хешем inline-скрипта. Сейчас XSS не найдено, все строки экранируются
Ограничение частоты: код входа — только по адресу (пауза 60 с, 5 попыток); по IP и устройству лимита нет; нет лимита на `POST /api/auth/fsp/start`	➖ / ⬜	rate limit на reverse proxy перед публичным стендом
Тело запроса без `Content-Length` (chunked) не ограничивается приложением	➖	`client_max_body_size` на прокси
HSTS не выставляется приложением	—	на прокси (Traefik в Dokploy)
Маскировка ПДн перед LLM — правила, не NER	➖	NER (Natasha) в `mask_pii` для облачной модели
Песочница — privileged-контейнер, ядро общее с хостом; seccomp — запретительный список	⬜	отдельная ВМ или gVisor/Firecracker; белый список по strace
Связь ИНН с юрлицом не проверяется (только контрольная сумма и домен почты)	⬜	проверка по реестру ЕГРЮЛ
Пересчёт валидации на демо-стенде открыт всем (один прогон за раз)	➖	только для стенда жюри; на публичном — лимит на прокси или вернуть пересчёт организатору
Автопроверка задания работодателя: новый аккаунт даёт новую попытку, варианты `choice`/`number` перебираются	⬜	ограничение по почте или ФСП ID, ротация вариантов
SSE чата: нет лимита потоков на пользователя и перепроверки токена на открытом потоке	➖	поток передаёт только сигнал «есть новое» без данных
ФСП ID: claim `fsp_achievements` считается доверенным	⬜	в Keycloak — protocol mapper только из атрибута, который правит реестр
Несовершеннолетние (15–17 лет): согласие законного представителя	➖	возраст в профиле и согласие представителя — при поддержке
Регулярное выражение для почты, а не RFC 5322	➖	адрес всё равно подтверждается кодом

Проверки безопасности закреплены тестами: `backend/tests/test_privacy.py`, `test_auth.py`, `test_fsp_id.py`, `test_chat.py`, `test_vacancies_tasks.py`, `test_code.py`, тесты песочницы `sandbox/tests/`. Тестовые наборы бэкенда и e2e описаны в разделе 8.

10. ML и LLM в решении

ТЗ (п. 3.2) называет преимуществом машинное обучение и обработку естественного языка для разбора резюме, категоризации и подбора. Постановщики в письменных ответах попросили описать, какие модели используются, где и зачем. Коротко:

- **категоризацию делает статистическая модель, а не LLM.** Грейд и категорию задаёт адаптивный тест на модели IRT 2PL (см. раздел 2). Языковая модель на грейд не влияет ни в одном месте;
- **LLM необязательна и помогает в четырёх местах:** разбор текста потребности, разбор резюме, эмбеддинги для поиска кандидатов и реранк топ-30 подборки. Отдельно от LLM есть OCR сканов резюме vision-моделью;
- **у каждого вызова есть фолбэк на правила.** По умолчанию (`LLM_PROVIDER=none`, так поднят и стенд жюри) всё работает на правилах и справочниках, без GPU и ключей;
- **персональные данные в модель не уходят.** Свободный текст перед вызовом проходит маскировку `parsing.mask_pii`, подбор отправляет только структурные поля без ФИО.

Провайдеры и модели

Все провайдеры подключаются через один OpenAI-совместимый клиент `backend/app/services/llm.py` (эндпоинты `/chat/completions` и `/embeddings`). Провайдер выбирается переменной `LLM_PROVIDER`. Модели по умолчанию заданы пресетами в `backend/app/config.py` и переопределяются переменными `LLM_MODEL` и `EMBED_MODEL`.

`LLM_PROVIDER`	Что нужно	Модель чата / эмбеддингов по умолчанию	Где обрабатываются данные
`none` (по умолчанию, в compose тоже)	—	—	модель не вызывается, работают правила
`ollama`	`ollama serve`, `ollama pull mistral:7b bge-m3`; адрес — `OLLAMA_URL` (в compose `http://host.docker.internal:11434/v1`)	`mistral:7b` / `bge-m3`	локально, данные не покидают инфраструктуру
`openai_compatible`	`LLM_BASE_URL` (vLLM и т. п.), при необходимости `LLM_API_KEY`, `LLM_MODEL`, `EMBED_MODEL`	задаются явно	свой шлюз
`yandexgpt`	Yandex AI Studio: `LLM_API_KEY` (API-ключ сервисного аккаунта с ролью `ai.languageModels.user`), `YANDEX_FOLDER_ID`	`yandexgpt-lite/latest` / `text-search-doc/latest`	облако Yandex; логирование запросов для дообучения выключено заголовком `x-data-logging-enabled: false`

LLM_PROVIDER	Что нужно	Модель чата / эмбеддингов по умолчанию	Где обрабатываются данные
`gigachat`	`GIGACHAT_AUTH_KEY`, `GIGACHAT_SCOPE` (по умолчанию `GIGACHAT_API_PERS`); OAuth-токен клиент получает и обновляет сам; `LLM_CA_BUNDLE` — сертификат «Russian Trusted Root CA» добавляется к системным корням, проверка TLS не отключается	`GigaChat` / `Embeddings`	облако Сбера

Почему так. Mistral 7B и bge-m3 — открытые модели, которые запускаются локально через Ollama: данные кандидатов не покидают нашу инфраструктуру. bge-m3 — многоязычная модель эмбеддингов, ей подходят русские и английские названия навыков. YandexGPT и GigaChat — российские облачные модели, им не нужен GPU на стенде. Постановщики разрешили облачные модели, включая зарубежные, с предпочтением открытых (письменные ответы постановщиков). Поэтому набор провайдеров — наш выбор, а не требование, и `openai_compatible` подключает любой совместимый шлюз.

`EMBED_MODEL=none` выключает эмбеддинги, тогда кандидаты ищутся по навыкам. `LLM_TIMEOUT` (по умолчанию 60 с) — таймаут запроса.

OCR сканов резюме

OCR не зависит от `LLM_PROVIDER` и по умолчанию выключен (`backend/app/services/ocr.py`). Он нужен для PDF без текстового слоя (скан). Включается адресом своего vLLM с vision-моделью:

```
vllm serve Qwen/Qwen2.5-VL-7B-Instruct --max-model-len 8192     # GPU ~24 ГБ
OCR_URL=http://<хост>:8000/v1 docker compose up -d
```

Переменные: `OCR_MODEL` (по умолчанию `Qwen/Qwen2.5-VL-7B-Instruct`), `OCR_API_KEY` (если vLLM запущен с `--api-key`) и `OCR_TIMEOUT` — общий срок на документ, 120 с. Из PDF берутся встроенные картинки страниц (`page.images` из pypdf, страница не рендерится): не больше 4 страниц и 40 млн пикселей на картинку. Каждая картинка уменьшается до 1600 px по длинной стороне и уходит отдельным запросом `/chat/completions` с `image_url` в base64. Распознанный текст разбирается как обычное резюме (`parse_cv`, перед LLM — `mask_pii`).

152-ФЗ: `OCR_URL` — только self-hosted контур. Картинку замаскировать нельзя: ФИО, контакты и фото уходят в модель как есть. Поэтому OCR подключается только к своему серверу, не к облачному API. В логи пишутся только тип ошибки и код ответа, текст скана — нет.

Где вызывается модель



Функция	Где в коде	Модель	Вход	Выход	Фолбэк
Разбор потребности работодателя	`parsing.parse_need`; `POST /api/employer/needs/parse`, план теста по тексту вакансии `POST /api/quest/plan`	`LLM_MODEL`	текст потребности после `_strip_personal` и `mask_pii`	JSON: специализация, трек, грейд, обязательные и желательные навыки, вилка, формат, город	правила `_rules_need`. Поле LLM берётся, только если проходит проверку по справочнику `catalog` (кроме города: свободный текст до 64 символов, если правила город не нашли); суммы всегда из регулярных выражений, если правила нашли хоть одну
Разбор резюме	`parsing.parse_cv`; `POST /api/candidate/cv`	`LLM_MODEL`	первые 9000 символов резюме после `mask_pii`, обрезанные до 8000	JSON: навыки, стаж, специализация, трек, роли, софт-скиллы	правила: навыки, роли и софт-скиллы объединяются с найденными правилами (только из справочников); контакты, места работы и подсказка грейда — всегда правилами
Эмбеддинги для поиска	`matching.embed_need` (при сохранении потребности), `matching.embed_candidate` (при сохранении профиля)	`EMBED_MODEL`	строка из структурных полей: «специализация · трек · грейд. Навыки: …»	вектор в `Need.embedding` / `Candidate.embedding`	отбор по косинусу векторов навыков из справочника. Включается сам, если вектора нет у потребности или у кого-то в пуле либо размерность не совпала (сменили модель)

Функция	Где в коде	Модель	Вход	Выход	Фолбэк
Реранк топ-30 подборки	`matching._llm_scores`	`LLM_MODEL`	структура потребности (специализация, трек, грейд, навыки — без текста) и 30 анонимных профилей: публичный id, грейд, трек, θ, подтверждённые и только заявленные навыки	оценка 0–1 на кандидата; подмешивается к структурному соответствию с весом 0,3 (`LLM_BLEND`), штрафы за грейд и трек сохраняются	структурное соответствие без LLM. Ответ кэшируется в памяти процесса, сбой не кэшируется
OCR скана резюме	`ocr.recognize`; `POST /api/candidate/cv`, только если в PDF нет текста	`OCR_MODEL` на своём vLLM	картинки страниц (без маскировки — только self-hosted)	текст страниц → дальше `parse_cv`	422 «Не удалось распознать скан резюме… заполните поля вручную»

Отбор в подборе берёт топ-200 кандидатов пула по близости (`RETRIEVE_K`). Структурный реранк сортирует их по соответствию, и только первые 30 (`RERANK_K`) уходят в LLM. Реранк вызывается, только если в потребности есть навыки. Подробно конвейер описан в разделе 3.

Что разбор отдаёт пользователю. Ответ разбора содержит поле `parsed_by` (`rules` или `llm`): работодатель и кандидат видят распознанную структуру в форме и правят её до сохранения. Автоматически ничего не применяется.

Где модели нет

- **Грейд и категория.** Только адаптивный тест (IRT 2PL, EAP-оценка θ ± SE, см. раздел 2). Резюме и LLM дают лишь подсказку грейда для предвыбора в форме (`parsing.grade_hint`, правила). Решает тест.
- **Достижения ФСП, Codeforces, CTFtime** — импорт и правила оценки (`integrations/`, раздел 5).
- **Автопроверка заданий работодателя** (`services/autocheck.py`) — правила: текст, число с допуском, выбор вариантов, шаблон.
- **Валидация.** Эталон релевантности считается правилами, а на время прогона LLM-реранк и эмбеддинги выключены. Метрики детерминированы и меряют структурный подбор (раздел 4).
- **Публичный предпросмотр плана теста** `/plan/preview` (только `DEMO=1`) — без LLM.
- **Лотерея, чат, регуляторы рынка** — без моделей.

NLP без LLM

Режим по умолчанию — это тоже обработка естественного языка, только на правилах (`backend/app/services/parsing.py`):

- справочник навыков `catalog.SKILLS`: 51 навык и 117 синонимов и написаний («Postgres», «k8s», русские варианты); навык ищется с учётом границ слов на кириллице и латинице;
- распознавание специализации, трека и грейда по маркерам («бэкендер», «сеньор», «тимлид»). Упоминания о других людях («менторил джунов») не считаются грейдом кандидата;

- вилка из текста («до 300 на руки», «250–320 тыс», «от 350 до 450к») с дозаполнением по рынку `catalog.market`;
- обязательные и желательные навыки по словам-маркерам («хорошо бы», «плюсом», «желательно»);
- места работы по периодам («03.2019 – н. в.») без учёбы, роли и софт-скиллы по справочникам `catalog.ROLES` и `catalog.SOFT_SKILLS`;
- `_strip_personal` вырезает до разбора признаки, которые подбору запрещены: возраст, пол, вуз, фото, национальность. Работодатель видит подсказку «Персональные признаки не используются в подборе».

Фолбэк: модель никогда не обязательна

Контракт `llm.py`: любая функция возвращает `None`, если модель выключена, недоступна, не ответила за `LLM_TIMEOUT`, вернула не JSON или JSON не той формы. Вызывающий код обязан иметь фолбэк на правила, и так устроены все четыре вызова из таблицы. После сетевого сбоя или ошибки HTTP модель не вызывается минуту (`_down_until`), чтобы запросы не копили таймауты. В лог пишется только тип ошибки и код ответа: в тексте ошибки бывает адрес. Для GigaChat при ответе 401 токен обновляется один раз. Запрос токена идёт под общей блокировкой: при истечении токена потоки ждут один запрос OAuth, а не шлют десятки.

Ответ модели не принимается на веру. Специализация, трек, грейд, формат, навыки, роли и софт-скиллы принимаются, только если они есть в справочниках `catalog`. Стаж — только в пределах 0–50 лет, зарплата — в допустимом диапазоне. Всё, что не прошло проверку, добирают правила.

Персональные данные и модель

- **Свободный текст маскируется.** Перед каждым вызовом с текстом (`parse_cv`, `parse_need`) работает `parsing.mask_pii`. ФИО (имя из шапки, подписи «ФИО:», отчества, инициалы, «Фамилия Имя» по окончаниям, частые имена и все повторы этих слов), почта, телефон, Telegram, ссылки на профили, подписанные контакты, строки с адресом, датой рождения и гражданством заменяются метками вроде `[имя]`, `[почта]`, `[личное]`. Контакты для формы кандидата распознаются правилами локально, до маскировки, и в модель не уходят. Контакты HR в тексте потребности тоже маскируются.
- **Подбор отправляет только структуру.** В реранк и эмбеддинги не идут ни ФИО, ни текст резюме, ни текст потребности (в нём бывают «пол/возраст»), только специализация, трек, грейд, θ и навыки. ФИО, пол, возраст и вуз в ранжировании не участвуют вообще.
- **OCR — только свой сервер** (см. выше).
- **Тесты.** `test_cv_pii_not_sent_to_model` и `test_need_pii_not_sent_to_model` (`backend/tests/test_llm_parsing.py`) проверяют, что в исходящем запросе к модели нет ни одного ПДн кандидата и контактов HR. `test_scan_text_masked_before_llm` (`backend/tests/test_flows.py`) проверяет то же для распознанного скана.

Ограничение: маскировка сделана на правилах, а не на NER. Редкое имя не из списка, без фамилии и отчества вне шапки, или фамилия без «фамильного» окончания вне шапки пройдут. Для облачной модели в боевом режиме план — подключить NER (Natasha) в `mask_pii` (`table.md`, ➖ «Маскировка ПДн — правила, не NER»).

LLM-читер как инструмент валидации

LLM учитывается и как угроза: кандидат может проходить тест с подсказками модели. Валидация моделирует такого «читера» (`validation.cheater`). Реальная модель при этом не запускается, вероятности верного ответа по типам заданий заданы руками. Итог (README):

Тип заданий	Грейд
Вопросы с вариантами	Senior
Ревью кода	Intern
Задачи с контекстом	Intern
Задачи на код (песочница)	Junior
Смешанный адаптивный тест	Junior

Поэтому банк заданий построен на ревью кода с тонкими багами и задачах с логами и графиками, где LLM помогает мало. Проверка на несогласованность ответов ловит 30 % читеров при 2 % ложных тревог. Подробности и ограничения модели читера — в разделе 4, механика защиты — в разделе 2. План — заменить заданные руками вероятности долями верных ответов реальной модели на нашем банке.

Не реализовано (концепция): асинхронный «разбор решения» — 3–4 уточняющих вопроса от LLM по попытке с флагом `review_required`. Сейчас это только флаг (`# ponytail` в `routers/quest.py`).

Как включить и проверить

Локальная модель:

```
ollama serve
ollama pull mistral:7b bge-m3
LLM_PROVIDER=ollama docker compose up -d --build
```

Облачная модель, на примере GigaChat (файл сертификата смонтируйте в контейнер):

```
LLM_PROVIDER=gigachat GIGACHAT_AUTH_KEY=… LLM_CA_BUNDLE=/certs/russian_trusted_root_ca.pem docker
compose up -d
```

Все переменные с пояснениями собраны в `.env.example`. Ключи берутся только из окружения. Векторы считаются при сохранении профиля и потребности. Засеянная синтетика векторов не имеет, и подбор по ней идёт по навыкам. После пересдачи теста вектор кандидата не пересчитывается (`table.md`, ━).

В CI модели нет: все провайдеры замоканы (`backend/tests/test_llm_parsing.py`). Тесты покрывают формат запросов YandexGPT и GigaChat, обновление токена, сертификаты, фолбэк при сбое и мусоре, проверку ответа по справочнику и отсутствие ПДн в запросе. Живой смоук запускается только вручную:

```
cd backend && LIVE_LLM=1 LLM_PROVIDER=ollama uv run pytest -q -s tests/test_llm_live.py
cd backend && LIVE_LLM=1 OCR_URL=http://localhost:8000/v1 uv run pytest -q -s
tests/test_ocr_live.py
```

Замер смоука на `mistral:7b` (Ollama, 07.10.2026, Apple M2 Pro с Metal; `table.md`):

Что	Результат
Валидный JSON	10/10 разборов, 30/30 оценок реранка
`parse_need`	6,4 с в среднем (макс. 15,7)
`parse_cv`	4,3 с в среднем (макс. 7,5)
Реранк топ-30	37 с
Поля проходят проверку по справочнику	25 из 35 (чаще всего мимо — трек: модель пишет «Frontend» вместо «React», недостающее добирают правила)
Только CPU	60–160 с на разбор, под нагрузкой больше 10 мин: дольше `LLM_TIMEOUT`, срабатывают правила

Поэтому на стенде жюри LLM выключена. 7B-модель на CPU отвечает минутами, для интерактивного разбора это неприемлемо, а реранк топ-30 на CPU не укладывается в таймаут. Облачные YandexGPT и GigaChat работают без GPU, но требуют ключей.

Ограничения

- YandexGPT и GigaChat вживую не проверены: ключей у команды нет. Формат запросов сверен с документацией и замокан в тестах (`table.md`,).
- Живой смоук OCR на Qwen2.5-VL-7B не прогнан: на стенде нет GPU (`table.md`,).
- OCR распознаёт только встроенные растровые картинки страниц. Текст в кривых, страница из нескольких тайлов, JBIG2 и страницы после 4-й не распознаются: такой PDF получает 422 и предложение заполнить поля вручную (`table.md`, —).
- Вклад LLM-реранка и эмбеддингов в качество выдачи не измерен: валидация идёт без модели (раздел 4).
- Маскировка ПДн сделана правилами, а не NER (см. выше).

11. Дополнительный функционал за рамками MVP: защита от недобросовестных работодателей и интеграция с ATS

ТЗ выносит обе темы за рамки MVP и ждёт по ним концепцию и архитектурный подход, а не рабочую реализацию («Будет плюсом», критерий оценки № 6). В разделе 2.2 ТЗ сказано, что верификация компании и модерация вакансий в MVP не требуются.

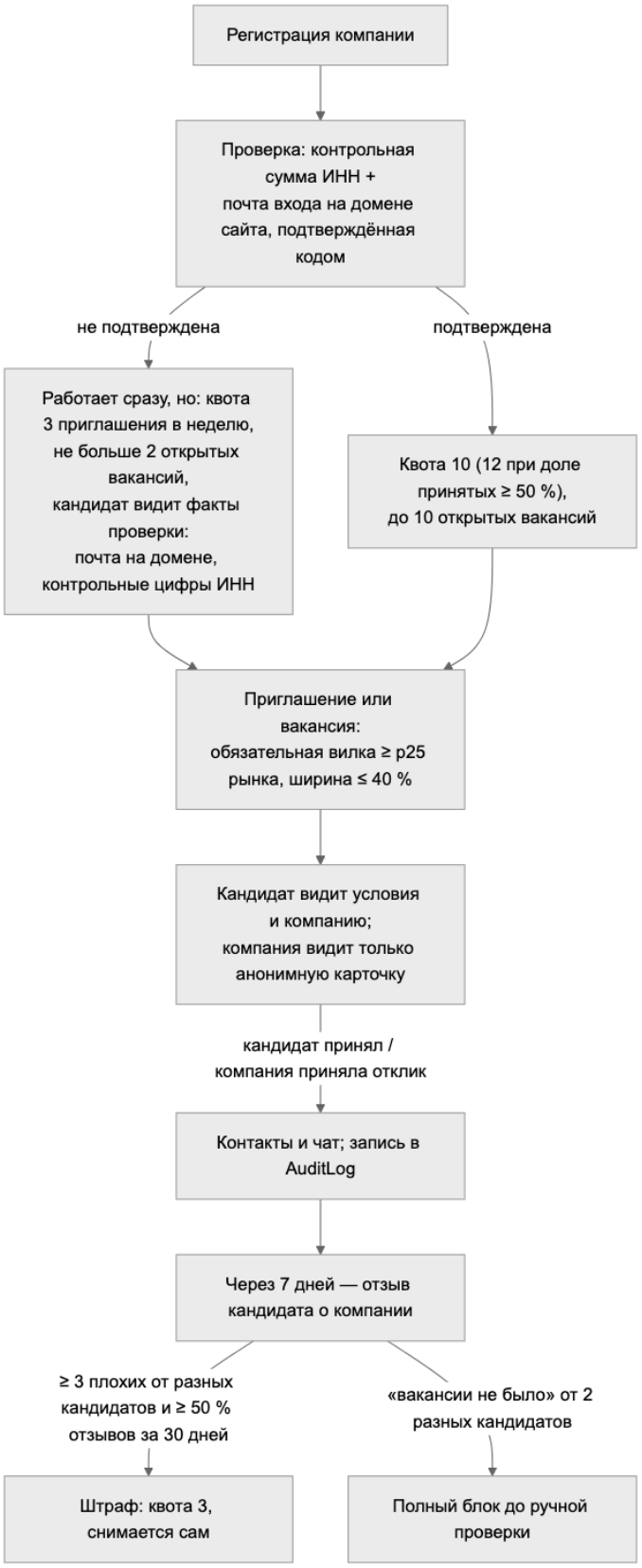
Мы пошли дальше концепции там, где без защиты ломается сама механика обратного найма. Если контакты кандидата раскрываются по приглашению или отклику, фиктивная компания может использовать платформу как сборщик контактов. Поэтому часть защиты уже реализована и покрыта тестами (11.1). Следующие шаги описаны как концепция (11.2). Интеграция с ATS (Applicant Tracking System — система учёта кандидатов и найма у работодателя) реализована частично: ключи API, входящий API и лента событий работают, вебхуки и готовые адаптеры остаются концепцией (11.3).

11.1. Защита от фиктивных вакансий и недобросовестных работодателей: что уже работает

Модель угроз

Угроза	Что хочет недобросовестная компания
Сбор контактов	Разослать приглашения или опубликовать вакансию, получить ФИО, почты и Telegram кандидатов и использовать их вне платформы: спам, перепродажа баз, «курсы»
Фиктивное приглашение	Заманить кандидата условиями, которых нет: вилка-приманка или «вакансии не было» — просят денег или документы
Массовая рассылка	Зайти ко всем подряд и забить внимание кандидатов
Подмена личности	Выдать себя за известную компанию

Слои защиты



1. Проверка компании — `routers/employer.py`, функции `inn_valid`, `mail_confirmed`, `refresh_verified`.

- ИНН проверяется по контрольным цифрам (10 знаков — юрлицо, 12 — ИП). ИНН уникален: вторая компания с тем же ИНН получает 409. После подтверждения или начала работы ИНН нельзя сменить (409).

- Почта входа должна стоять на домене сайта компании и быть подтверждена нашим кодом. Почте из ФСП ID, Keycloak и сторонних провайдеров для этой проверки мы не доверяем, даже если у провайдера включён «Trust Email» (раздел 5).
- Компания с обеими галочками получает внутренний флаг `verified`: полную квоту и лимит вакансий.
- Кандидату показываются отдельные факты, а не общее «компания проверена»: «почта на домене подтверждена / не подтверждена» и «ИНН: контрольные цифры не сходятся» (`employer.company_trust`). Связь ИНН с юрлицом без реестра не проверить, поэтому обещать «проверена» было бы нечестно.
- Проверка по ЕГРЮЛ в кабинете компании показана как невыполненная (заглушка `egrul` в `employer._checks`).

2. Неподтверждённая компания работает, но ограниченно. Так ТЗ не требует верификации, а злоупотребление дорого:

- 3 приглашения в неделю вместо 10 (`catalog.UNVERIFIED_QUOTA`), без бонуса;
- не больше 2 открытых вакансий вместо 10 (`VACANCY_LIMIT_UNVERIFIED`);
- у кандидата — факты проверки: «почта на домене не подтверждена» и/или «ИНН: контрольные цифры не сходятся».

3. Квота работает и как поощрение. Базовая квота — 10 приглашений в неделю, при доле принятых приглашений ≥ 50 % — 12 (`employer.quota`). Компания, чьи приглашения массово отклоняют или игнорируют, бонуса не получает.

4. Обязательная вилка. Вилка «от–до» обязательна и в приглашении, и в вакансии (`employer.validate_fork`). Нижняя граница не ниже 25-го перцентиля рынка, ширина не больше 40 %, любая сумма не больше 10 млн ₽ (см. раздел 3 документа, «Выход на контакт»). Широкая вилка-приманка «от 50 до 500 тысяч» не пройдёт. Кандидат видит условия до начала общения и может сверить их с тем, что ему предложат потом.

5. Анонимность до согласия кандидата.

- По приглашению работодатель видит анонимную карточку без ФИО, почты, Telegram и резюме. Контакты раскрываются только после принятия, раскрытие пишется в `AuditLog` (`contacts_revealed`).
- По отклику на вакансию — то же самое. Контакты открываются, только когда компания приняла отклик. До этого в заметке кандидата замаскированы ФИО, почта, телефон, Telegram и ссылки (`vacancies._note`, `parsing.mask_contacts`), чтобы фейк не выманил их через «напишите свой Telegram в заметке». Просмотр и отказ контактов не открывают (`vacancies._shown`).
- Чат открывается по тем же правилам: принятое приглашение или принятый отклик (`chat.peers_linked`, `chat.may_talk`). Отклик без ответа чат не открывает, иначе фиктивная вакансия выспросила бы контакты в переписке.
- Кандидат может скрыть профиль от конкретной компании (`PATCH /api/candidate/hidden-from`). После этого компания не видит его в выдаче, не может пригласить, теряет доступ к контактам и чату.

6. Срок жизни и лимит вакансий (`routers/vacancies.py`).

- Вакансия живёт 30 дней (`VACANCY_TTL_DAYS`). Истёкшая скрыта и не принимает отклики. Продлить её компания может только явно (`POST /api/vacancies/{id}/extend`), поэтому «мёртвые» вакансии не висят месяцами.

- Открытых вакансий не больше 2 у неподтверждённой компании и 10 у подтверждённой. Сверх лимита — 409 при публикации и при продлении истёкшей.

7. «Отвечает на N % откликов». Кандидат видит долю откликов за 90 дней, на которые компания ответила — приняла или отклонила (`vacancies.response_rates`). Доля показывается от 5 откликов. Вакансия, которая только собирает отклики, видна по низкой доле.

8. Отзывы кандидатов о компании (`services/reputation.py`).

Кто и когда может оставить отзыв:

- только кандидат с принятым приглашением, поэтому конкурент не может «накидать» отзывов, не получив приглашения;
- через 7 дней после принятия (`COMPANY_FEEDBACK_AFTER_DAYS`), чтобы у компании было время выйти на связь. На стенде с `DEMO=1` — сразу;
- по желанию и один раз (`POST /api/candidate/invites/{id}/company-feedback`).

Варианты отзыва: «Всё честно, условия совпали», «Не вышли на связь», «Условия не совпали с приглашением», «Вакансии не было: просили денег или данные».

Последствия:

- **Штраф.** Если за скользящие 30 дней пришло не меньше 3 плохих отзывов от разных кандидатов и они составляют не меньше половины всех отзывов окна, квота падает до 3 в неделю без бонуса (`FEEDBACK_BAD_MIN`, `FEEDBACK_BAD_SHARE`, `FEEDBACK_PENALTY_QUOTA`). Окно скользящее, поэтому штраф снимается сам. Доля нужна, чтобы три недовольных отзыва не топили крупную компанию.
- **Блок.** Если «вакансии не было» написали 2 разных кандидата (`FRAUD_BLOCK_MIN`), компания блокируется до ручной проверки:
  - приглашения, вакансии, задания, лотерея и чат закрыты — зависимость `deps.active_company` отвечает 403 с причиной;
  - вакансии и задания компании скрыты;
  - её открытые приглашения истекают, и кандидат видит нейтральное «истекло»;
  - контакты уже принявших кандидатов скрываются;
  - в журнал пишется `company_blocked`.
- **Снятие блока.** Блок снимает только человек: `python -m app.services.reputation blocked` показывает заблокированных, `… unblock <ИНН>` снимает блок и пишет `company_unblocked`. Отзывы, оставленные до проверки, блок повторно не вызывают.
- **Что видят стороны.** Кандидату репутация компании не показывается: отзывы меняют только квоту и блок. Работодатель видит сводку без авторов, и только от 3 отзывов (`FEEDBACK_SUMMARY_MIN`), чтобы по одному отзыву нельзя было вычислить автора.

Тесты: `tests/test_reputation.py` (отзывы, штраф, блок, снятие), `tests/test_vacancies_tasks.py` (анонимность отклика до принятия, срок и лимит вакансий, доля ответов), `tests/test_privacy.py` (обход всех ответов работодателю до и после принятия), `tests/test_chat.py`, `tests/test_flows.py` (неподтверждённая компания, квота, ёмкость).

Осознанные упрощения

Из `table.md`, со статусом ➖:

- **Модерация через CLI.** Блоки снимаются командой в консоли, а не в кабинете модератора. Роль модератора, очередь и переписка с компанией появятся вместе с поддержкой платформы.

- **Демо-стенд можно заблокировать за минуты.** В `DEMO=1` код входа приходит в ответе, а отзыв открыт сразу, поэтому двух новых кандидатов хватает, чтобы заблокировать демо-компанию. Так задумано: стенд показывает механику. Блок снимается командой `unblock 7704123450`.
- **Флаг `verified` ставится без проверки связи ИНН с юрлицом.** Это закрывается реестром ЕГРЮЛ (см. ниже).

11.2. Защита: концепция развития

Ниже — открытые пункты `table.md` (— запланировано, — — отложено) и предложения команды. В коде их нет.

Шаг	Что сделать	Почему
Проверка по ЕГРЮЛ ( )	Запрос статуса и данных организации по ИНН через API ФНС или коммерческий агрегатор. Проверка: организация действующая, название совпадает, домен сайта связан с юрлицом. Результат — новый факт для кандидата («организация действующая по ЕГРЮЛ») и условие для `verified`	Сейчас ИНН с верной контрольной суммой можно взять у чужой компании. Почта на домене подтверждает только доступ к домену
Вход компании через корпоративный SSO ( )	Брокер Keycloak уже поддерживает сторонних провайдеров (`kc_idp_hint`, раздел 5). Добавить флаг доверия провайдеру в `FSP_ID_IDPS` и проверку claim `identity_provider`: корпоративный SSO — только для компаний	Вход через корпоративного провайдера подтверждает принадлежность к организации сильнее, чем почта на домене
Жалобы кандидатов на вакансию ( )	Жалоба по отклику (один раз) идёт в те же штраф и блок, что отзывы по приглашениям (`reputation.py`)	Сейчас отзыв есть только по приглашению. Фиктивная вакансия может принять все отклики разом и получить контакты
Порог массовых принятий откликов ( )	Больше N принятых откликов за сутки — предупреждение и ручная проверка	Принятие отклика ничего не стоит, а у фейка ещё и поднимает «отвечает на N %»
Вилка выше 90-го перцентиля ( )	В `catalog.market` добавить p90. Вилка выше p90 — предупреждение работодателю и плашка «вилка заметно выше рынка» кандидату	Завышенная вилка — типичная приманка фиктивных вакансий
Кабинет модератора (—)	Роль модератора, очередь блоков и жалоб, переписка с компанией, история решений в `AuditLog`	Замена CLI, когда появится команда поддержки
Отзывы в демо-режиме (—)	В `DEMO=1` не учитывать отзывы по приглашениям демо-компаний	Чтобы демо-компанию на общем стенде нельзя было заблокировать
Предложение: проверка текста приглашений и вакансий	Правила (без LLM по умолчанию) на признаки мошенничества: предоплата, «обучение за свой счёт», просьба паспортных данных до собеседования. Срабатывание — предупреждение кандидату и пометка для модератора, не автоблок	Ловит «вакансии не было» до того, как пострадают двое кандидатов

Шаг	Что сделать	Почему
Предложение: учёт ошибочных блоков	Приглашения, истёкшие из-за блока, не считать непринятыми в доле принятых после снятия ошибочного блока (`table.md`, —)	Иначе снятый ошибочный блок ещё неделями режет компании бонус квоты

Принцип развития тот же, что в MVP. Неподтверждённая компания не отрезается, но работает с урезанными лимитами. Кандидату показываются проверяемые факты, а не общий ярлык доверия. Жёсткие меры (блок) включаются только по сигналам нескольких независимых кандидатов и снимаются человеком.

11.3. Интеграция с ATS: ключи API, лента событий и концепция вебхуков

Что понимается под интеграцией. По письменному ответу постановщиков: синхронизация вакансий, откликов и статусов с системой найма работодателя через API, без ручного дублирования. В MVP она не требовалась, но мы реализовали основу: ключи API компании, входящий API `/api/ats/v1`, ленту событий и демо-приёмник Mock ATS. Исходящие вебхуки и готовые адаптеры остаются концепцией.

Ограничение подхода: API конкретных ATS (Хантфлоу, Talantix, Поток и других) мы не изучали. Реализован наш контракт, который адаптер под конкретную систему вызывает и опрашивает. Готовых коннекторов нет.

Что реализовано

- **Ключи API компании.** Работодатель выпускает ключ в кабинете («Компания» → «Интеграция с ATS»): название и области доступа. Ключ вида `fsp_…` показывается один раз, в базе лежит только sha256 (`deps.hash_secret`) и начало ключа для списка. Не больше 10 активных ключей на компанию. Ключ можно отозвать. Выпуск и отзыв пишутся в `AuditLog` (`api_key_issued`, `api_key_revoked`) с идентификаторами. Эндпоинты — `GET/POST /api/employer/api-keys`, `DELETE /api/employer/api-keys/{id}`.
- **Области доступа (scopes).** `vacancies:write` — публиковать, править, закрывать и продлевать вакансии; `applications:read` — читать события по откликам; `applications:write` — менять статус отклика; `invites:read` — читать события по приглашениям; `invites:write` — ставить отметку по принятому приглашению. Нет нужной области — 403.
- **Входящий API** `/api/ats/v1`, заголовок `X-Api-Key`. Это тонкая обёртка над существующей логикой `routers/vacancies.py` и `routers/employer.py`, правила не дублируются:
  - `PUT /vacancies/{external_id}` — создать или заменить вакансию по внешнему id системы работодателя. Вилка проверяется `validate_fork` как обычно (422 с ошибками). Повторный вызов правит ту же вакансию, дублей нет;
  - `POST /vacancies/{external_id}/close` и `/extend` — закрыть и продлить (закрыть можно и заблокированной компании, как в кабинете);
  - `POST /applications/{id}/status` — статус отклика (seen / accepted / declined с причиной из `DECLINE_REASONS`); повтор того же статуса возвращает 200, а недопустимый переход — 409, как в интерфейсе;
  - `POST /invites/{id}/feedback` — отметка confirmed / not_confirmed / hired по принятому приглашению; повтор той же отметки — 200;
  - `GET /events?after=<курсор>&limit=<1..500>` — лента событий; курсор — непрозрачная строка `<txid>.<id>` из поля `next` (для чтения с начала — `0`, неверный формат — 422).

- **Соответствие внешних id.** Таблица `ats_links (company_id, entity, external_id, internal_id)` с уникальностью `(company_id, entity, external_id)`; создание вакансии и запись связи идут в одной транзакции.
- **Outbox и лента событий.** Каждое событие (`application.created`, `application.withdrawn`, `application.status`, `invitation.seen`, `invitation.accepted`, `invitation.declined`, `invitation.expired`, `candidate.access_revoked`) пишется в таблицу `outbox_events` в той же транзакции, что смена статуса: откат транзакции не оставляет события. В записи — только идентификаторы и статусы. Система работодателя забирает ленту опросом по курсору `next` — строке `<txid>.<id>` последнего события (`txid` — id транзакции Postgres, вставившей событие, `id` — номер события); повтор запроса с тем же `after` безопасен. Ключ видит только те типы событий, на чтение которых у него есть область. Id событий выдаются не в порядке коммитов, поэтому лента отдаёт только события завершённых транзакций (`txid` меньше `xmin` текущего снимка) в порядке `(txid, id)`: у любой ещё идущей транзакции xid не меньше `xmin`, её события лягут после курсора, когда она закоммитится, — ATS не теряет событие с меньшим id, закоммиченное позже. Блокировок `emit` не берёт, а `outbox_events.company_id` — без внешнего ключа, чтобы вставка события не блокировала строку компании. Цена: долгая посторонняя транзакция в базе задерживает (но не теряет) выдачу новых событий. Истечение приглашений ленивое, поэтому `GET /events` перед чтением проверяет истёкшие приглашения компании и порождает `invitation.expired`.
- **Mock ATS.** При `ATS_MOCK=1` (по умолчанию включается вместе с `DEMO=1`) страница `/mock-ats` — демо-приёмник: вводится ключ API (хранится только в `sessionStorage`), страница раз в 3 секунды опрашивает ленту, показывает карточки событий и позволяет ответить через API: пригласить, отказать, отметить просмотренным, поставить «вышел на работу», опубликовать вакансию. Это стенд для жюри, а не настоящая ATS; без `ATS_MOCK` маршрут отвечает 404.

Персональные данные: как в кабинете, собирается при чтении

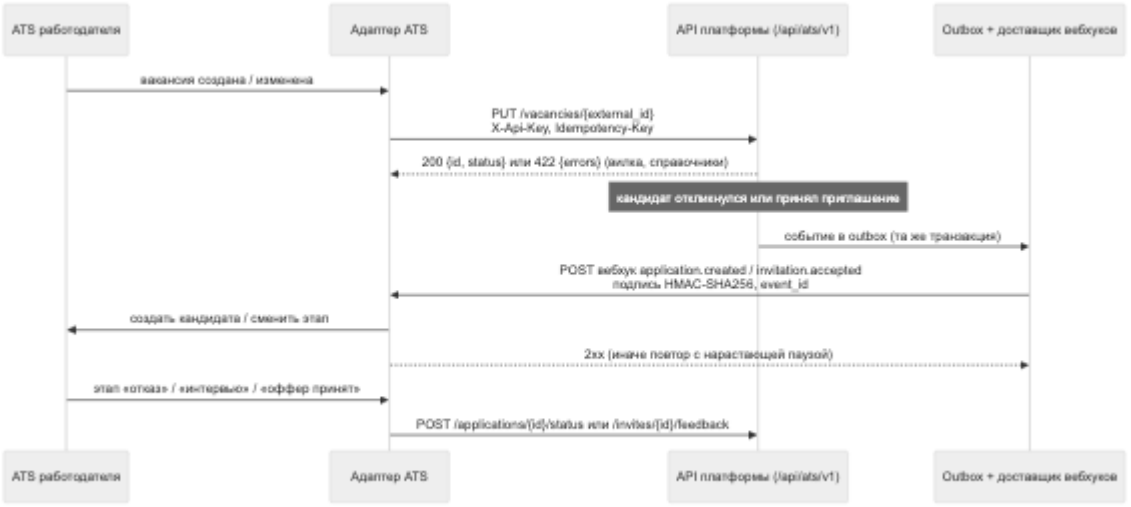
В outbox ПДн не хранятся. Событие обогащается в момент чтения ленты по тем же правилам, по которым данные показал бы кабинет, поэтому через API работодатель видит ровно то, что видит в интерфейсе:

- в событиях по откликам `candidate` — анонимная карточка (`matching.card`: публичный id, категория, θ ± SE, навыки, достижения), пока у кандидата есть согласие на обработку и ИНН компании не в его списке «скрыть от компаний» (отклик — инициатива кандидата, согласие на публикацию не нужно). В событиях по приглашениям `candidate` — только `{public_id}`, как в воронке кабинета; карточку работодатель берёт через `GET /api/employer/candidates/{public_id}` по её правилам видимости (согласие на публикацию, «скрыть от»); удалённое приглашение — `null`;
- ФИО, почта и Telegram (`contacts`) приходят только если отклик сейчас `accepted` и контакты раскрыты (`vacancies._shown`), или приглашение сейчас `accepted` и чат разрешён (`chat.may_talk`). Раскрытие пишется в `AuditLog` (`contacts_revealed`) при принятии; если отклик был принят, пока согласие не действовало, лента сама пишет раскрытие при первой выдаче контактов (`vacancies._reveal`, одна запись на отклик), а повторные выдачи не логируются;
- после отзыва согласия, удаления аккаунта или скрытия профиля от компании кандидат и контакты в событиях становятся `null` (для приглашений — остаётся только `public_id`), даже если событие было создано раньше. Событие `candidate.access_revoked` уходит только тем компаниям, у которых были открытые контакты, и несёт публичный id и причину (`consent`, `deleted`, `hidden`). Дальнейшее хранение данных, уже выгруженных в ATS, — ответственность работодателя как самостоятельного оператора ПДн. Об этом кандидата нужно предупреждать в тексте согласия;

- записывающие запросы заблокированной компании получают 403, как в кабинете (`deps.BLOCKED`). Квота приглашений, правила вилки и переходы статусов действуют те же.

Подборка и ранжирование в ATS массово не выгружаются: это анонимный банк, и выгрузка превратила бы его в базу контактов. Результаты теста уходят только в составе карточки. Чат остаётся на платформе.

Что остаётся концепцией



В схеме реализованы вызовы адаптера в платформу и запись в outbox. Не реализовано то, что находится справа от outbox:

1. Исходящие вебхуки. Отдельный доставщик читает outbox и отправляет событие на адрес компании с повторами и нарастающей паузой. Тело подписывается HMAC-SHA256 общим секретом компании и временем отправки (защита от подмены и повтора), событие несёт `event_id`, и ATS отбрасывает повторы. Пока вместо вебхуков работает опрос ленты.

2. `Idempotency-Key` для изменяющих POST. Повтор с тем же ключом возвращает сохранённый ответ. Сейчас идемпотентны `PUT` по `external_id` и повтор статуса или отметки (200); остальные POST защищены переходами статусов (409).

3. Маппинг этапов в кабинете. У каждой ATS свои этапы воронки, поэтому компания настраивает соответствие. Типовое:

Платформа	Этап ATS (типовой)
Отклик `sent` / `seen`	Новый / Рассмотрение
Отклик `accepted` («Пригласить на интервью»)	Интервью
Отклик `declined` + причина	Отказ (причина из `DECLINE_REASONS`)
Отклик `withdrawn`	Кандидат отказался
Приглашение `accepted`	Новый кандидат (источник «ФСП Карьера»), сразу этап «Контакт»
Приглашение `declined` + причина / `expired`	Отказ кандидата / Нет ответа
Этап «Оффер принят», «Вышел на работу»	отметка `hired` (`POST /api/ats/v1/invites/{id}/feedback`)

Сегодня сопоставление делает адаптер на стороне работодателя.

4. Готовые адаптеры под 1–2 ATS (Хантфлоу и другие), которыми пользуются первые работодатели ФСП. Выбор — вместе с ФСП после хакатона. Потребность `Need` из ATS (заявка на подбор) в API пока тоже не передаётся.

Порядок дальнейшего внедрения

1. Доставщик вебхуков с HMAC-подписью поверх готового outbox.
2. `Idempotency-Key` для изменяющих запросов.
3. Маппинг этапов в кабинете компании.
4. Адаптеры под 1–2 ATS.